

UNIVERSIDAD ESAN

FACULTAD DE INGENIERÍA

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN Y SISTEMAS

Propuesta de modelo para la detección de información falsa del COVID-19 mediante técnicas de Machine Learning y Natural Language Processing

Trabajo de Tesis II

Yair Andrey Salinas Bolaños

Asesor: Wilfredo Mamani Ticona

Lima, 4 de julio del 2022

Dedicatoria

De todo corazón dedico mi tesis a mis padres y hermanos, ya que fueron los principales pilares de mi motivación, siempre estuvieron para mí cuando los necesitaba y me dieron fuerzas para no rendirme.

Agradecimientos

A mis padres por darme su apoyo incondicional desde el principio

A mis hermanos por darme fuerza para no tirar la toalla.

A mi asesor de tesis por orientarme y guiarme a lo largo de la investigación

A mis compañeros de universidad por animarme en los momentos difíciles

A mis profesores quienes se me convirtieron en un ejemplo a seguir

A la licenciada Margarita Arias por ayudarme en la validación del etiquetado

ÍNDICE GENERAL

INTRODUCCIÓN 3

CAPÍTULO 1: PLANTEAMIENTO DEL PROBLEMA 5

1.1 Descripción de la Realidad Problemática 5

1.2 Formulación del Problema 10

1.2.1 Problema Principal 10

1.2.2 Problemas Secundarios 10

1.3 Objetivos de la Investigación 11

1.3.1 Objetivo General 11

1.3.2 Objetivos Específicos 11

1.4 Justificación de la Investigación 11

1.4.1 Justificación Teórica 11

1.4.2 Justificación Práctica 11

1.4.3 Justificación Metodológica 12

1.5 Delimitación del Estudio 12

1.5.1 Espacial 12

1.5.2 Temporal 12

1.5.3 Conceptual 12

CAPÍTULO 2: MARCO TEÓRICO 13

2.1 Antecedentes 13

2.1.1 Resumen de artículos 13

2.1.2 Tabla resumen de artículos 37

2.2 Bases Teóricas 38

2.2.1 Inteligencia Artificial 38

2.2.1.1 Machine Learning 39

2.2.1.1.1 Aprendizaje supervisado 39
2.2.1.1.2 Aprendizaje no supervisado 51
2.2.1.1.3 Aprendizaje por refuerzo 52
2.2.2 Natural Language Processing (NLP) 53
2.2.2.1 Text Mining 54
2.2.2.2 Bag of Word (BOW) 55
2.2.2.3 Tf-idf (Term frequency - Inverse document frequency) 56
2.2.2.4 Word2Vec 57
2.2.2.4.1 Continuous Bag of Words Model (CBOW) 58
2.2.2.4.2 Continuous Skip-Gram Model 59
2.2.2.5 FastText 60
2.2.2.5.1 Generación de sub-palabras 61
2.2.2.5.2 Skip-Gram con muestreo negativo 61
2.2.3 Información falsa 62
2.2.3.1 Contexto en el que surgen 63
2.2.3.2 Tipos 64
2.2.3.3 Categorías 65
2.2.3.4 Formas de detección 66
2.3 Marco Conceptual 67
2.4 Hipótesis 72
2.4.1 Hipótesis General 72
2.4.2 Hipótesis Específicas 73
CAPÍTULO 3: METODOLOGÍA 73
3.1 Diseño, tipo y enfoque de investigación 73
3.1.1 Diseño 73

3.1.2 Tipo 74
3.1.2.1 Nivel 74
3.1.3 Enfoque 74
3.2 Población y muestra 74
3.3 Operacionalización de variables 75
3.4 Técnicas de recolección de datos 77
3.5 Metodología de implementación de la investigación 78
3.5.1 Creación del dataset 79
3.5.2 Preprocesamiento 90
3.5.3 Extracción de características 93
3.5.4 Construcción del modelo 97
3.5.5 Evaluación del modelo 103
3.6 Metodología para la medición de resultados de la implementación 104
3.7 Cronograma de actividades 106
3.8 Presupuesto 107
CAPÍTULO 4: DESARROLLO DE LA SOLUCIÓN 107
4.1 Propuesta de solución 107
4.1.1 Creación del dataset 107
Actividad 1: Identificar fuentes de datos 107
Actividad 2: Desarrollar los algoritmos de scrapping 109
Actividad 3: Homogeneizar los dataset 127
Actividad 4: Etiquetar manualmente los registros 130
Actividad 5: Validar etiquetado con personal del MINSA 133
4.1.2 Preprocesamiento 134
Actividad 1: Eliminar los links, hashtags, menciones y emojis. 134

Actividad 2: Limpiar el contenido 136
4.1.3 Extracción de características 140
Actividad 1: Realizar la extracción de características con BoW 140
Actividad 2: Realizar la extracción de características con Tf-Idf 141
Actividad 3: Realizar la extracción de características con Word2Vect 142
Actividad 4: Realizar la extracción de características con FastText 145
4.1.4 Construcción del modelo 148
Actividad 1: Codificar la clase 148
Actividad 2: Particionar el dataset 148
Actividad 3: Aplicar técnicas de Machine Learning 149
4.1.5 Evaluación del modelo 159
4.2 Medición de la solución 163
CAPÍTULO 5: CONCLUSIONES Y RECOMENDACIONES 165
5.1 Conclusiones 165
5.2 Recomendaciones 168
REFERENCIAS 170
ANEXOS 176

ÍNDICE DE FIGURAS

Figura 1. Rumores, estigma y teorías de conspiración asociadas con COVID-19 y cronograma de informes detectados durante el estudio, 2020. 6

Figura 2. Precisión fáctica de los rumores, el estigma y las teorías de la conspiración relacionadas con las causas de enfermedad, tratamiento y medidas de control del COVID-19..... 7

Figura 3. Caracterización de los países analizados, según las variables de estudio, noviembre del 2020..... 8

Figura 4. Marco propuesto para detección de información engañosa 15

Figura 5. Accuracy, Error Rate, and Area Under Curve de los resultados de la validación..... 16

Figura 6. Combinaciones de parámetros de árboles de decisión 17

Figura 7. Porcentaje de precisión de clasificación promedio: 10 iteraciones de algoritmos independientes..... 18

Figura 8. Diagrama de flujo de la detección de noticias falsas. 20

Figura 9. Comparación de precisión en conjuntos de datos y parámetros experimentales. Los mejores resultados están en negrita..... 21

Figura 10. Métricas de evaluación (Precisión, Exactitud, Recall y Puntaje F1) para todos los clasificadores, considerando solo la selección de características textuales. 23

Figura 11. Marco de Investigación. 25

Figura 12. Matriz de confusión en el equipo de prueba usando BERT. 26

Figura 13. Sistema de anotación. 28

Figura 14. Arquitectura de detección de noticias falsas. 32

Figura 15. Precision (P), recall (R), y F1-score (F1) (conjunto de datos anotados manualmente)32

Figura 16. Precision (P), recall (R), y F1-score (F1) (conjunto de datos anotados automáticamente). 33

Figura 17. Resultados de la métrica Precision. 34

Figura 18. Estadísticas de CoAID..... 35

Figura 19. Desempeño de detección de información errónea 37

Figura 20. Aplicaciones de la inteligencia artificial. 39

Figura 21. Esquema de aprendizaje supervisado. 40

Figura 22. Algoritmo KNN. 41

Figura 23. Rutas de datos al dividir. 42

Figura 24. Los datos linealmente separables se muestran en el cuadro A. Los cuadros B, C y D muestran posibles líneas válidas que separan las dos clases de datos. 44

Figura 25. Cuatro ejemplos de conjuntos de datos que no se pueden separar linealmente. 45

Figura 26. Gráfico de la función escalón 46

Figura 27. Gráfico de la función lineal 47

Figura 28. Gráfico de la función sigmoide 47

Figura 29. Gráfico de la función tangente hiperbólica 48

Figura 30. Modelo neuronal perceptron 48

Figura 31. Clasificación en un hiperplano 49

Figura 32. Modelo neuronal MultiLayer Perceptron 50

Figura 33. Aprendizaje no supervisado 51

Figura 34. Cinco iteraciones de agrupamiento de k-medias. En la gráfica superior izquierda, tres centros iniciales se generan aleatoriamente en el espacio de características (cruces) 52

Figura 35. Diagrama de flujo de aprendizaje por refuerzo 53

Figura 36. N-gramas de Google 55

Figura 37. Bag of Words 56

Figura 38. Representación de la arquitectura de la red neuronal 57

Figura 39. Word2Vec: Las palabras que comúnmente aparecen juntas están más juntas 58

Figura 40. Arquitectura del modelo CBOW. 59

Figura 41. Arquitectura del modelo Skip-gram. 60

Figura 42. 3-grams de la palabra *eating* 61

Figura 43. Skip-gram aplicado a la palabra *eating* 62

Figura 44. Ejemplo de curva de infodemia 63

Figura 45. Tipos de desórdenes de información 65

Figura 46. Funcionamiento del Web Scraping 77

Figura 47. Metodología de la investigación 79

Figura 48. Pseudocódigo del algoritmo de scraping de Facebook 82

Figura 49. Pseudocódigo del algoritmo de extracción de comentarios de los posts de Facebook 83

Figura 50. Pseudocódigo del algoritmo de scraping de Twitter 84

Figura 51. Pseudocódigo del algoritmo de scraping de la página web del MINSA 85

Figura 52. Pseudocódigo del algoritmo de extracción de noticias del html del MINSA 85

Figura 53. Pseudocódigo del algoritmo de scraping de la página web de Verificador 86

Figura 54. Pseudocódigo del algoritmo de extracción de noticias del html de Verificador 86

Figura 55. Pseudocódigo del algoritmo de scraping de la página web de Ojo Público 87

Figura 56. Pseudocódigo del algoritmo de extracción de noticias del html de Ojo Público 88

Figura 57. Validación cruzada de 5 99

Figura 58. Matriz de confusión para un problema de dos clases, con diferentes resultados etiquetados 104

Figura 59. Cronograma de actividades 107

Figura 60. Ejemplo de tweet falso 108

Figura 61. Colaboración entre Global Investigative Journalism Network y Ojo Público 109

Figura 62. Colaboración entre International Fact-Checking Network y Verificador LR 109

Figura 63. Extensión Get cookies.txt 110

Figura 64. Datos guardados del inicio en Facebook con la extensión Get cookies.txt 110

Figura 65. Cuenta de Facebook del MINSA 111

Figura 66. Archivo csv de posts 113

Figura 67. Archivo csv de comentarios 113
Figura 68. Credenciales de Tweepy 114
Figura 69. Archivo csv de tweets 117
Figura 70. Inspección de la web de noticias del MINSA 119
Figura 71. Atributos del div de noticias del MINSA 120
Figura 72. Título de la noticia dentro del atributo “text” 120
Figura 73. Fecha de la noticia dentro del atributo “time” 121
Figura 74. Link de la noticia dentro del atributo “a” 121
Figura 75. Archivo csv noticias del Minsa 122
Figura 76. Botón “VER MÁS” de Verificador 123
Figura 77. Noticia dentro de la clase “boxVerificador” 123
Figura 78. Título y link de la noticia dentro del atributo “a” 124
Figura 79. Archivo csv noticias de Verificador LR 124
Figura 80. Página siguiente con la clase “fas fa-chevron-right” 125
Figura 81. Noticia dentro de la etiqueta “article” 126
Figura 82. Título y link de la noticia de Ojo Público dentro de la etiqueta “a” 126
Figura 83. Fecha de la noticia de Ojo Público dentro de la etiqueta “span” 127
Figura 84. Archivo de noticias de Ojo Público 127
Figura 85. Ejemplo de atributos de un post 128
Figura 86. Atributos de un comentario 128
Figura 87. Ejemplo de atributos de un tweet 129
Figura 88. Lista de keywords 130
Figura 89. Dataset homogeneizado 130
Figura 90. Registro con tendencias religiosas 131
Figura 91. Noticia registrada como “engañoso” 132

Figura 92. Dataset etiquetado 133
Figura 93. Documento de validación del etiquetado 134
Figura 94. Expresión regular para eliminar links, hashtags y menciones 135
Figura 95. Ejemplo de registro original del dataset 135
Figura 96. Ejemplo de registro sin links, hashtags ni menciones 136
Figura 97. Ejemplo de registro sin links, hashtags, menciones ni emojis 136
Figura 98. Registros sin links, hashtags, menciones ni emojis 136
Figura 99. Registro en formato ASCII 137
Figura 100. Registro normalizado 137
Figura 101. Tokens del registro 138
Figura 102. Registro sin signos de puntuación ni números 138
Figura 103. Ejemplos de stop words 139
Figura 104. Registro sin stop words 139
Figura 105. Corpus de palabras preprocesadas 140
Figura 106. Vector de características con BoW 140
Figura 107. Vectorizadores con BoW 141
Figura 108. Vector de características con Tf-Ifd 142
Figura 109. Vectorizadores con Tf-Idf 142
Figura 110. Similaridades del modelo w2v con la palabra “coronavirus” 144
Figura 111. Vector de características con w2v 144
Figura 112. Vectorizadores con w2v 145
Figura 113. Similaridades del modelo fastText con la palabra “coronavirus” 146
Figura 114. Vector de características con fastText 147
Figura 115. Vectorizadores con fastText 147
Figura 116. Dataset numérico con las técnicas *cv*_1, tf_*idf*_1, w2v_1 y fastText_*1* 148

Figura 117. Porción de la data train (*X_train, Y_train*) 149

Figura 118. Grilla de KNN 149

Figura 119. Grilla de Árbol de Decisión 151

Figura 120. Grilla de Naive Bayes 153

Figura 121. Grilla de SVM 154

Figura 122. Grilla de Regresión Logística 156

Figura 123. Grilla de MLP 158

Figura 124. Matriz de confusión del mejor modelo de KNN 160

Figura 125. Matriz de confusión del mejor modelo de Árbol de Decisión 160

Figura 126. Matriz de confusión del mejor modelo de Naive Bayes 161

Figura 127. Matriz de confusión del mejor modelo de SVM 162

Figura 128. Matriz de confusión del mejor modelo de Regresión Logística 162

Figura 129. Matriz de confusión del mejor modelo de MLP 163

ÍNDICE DE TABLAS

Tabla 1. *Resumen de antecedentes de la investigación.* 37

Tabla 2. *Población, muestra y unidad de análisis del estudio* 74

Tabla 3. *Operacionalización de variables* 75

Tabla 4. *Actividades de la fase de Creación del dataset* 79

Tabla 5. *Actividades de la fase de Preprocesamiento* 90

Tabla 6. *Actividades de la fase de Extracción de características* 93

Tabla 7. *Actividades de la fase de Construcción del modelo* 97

Tabla 8. *Actividades de la fase de Evaluación del modelo* 103

Tabla 9. *Recursos del proyecto* 107

Tabla 10. *Datos recopilados por cuenta de Facebook* 111

Tabla 11. *Datos recopilados de Twitter por departamento* 115

Tabla 12. *Parámetros de CountVectorizer* 140

Tabla 13. *Parámetros de TfidfVectorizer* 141

Tabla 14. *Parámetros de Word2Vec* 143

Tabla 15. *Parámetros de FastText* 145

Tabla 16. *Mejores vectorizadores con KNN* 150

Tabla 17. *Resultados del modelo KNN* 150

Tabla 18. *Mejores vectorizadores con Árbol de Decisión* 151

Tabla 19. *Resultados del modelo Árbol de Decisión* 152

Tabla 20. *Mejores vectorizadores con Naive Bayes* 153

Tabla 21. *Resultados del modelo Naive Bayes* 154

Tabla 22. *Mejores vectorizadores con SVM* 155

Tabla 23. *Resultados del modelo SVM* 155

Tabla 24. *Mejores vectorizadores con Regresión Logística* 156

Tabla 25. *Resultados del modelo Regresión Logística* 157

Tabla 26. *Mejores arquitecturas del MLP* 158

Tabla 27. *Comparación de las mejores métricas de cada modelo* 163

Resumen

Uno de los problemas surgidos a raíz de esta emergencia sanitaria fue la circulación de información falsa del COVID-19, la cual se ve potenciada debido al gran número de publicaciones existentes en la actualidad, lo que hace difícil determinar la validez de la información que se recibe a diario. Es por esto que el estudio realizado tuvo como objetivo encontrar el mejor clasificador de información falsa del COVID-19 en el contexto peruano, para ello se recolectaron 2022 registros de información relacionado al COVID-19 mediante web scraping de portales webs, publicaciones y comentarios de Facebook y Twitter, los cuales fueron etiquetados manualmente como *Verdadero* o *Falso*, tomando como referencia fuentes confiables como el MINSA, OPS y OMS, y validadas posteriormente por personal del MINSA. Luego, se utilizaron técnicas de Natural Language Processing como Bag of Words, TF-IDF, word2vec y FastText para extraer las características de cada registro. Finalmente, se realizaron diversos experimentos con algoritmos de Machine Learning como KNN, Árbol de decisión, Naive Bayes, SVM, Regresión Logística y MLP, con el fin de encontrar la mejor combinación de técnicas para el modelo de clasificación. Los resultados se evaluaron según las métricas de Accuracy, Precision, Recall y F1-score, y se encontró que el mejor modelo resultó de la combinación del algoritmo SVM (C (0.5), gamma (1) y kernel (*rbf*)) con TF-IDF de dimensión 300 y n-gramas de 1 a 2, cuyas métricas fueron superiores al resto con 87,41% de Accuracy, 88,63% de Precision, 87,39% de Recall y 88% de F1-score.

Palabras Clave: *Información falsa, COVID-19, Machine Learning, Natural Language Processing, Accuracy*

Abstract

One of the problems that arose as a result of this health emergency was the circulation of false information on COVID-19, which is enhanced due to the large number of publications that currently exist, which makes it difficult to determine the validity of the information received daily. Therefore, the study carried out aimed to find the best classifier of false information on COVID-19 in the peruvian context, for which 2022 records of information related to COVID-19 were collected through web scraping of web portals, Facebook posts and comments. and Twitter, which were manually labeled as *True* or *False*, taking as reference reliable sources such as MINSA, PAHO and WHO, and later validated by MINSA personnel. Then, Natural Language Processing techniques such as Bag of Words, TF-IDF, word2vec and FastText were used to extract the features of each record. Finally, several experiments were carried out with Machine Learning algorithms such as KNN, Decision Tree, Naive Bayes, SVM, Logistic Regression and MLP, in order to find the best combination of techniques for the classification model. The results were evaluated according to the Accuracy, Precision, Recall and F1-score metrics, and it was found that the best model resulted from the combination of the SVM algorithm (C (0.5), gamma (1) and kernel (rbf)) with TF -IDF of dimension 300 and n-grams from 1 to 2, whose metrics were superior to the others with 87.41% Accuracy, 88.63% Precision, 87.39% Recall and 88% F1-score.

Palabras Clave: *False information, COVID-19, Machine Learning, Natural Language Processing, Accuracy*

INTRODUCCIÓN

Actualmente es común estar desinformado debido a la gran cantidad de información falsa que circula en la web, este problema se ve potenciado por las redes sociales. Tomemos como ejemplo un post hipotético de Facebook acerca de una hierba milagrosa que cura el COVID-19, donde una fuente "confiable" repostea esta noticia y la ven 1 000 personas, de las cuales 100 la comparten, y, suponiendo que cada persona tiene una cantidad promedio de 338 amigos, y que a su vez, al menos 100 usuarios verán la publicación compartida por dicha persona en su red social, entonces alrededor de 10 000 personas lograrían enterarse de este contenido, esto quiere decir que, si 100 de estas fuentes "confiables" comparten la misma noticia en sus redes sociales, la cantidad de personas "informadas" crecería exponencialmente (Osman, 2021). En este caso, solamente se tiene en cuenta que la propagación de la información fue mediante una red social (Facebook); sin embargo, es ilógico pensar que solo se difundirá por esta red, puesto que existen otras redes de gran popularidad como Twitter, YouTube, TikTok, Instagram y WhatsApp.

Entonces, se puede decir que las plataformas de redes sociales juegan un papel vital en la difusión de información engañosa relacionada con el COVID-19. Esto se ve potenciado con la presencia de nuevas variantes cuyas características y detalles aún no se conocen por completo, lo cual produce miedo y pánico entre el público en general. Esta propagación puede darse de manera inconsciente o intencionalmente buscando cumplir algún fin en particular como desestabilizar la economía o promover un producto específico para lograr enormes ganancias.

En el contexto de la pandemia en Perú, la información engañosa compartida sobre cierres de lugares públicos, vacunas y estadísticas de muertes ha avivado el pánico de comprar comestibles, desinfectantes, mascarillas y productos de papel, lo que a su vez condujo a una escasez que interrumpió la cadena de suministro. Dichas desinformaciones se propagan comúnmente mediante las plataformas de Facebook y Twitter, generando miedo, rechazo y desconfianza de los funcionarios públicos, por ello es habitual que el Ministerio de Salud (MINSA) realice publicaciones para desmentir la información falsa del COVID-19 circulante en el país; sin embargo, no pueden desmentirlas todas puesto que hay muchas publicaciones y comentarios en el que se comparte contenido falso, lo cual hace difícil la detección manual de cada una de ellas.

Para hacer frente a esta problemática, las técnicas de Machine Learning y Natural Language Processing son de gran ayuda, puesto que facilitan la automatización del reconocimiento de información verdadera o falsa del COVID-19 mediante el análisis de patrones e identificación de características de cada documento. Mediante estas, se pueden diseñar modelos predictivos que ayuden a combatir el virus de la desinformación que se vive actualmente en el país.

La estructura del presente trabajo se divide en 5 capítulos. El primero de ellos corresponde al "Planteamiento del problema" en donde se explica la realidad problemática y se plantean los objetivos de la investigación junto a su respectiva delimitación. El capítulo 2 trata acerca del "Marco teórico", el cual es necesario para brindar un fundamento teórico y tomar de referencia trabajos anteriores que servirán de guía en la investigación. El capítulo 3 titulado "Metodología" describe y detalla la metodología a emplear, así como la población, muestra y el diseño de la investigación. En el capítulo 4, "Desarrollo de la solución" se detalla paso a paso el desarrollo de cada una de las fases de la metodología, obteniendo los resultados de todos los experimentos llevados a cabo para encontrar el mejor modelo de clasificación según las métricas establecidas. Finalmente, el capítulo 5 titulado 'Conclusiones y recomendaciones' se da a conocer las conclusiones obtenidas del trabajo realizado y algunas recomendaciones para investigaciones futuras.

CAPÍTULO 1: PLANTEAMIENTO DEL PROBLEMA

1.1 Descripción de la Realidad Problemática

En la actualidad, la desinformación abunda en todo el mundo, tanto en temas políticos, como en culturales y principalmente en temas de salud, siendo estos de los más importantes. Si bien cada persona posee algún tipo de fuente confiable, ya sean periódicos, reporteros, influencers o páginas de redes sociales, no siempre se puede contar con la fiabilidad de la información que dichos actores comparten. Es por ello que la desinformación cada vez es mayor.

En lo que respecta a la salud, los rumores relacionados al COVID-19 son un peligro presente en cada país, tanto así que incluso el secretario general de la ONU, Antonio Guterres, identificó a esos rumores como un enemigo global, pues muchas personas desinformadas que leen esos rumores pueden llegar a propagar la desinformación en su entorno social. (TIME, 2020). Asimismo, no solo son rumores los que se propagan en todo el mundo (a través de páginas webs o redes sociales), también lo hacen los estigmas y las teorías conspiracionales relacionados a la pandemia del COVID-19, lo cual es grave, ya que hace más difícil el control de la pandemia, debido a ello, la Organización Mundial de la Salud (OMS) los declaró como amenazas emergentes y recomendó medidas sistemáticas de monitoreo y control para combatir este tipo de desinformación (CDC, 2020). Es vital tener una percepción cuestionadora ante cualquier tipo de información encontrada pues no es seguro que dicha información sea confiable o no. Entonces, podemos decir que existe una enorme facilidad de estar desinformados, es por ello que diversos estudios realizados alrededor del mundo evidencian la gravedad de este problema, cuál fue su comportamiento a inicios de la pandemia, así como durante la misma y cuáles fueron los países más propensos a estar desinformados en temas del COVID-19.

El estudio realizado por Saiful et al. (2020) demuestra cómo fue el comportamiento de las personas a nivel mundial respecto a la propagación de desinformación del COVID-19 en los primeros meses de la pandemia (desde el 31 de diciembre de 2019 hasta el 5 de abril de 2020). En este estudio se recopiló la información falsa (rumores, estigmas, teorías conspiracionales) relacionada al COVID-19 distribuida por Facebook, Twitter y sitios web de cadenas televisivas y periódicos en 87 países. En la siguiente figura se puede observar que el tipo de desinformación

más común propagada en este periodo de tiempo fue los rumores (89%), los cuales iban creciendo con el pasar de los días (llegando a sus picos más altos en el mes de marzo), en cuanto a las teorías conspiracionales (7.8%), aunque estuvieron presentes en todos los meses, su cantidad fue mucho menor comparada a la de los rumores, finalmente los estigmas tuvieron cierta propagación en el mes de febrero, siendo este tipo de desinformación la más baja respecto a las demás (3.5%).

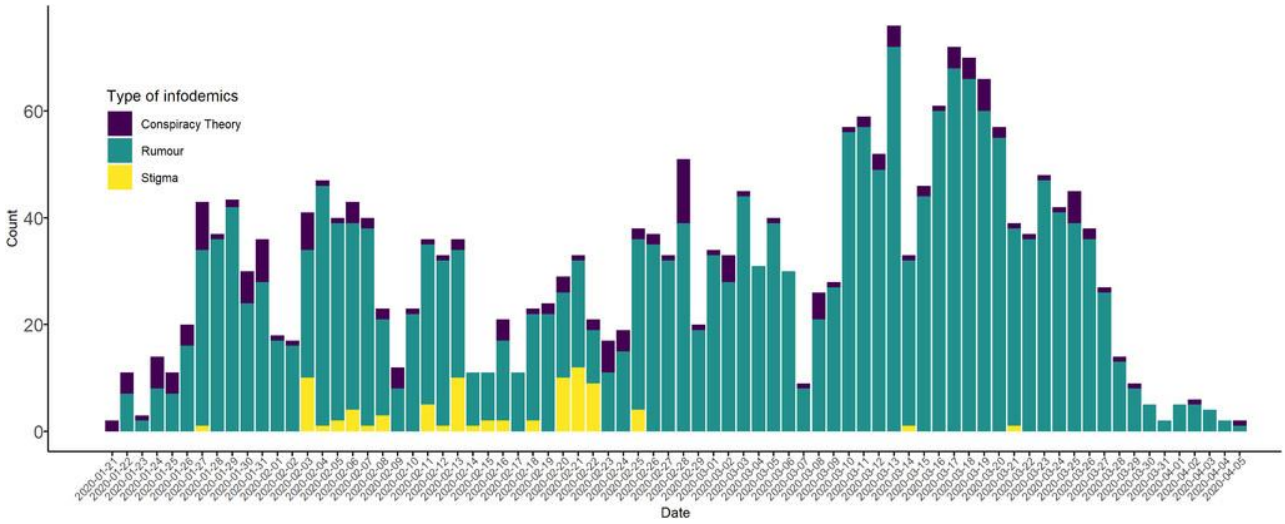


Figura 1. Rumores, estigma y teorías de conspiración asociadas con COVID-19 y cronograma de informes detectados durante el estudio, 2020.
Fuente: Islam et al. (2020). *COVID-19–related infodemic and its impact on public health: A global social media analysis*. (p. 1623)

De toda esta desinformación propagada, se detectó que un gran porcentaje del total de información era falso (82%), mientras que solo el 9% era correcto; por otro lado, el 8% era información engañosa y el porcentaje de las noticias cuya verificación de su falsedad o veracidad fue imposible era el 1%. La siguiente figura muestra la clasificación de dicha información respecto a las áreas temáticas del COVID-19:

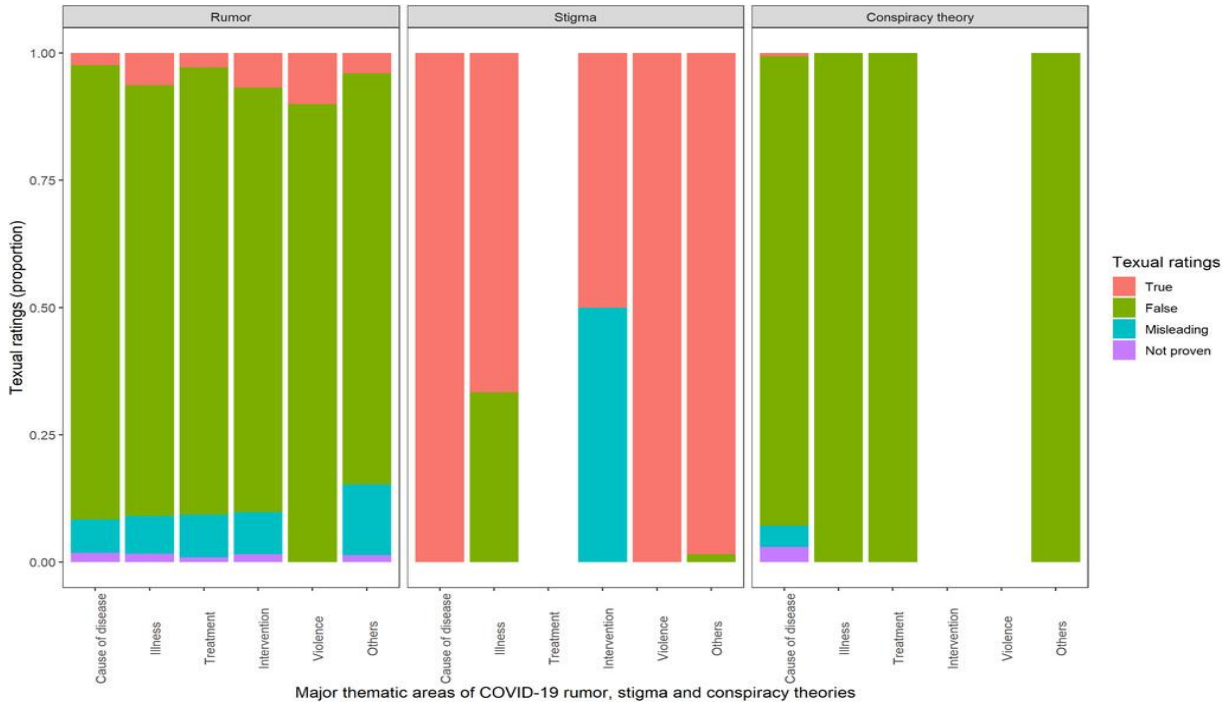


Figura 2. Precisión fáctica de los rumores, el estigma y las teorías de la conspiración relacionadas con las causas de enfermedad, tratamiento y medidas de control del COVID-19.

Fuente: Islam et al. (2020). *COVID-19–related infodemic and its impact on public health: A global social media analysis.* (p. 1624)

Teniendo un panorama general más claro, pasaremos a ver cómo se desarrolló este problema en un contexto más cercano al nuestro, para ello, tomaremos como referencia el estudio realizado por Nieves-Cuervo et al. (2021), en el que se detalla el comportamiento de la propagación de noticias falsas en el contexto de la mortalidad por COVID-19 en seis países latinoamericanos: Argentina, Brasil, Chile, Colombia, México y Perú.

Los autores mencionan que, en Latinoamérica, la facilidad de acceso a grandes volúmenes de datos en línea, de los cuales muchas veces se desconoce su veracidad, ha generado dudas sobre la calidad y la fiabilidad de la información disponible en temas de salud. La siguiente figura presenta la capacidad de reconocimiento de información falsa del COVID-19 de acuerdo a las 6 variables estudiadas:

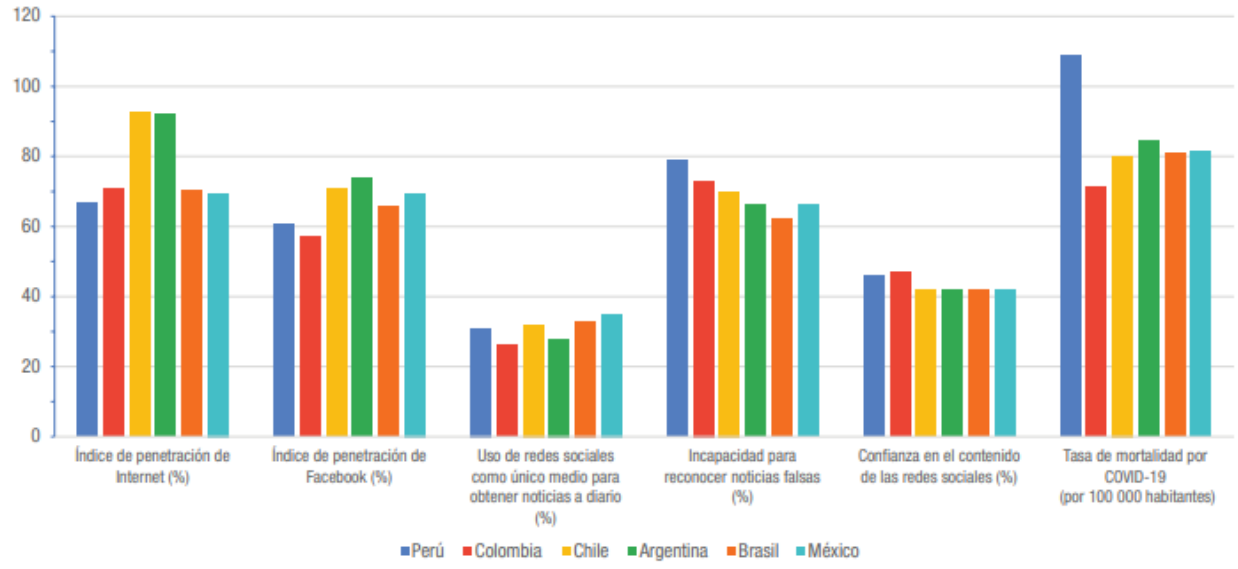


Figura 3. Caracterización de los países analizados, según las variables de estudio, noviembre del 2020.

Fuente: Nieves-Cuervo et al. (2021). *Infodemia: noticias falsas y tendencias de mortalidad por COVID-19 en seis países de América Latina*. (p. 4)

De acuerdo a este gráfico podemos mencionar que Chile y Argentina fueron los países con los mayores índices de penetración de Internet (92,4% y 92,0%) y Facebook (70,9%, 73,9%), también, ambos países hacen un considerable uso de las redes sociales como único medio para la obtención de noticias a diario (32,0% y 28,0%); mientras que Brasil y Colombia mostraron un comportamiento intermedio según el índice de penetración de Internet y tuvieron índices de penetración de Facebook relativamente bajos respecto a los otros países; México presentó el uso más alto de redes sociales (35%), mientras Perú (79,0%) y Colombia (73,0%) presentaron los mayores valores del índice de incapacidad para reconocer noticias falsas y los menores índices de penetración de Facebook (60,7% y 56,9%, respectivamente). Asimismo, nuestro país fue el que presentó el porcentaje mayor respecto a la incapacidad para reconocer noticias falsas (79,0%) y que fue el segundo con mayor confianza en el contenido de redes sociales (46,0%) conllevando a una mayor tasa de mortalidad por COVID-19 (108,7 por 100 000 habitantes). Claramente se nota que Perú tiene gran incapacidad para reconocer la información falsa, lo que significa que la desinformación se propaga fácilmente en nuestro país a diferencia de los demás países evaluados, pues se observó un patrón más favorable en Brasil (62,0%),

Argentina (66,0%) y México (66,0%) que, además, tuvieron tasas de mortalidad más bajas (entre 81 y 85 por 100 000 habitantes) que nuestro país. Debido a este riesgo de mortalidad por incapacidad crítica de noticias, el gobierno peruano fue el primero en implementar acciones de orden político para controlar las noticias falsas: sentencias de prisión para quienes creen o difundan información falsa (Artículos 315A y 348 del Código Penal), sumado a ello, el Ministerio de Salud (MINSA), ente máximo de salud en nuestro país, se encargó de divulgar información verídica de temas del coronavirus mediante sus redes sociales y página web, alertando y concientizando a la población a evitar la difusión de noticias falsas (Álvarez-Risco et al., 2020).

De igual manera, el estudio nacional realizado por Palomino et al. (2020), buscó identificar la participación del personal sanitario en relación con la difusión de noticias falsas del COVID-19. Para ello, se investigó a 60 profesionales de la salud, entre médicos, enfermeros, técnicos, etc. Los resultados mostraron que el 90% de los participantes manifestó haber recibido algún tipo de noticia falsa por medio de sus redes sociales, el 94,9 % indicó que compartió noticias no muy confiables sin verificar su veracidad, mientras que el 53,3% consideró que se encuentra parcialmente capacitado para determinar la validez de una noticia, finalmente, el 81,4% contrastó la información con diversos medios para conocer si la noticia es verdadera o no. Con estos resultados se llegó a la conclusión que, en teoría el personal de salud debería poder detectar cuando se trata de una noticia falsa; sin embargo, aún no se sienten completamente seguros de hacerlo debido a que muchos de ellos consideran que no están completamente capacitados, lo cual resulta un problema grave tratándose del personal sanitario, quienes son un agente fundamental en la lucha contra el COVID-19.

Por otro lado, los investigadores de la Universidad del Pacífico: Matthew Bird, Paula Muñoz y Felipe Freier, realizaron una encuesta a 2 000 peruanos a cerca de sus posiciones respecto a la vacuna contra el COVID-19. Los resultados mostraron que el 48% de los encuestados no se encuentra convencido de la efectividad de las vacunas por lo que preferían no vacunarse, mientras que el 13,8% no lo haría debido a la desinformación en torno a la vacuna. Asimismo, el 18,5% consideró que las vacunas forman parte de un programa mundial de conspiración; por otro lado, el 15,4% cree que las vacunas alteran el ADN, mientras que el 14,7% piensa que podrían contener un chip para rastrearlos. Toda esta resistencia en contra de

las vacunas se debe a la forma en cómo las personas reciben y procesan la información, en qué medida confían en el contenido y la fuente de distribución, y los valores y creencias con las que se identifican; también, se descubrió que los ciudadanos de niveles socioeconómicos más bajos quienes rechazan la vacuna lo hacen en mayor proporción que los que pertenecen a niveles altos, puesto que tienen mayores creencias conspirativas y mayor percepción de no efectividad de la vacuna. (CIUP, 2021)

En este contexto, la incertidumbre, el miedo y la preocupación se convierten en factores perfectos para la desinformación en sus diversas variantes. Sin embargo, no todo está perdido pues podemos hacer frente a esta problemática con la ayuda del Machine Learning, ya que, gracias a este, es posible desarrollar un modelo de clasificación que permita identificar cualquier información relacionada al COVID-19 y etiquetarla como verdadera o falsa, esto con la finalidad de disminuir la gran cantidad de información falsa que circula en la web peruana.

1.2 Formulación del Problema

1.2.1 Problema Principal

PP: ¿Es posible detectar información falsa del COVID-19 aplicando técnicas de Machine Learning y Natural Language Processing?

1.2.2 Problemas Secundarios

PE1: ¿Existe suficiente información del COVID-19 en la web peruana para la creación del dataset?

PE2: ¿Realizar el preprocesamiento permite un mejor entrenamiento del modelo?

PE3: ¿Qué técnicas de Natural Language Processing podemos utilizar para una adecuada vectorización de características?

PE4: ¿Qué técnicas de clasificación de Machine Learning podemos aplicar para construir un modelo de clasificación robusto de información falsa del COVID-19?

PE5: ¿Qué métricas existen para evaluar el desempeño del modelo de clasificación de información falsa del COVID-19?

1.3 Objetivos de la Investigación

1.3.1 Objetivo General

OP: Desarrollar un modelo para la detección de información falsa del COVID-19 aplicando técnicas de Machine Learning y Natural Language Processing.

1.3.2 Objetivos Específicos

OE1: Crear un dataset de información verdadera y falsa del COVID-19 en el contexto peruano.

OE2: Desarrollar el preprocesamiento para cada documento del dataset.

OE3: Utilizar técnicas de Natural Language Processing que permitan obtener vectores característicos de cada documento.

OE4: Utilizar técnicas de clasificación de Machine Learning para construir un modelo de clasificación robusto de información falsa del COVID-19.

OE5: Utilizar métricas para evaluar el desempeño del modelo de clasificación de información falsa del COVID-19.

1.4 Justificación de la Investigación

1.4.1 Justificación Teórica

La investigación se realiza para aportar un nuevo enfoque en el campo de Machine Learning, ya que el modelo trata de detectar una noticia falsa relacionada al COVID-19 y prevenir la desinformación de parte los peruanos. Si bien, actualmente existen trabajos parecidos, estos tienen un enfoque distinto pues están inglés, por lo que no existe un estudio realizado en el contexto peruano, es por ello que nuestro enfoque busca aplicarlo a este contexto, a fin de posibilitar la llegada de información correcta, no solo a nuestros compatriotas, sino también a todos los hispanohablantes.

1.4.2 Justificación Práctica

El presente modelo trata con un problema común en Perú, el cual es la desinformación, sobretodo, en épocas de pandemia, estar desinformado en lo que respecta al COVID-19 puede ser perjudicial para la salud propia y la de los demás. Es por ello que, gracias a esta

investigación, cualquier ciudadano peruano podrá mantenerse informado acerca de las noticias falsas existentes del COVID-19 en nuestro país y no dejarse llevar por la masa de información errónea que circula en la web.

1.4.3 Justificación Metodológica

El aporte metodológico de esta investigación, no solo se basa en proporcionar un nuevo contexto, sino también aporta en la manera de recolectar los datos, ya que, investigaciones anteriores trabajaron con noticias falsas del COVID-19 que estaban disponibles en una base de datos o los websites de noticias, pero en este caso, no solos se trabajará con los websites, sino que también se incorporará data proveniente de las redes sociales, como puede Twitter, y para ello, se describe una forma de recolectar datos de esta red social y almacenarlos en un archivo estructurado.

1.5 Delimitación del Estudio

1.5.1 Espacial

El estudio se realizará en Perú, pues se busca que los peruanos dejen de estar desinformados respecto a las publicaciones falsas que circulan por la red. Así pues, se extraerá la mayor cantidad de registros (tanto falsos como reales) de las webs peruanas (principales diarios electrónicos peruanos) así como de los comentarios realizados en Twitter y Facebook.

1.5.2 Temporal

El periodo en que se llevará a cabo el estudio es de 7 meses, iniciando con la recolección de información referente al problema y sus consecuencias, en el mes de enero de 2021 y finalizando con la elección del mejor modelo que clasifique adecuadamente las noticias falsas del COVID-19 a finales del mes de julio de 2022.

1.5.3 Conceptual

La investigación se centrará en el desarrollo de un modelo de clasificación de información falsa del COVID-19 que circula en la web peruana, para ello, se aplicarán técnicas de Machine Learning y Natural Language Processing. Para explicar mejor los límites del estudio, es necesario mencionar que solo se planea crear un modelo de clasificación (entrenado y evaluado),

mas no se planea crear una página web o alguna aplicación en la que esté operativo el modelo. Así pues, para la creación del modelo se aplicarán todos los conceptos existentes en Machine Learning y Natural Language Processing, a fin de encontrar el mejor modelo de clasificación.

CAPÍTULO 2: MARCO TEÓRICO

2.1 Antecedentes

2.1.1 Resumen de artículos

Elhadad, M., Li, K., y Gebali, F. (2020). ***Detecting misleading information on covid-19. IEEE Access, 8,*** **165201-165215. doi: 10.1109/ACCESS.2020.3022867.**

- **Resumen:**

  Este artículo busca dar solución al problema de detectar información engañosa relacionada al COVID-19, para ello, se propone un modelo de detección de información engañosa, tomando fuentes de información confiable (OMS, UNICEF, ONU), lo que asegura la validez de los datos. Se construyó el modelo con 10 algoritmos de Machine Learning y 7 técnicas de extracción de características. Luego, se realizó una validación cruzada de 5 para verificar la validez de los datos recopilados e informar la evaluación mediante 12 métricas de desempeño. Los resultados de la evaluación indican la calidad y validez de los datos recopilados de verdad sobre el terreno y su efectividad en la construcción de modelos para detectar información engañosa.

- **Base de datos:**

  Para poder realizar la investigación se necesitaron fuentes de información confiables con las cuales alimentar el modelo, se trabajó con la API de verificación de datos de Google, que permite a los usuarios navegar y buscar datos en diferentes sitios web de todo el mundo. Por lo cual se usó la herramienta para poder introducir distintas palabras clave de búsqueda relacionadas al COVID19. Estos datos se almacenaron en un servidor usando MySQL, con los datos de cada fuente en una tabla diferente. Además, se hizo el etiquetado respectivo a cada dato ingresado, ya sea verdadero o falso. En este caso solo se recopilaron datos en inglés, por lo que se hizo la limpieza respectiva para obtener

más información en ese idioma. La recopilación se llevó a cabo desde el 4 de febrero de 2020 hasta el 10 de marzo de 2020.

- **Metodología:**

  La metodología utilizada consistió en dos partes. (Figura 4)

  Para la parte de preprocesamiento de datos, se recurrió a los siguientes pasos:

  • *Extracción duplicada:* Verificar los datos recolectados y eliminar los redundantes.

  • *Estandarización de datos y binarización de etiquetas*: Aseguraron la consistencia de datos asignándoles una estructura.

  • *Integración de datos:* Luego de realizar los puntos anteriores los datos se insertaron en una tabla de hechos.

  Y para la parte de construcción del modelo se realizaron los siguientes pasos:

  • *Preparación:* Se prepararon los datos introduciendo 3 etiquetas (FactData, etiquetaeidioma).

  • *Ingeniería de características:* Se preprocesan y seleccionan las características, luego se extrajeron las características principales con las técnicas de tf-idf y word embedding.

  • *Aprendizaje:* Los vectores de características extraídos, que representaban cada documento en los datos de entrenamiento, se introdujeron en diferentes algoritmos de clasificación. Haciendo uso de 80 % de los datos de entrenamiento y 20 % de pruebas, tomando en la validación cruzada un valor de 5.

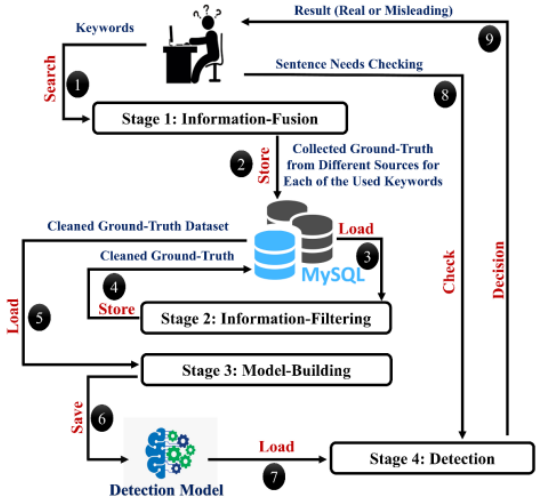


Figura 4. Marco propuesto para detección de información engañosa

Fuente: Elhadad et al. (2020). *Detecting misleading information on covid-19.* (p. 165204)

- **Resultados:**

  Para la evaluación del modelo se utilizaron 12 métricas: Accuracy, Error Rate, Precision, Recall, F1-Score, AUC, Specificity, Geometric-Mean, Miss Rate, Fall-Out Rate, False Discovery Rate y False Omission Rate. Los resultados en general demostraron la validez de los datos de verdad sobre los recopilados del terreno. Los mejores resultados se obtienen de los clasificadores NN, DT y LR.

			Classification Algorithm									
Metric	Feature Extraction		DT	MNB	BNB	LR	kNN	Perceptron	NN	LSVM	ERF	XGBoost
Accuracy (%)	TF		99.04	98.29	95.09	99.36	96.42	98.93	99.68	99.52	99.52	99.15
	TF-IDF	Unigram	98.93	98.01	93.96	98.18	98.67	99.57	99.63	99.52	99.15	99.25
		Bigram	98.02	97.92	90.55	97.49	97.06	98.72	98.99	98.99	98.56	96.80
		Trigram	93.00	92.57	92.68	92.47	90.92	93.54	93.75	93.91	93.43	91.45
		N-gram (n=2:3)	98.56	98.08	90.97	97.81	96.64	98.99	99.57	99.57	98.72	97.44
		Characters Level	99.20	97.97	91.35	98.08	99.36	99.52	99.63	99.63	99.09	99.41
	Word Embeddings		97.33	55.45	64.80	80.61	89.16	72.12	93.86	65.99	98.72	99.52
Error Rate (%)	TF		0.96	1.71	4.92	0.64	3.58	1.07	0.32	0.48	0.48	0.86
	TF-IDF	Unigram	1.07	1.99	6.04	1.82	1.34	0.43	0.37	0.48	0.86	0.75
		Bigram	1.98	2.08	9.46	2.51	2.94	1.28	1.02	1.02	1.44	3.21
		Trigram	7.00	7.43	7.32	7.53	9.08	6.46	6.25	6.09	6.57	8.55
		N-gram (n=2:3)	1.44	1.92	9.03	2.19	3.37	1.02	0.43	0.43	1.28	2.56
		Characters Level	0.80	2.03	8.65	1.92	0.64	0.48	0.37	0.37	0.91	0.59
	Word Embeddings		2.67	44.55	35.20	19.39	10.84	27.89	6.14	34.01	1.28	0.48
Area Under the Curve (%)	TF		97.91	96.87	90.58	98.74	93.45	98.83	99.41	99.03	99.03	98.33
	TF-IDF	Unigram	97.83	97.36	88.75	97.75	98.65	99.16	99.28	99.12	98.14	98.58
		Bigram	97.09	97.62	84.60	98.13	97.75	98.00	99.07	99.07	98.06	96.75
		Trigram	92.86	94.16	92.29	94.98	88.41	95.06	95.06	95.70	94.34	93.26
		N-gram (n=2:3)	97.15	97.67	84.66	98.43	96.91	97.96	99.34	99.34	97.68	97.24
		Characters Level	98.19	97.23	85.12	97.30	99.20	99.03	99.47	99.28	98.20	98.78
	Word Embeddings		94.68	63.05	65.02	70.54	82.48	60.20	89.46	57.59	97.85	99.21

Figura 5. Accuracy, Error Rate, and Area Under Curve de los resultados de la validación
Fuente: Elhadad et al. (2020). *Detecting misleading information on covid-19.* (p. 165209)

Isaakidou, M., Zoulias, E., y Diomidous, M. (2021). *Machine Learning to Identify Fake News for COVID-19*. In Public Health and Informatics (pp. 108-112). IOS Press.

- **Resumen:**

  La Comisión Europea realizó una encuesta sobre "fake news" a través de ciudadanos de la UE para estimar la conciencia y el comportamiento de las personas en relación con la aparición de noticias falsas y desinformación en formato electrónico. Los hallazgos son bastante preocupantes, ya que alrededor del 40% se encuentra con noticias falsas a diario y el 85% evalúa las noticias falsas como un problema. Es por eso que el estudio busca es introducir un enfoque de Inteligencia Artificial y el algoritmo de Árboles de Decisión para identificar noticias falsas sobre el COVID-19.

- **Base de datos:**

La base de datos que se usó en el estudio fue recuperada del repositorio ReCOVery, el cual facilita la evaluación de la confiabilidad de las noticias sobre el COVID-19. Son 2304 artículos sobre coronavirus de 22 sitios web confiables y 38 no confiables. Incluyen la URL, el autor, el título, la imagen, el cuerpo, el país, entre otros. Por lo cual, en el trabajo presentado se evalúa el cuerpo y el texto principalmente.

- **Metodología:**

La metodología que siguieron tuvo los siguientes pasos en el preprocesamiento de datos:

- Mezcla de los datos para evitar sesgos
- Pasar a minúsculas todas las palabras
- Se eliminaron los signos de puntuación
- Se hizo el filtrado de *stopword* para excluir palabras comunes
- Finalmente se hizo la tokenización

Luego, para el algoritmo de la clasificación se realizó lo siguiente:

- Se vectorizan las columnas TextBody y Title para luego poder aplicar TF-IDF.
- Luego se calculó la frecuencia de término normalizada TF.
- Se particionó los datos en datos de entrenamiento y de prueba.
- Se colocó un valor semilla de 42.
- Además, se apoyaron del valor de impureza de Gini, junto con el parámetro Splitter, una estrategia utilizada para elegir el método de división que se lleva a cabo en los nodos.
- Finalmente, se introdujo el parámetro de Profundidad máxima del árbol, que es el número de nodos del árbol que se expanden hasta que queden solo hojas puras.

Criterion	Splitter	Max Depth
Entropy (for the information gain)	Best	20
Gini (Gini impurity)	Best	20
Entropy (for the information gain)	Random	20
Gini (Gini impurity)	Random	20
Entropy (for the information gain)	Best	40
Gini (Gini impurity)	Best	40
Entropy (for the information gain)	Random	40
Gini (Gini impurity)	Random	40

Figura 6. Combinaciones de parámetros de árboles de decisión

Fuente: Isaakidou et al. (2020). *Machine Learning to Identify Fake News for COVID-19.* (p. 110)

- **Resultados:**

  Los resultados mostraron el mejor resultado y el peor resultado de precisión para cada uno de los árboles creados, cada uno junto con sus respectivos parámetros. Los resultados revelan que la combinación de parámetros más eficiente 100 % de precisión general, entre los probados es "Entropía" como criterio, "Mejor: como método de división y 40 como árbol de longitud máxima utilizado". Sin embargo, surge un punto interesante de que la aplicación del algoritmo en los títulos tiene una precisión general del 72,09 % en todas las combinaciones de parámetros.

Parameters Combination	Overall Accuracy Body	Overall Accuracy Title
Entropy (for the information gain) - Best - 20	98,09%	72,09%
Gini (Gini impurity) - Best -20	96,05%	72,09%
Entropy (for the information gain) - Random - 20	95,53%	72,09%
Gini (Gini impurity) - Random - 20	94,01%	72,09%
Entropy (for the information gain) - Best - 40	100%	72,09%
Gini (Gini impurity) - Best - 40	99,13%	72,09%
Entropy (for the information gain) - Random - 40	99,87%	72,09%
Gini (Gini impurity) - Random - 40	96,44%	72,09%

Figura 7. Porcentaje de precisión de clasificación promedio: 10 iteraciones de algoritmos independientes

Fuente: Isaakidou et al. (2020). *Machine Learning to Identify Fake News for COVID-19.* (p. 110)

Bojjireddy, S., Chun, A., y Geller, J. (2021). *Machine Learning Approach to Detect Fake News, Misinformation in COVID-19 Pandemic*. In DG. O2021: The 22nd Annual International Conference on Digital Government Research (pp. 575-578).

- **Resumen:**

  Uno de los problemas sociales más graves de desinformación, específicamente utilizando noticias falsas relacionadas con la salud y COVID-19. Este problema es tan común que puede traer consecuencias graves para las personas desinformadas, es por ello que el estudio presenta un enfoque de Machine Learning como solución técnica para automatizar la detección de noticias falsas y contenidos engañosos.

- **Base de datos:**

Se crearon tres conjuntos de datos mediante web scraping y la combinación de varios conjuntos de datos más pequeños disponibles públicamente.

• **Dataset1:** Se combinaron múltiples fuentes de datos de noticias falsas disponibles públicamente: los conjuntos de datos políticos, el conjunto de datos de Kaggle, fakeorreal, Kaggle fakereal. Este dataset contiene campos de título y texto del cuerpo de los artículos de noticias. Cuenta con un total de 65 938 noticias reales y 28 995 noticias falsas después de combinar y limpiar.

• **Dataset2:** Este dataset es el conjunto de datos LIAR, no contiene cuerpo de texto extenso de los artículos de noticias, sino solo los títulos. Cuenta con un total de 31 950 noticias reales y 17 809 noticias falsas.

• **Dataset3:** Este conjunto de datos se formó como una combinación de Dataset1 y Dataset2. Solo contiene la columna "título" pero no tiene la columna "texto". En total, contiene 83 049 reales y 59 968 falsos registros de datos de noticias.

- **Metodología:**

La metodología que emplearon se dividió en las siguientes etapas:

1. Recolección de datos: Se recolectaron los 3 datasets y se combinaron en uno solo.

2. Preprocesamiento de datos: De los 3 datasets, se hizo la limpieza respectiva, eliminando aquellos datos que no mostraban información relevante o aquellos datos cuya información era nula.

3. Vectorización: Se aplicaron herramientas de vectorización como Count Vectorizer, disponible en la biblioteca ski-learn en Python, y la medida estadística TF-IDF, para la extracción de características.

4. Partición de la data: Se entrenaron distintos modelos con la partición de 80-20 y 70-30 para la data de entrenamiento y prueba.

5. Construcción del modelo: Posteriormente, se evaluaron 6 algoritmos de machine learning: Multinomial Naıve Bayes (MNB), Support Vector Machine (SVM), Multilayer Perceptron (MLP), Decision Tree (DT), Random Forest (RF) y Gradiente de Boosting (GB).

6. *Evaluación del modelo:* Solo se utilizó una métrica para evaluar la calidad del modelo el cual fue el accuracy.

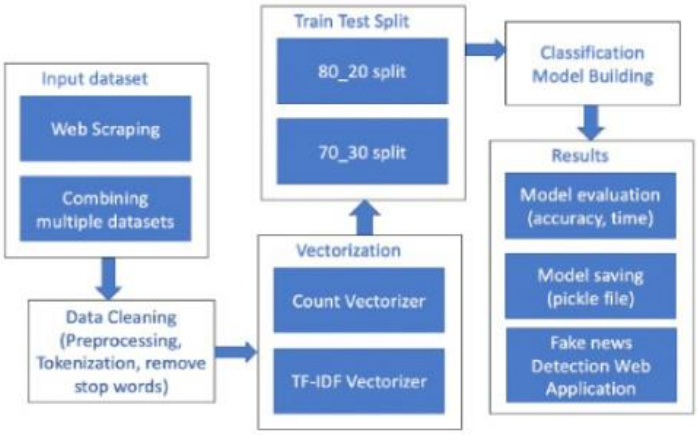


Figura 8. Diagrama de flujo de la detección de noticias falsas.

Fuente: Bojjireddy et al. (2020). *Machine Learning Approach to Detect Fake News, Misinformation in COVID-19 Pandemic.* (p. 577)

- **Resultados:**

Los resultados muestran que ningún modelo es malo, ya que la mayoría de ellos tienen accuracy bastante altos. Sin embargo, se decidió optar por el modelo Multilayer Perceptron con CV y una partición de data de 80-20 para el primer dataset; en cuanto al segundo, se observa que el mejor modelo es Support Vector Machine (SVM) con TF-IDF y partición de la data de 80-20; finalmente, para el tercer dataset, el mejor modelo es el Multilayer Perceptron (MLP) con CV y partición de 80-20.

Classifier	Dataset 1				Dataset 2				Dataset 3			
	CV 80_20	TFIDF 80_20	CV 70_30	TFIDF 70_30	CV 80_20	TFIDF 80_20	CV 70_30	TFIDF 70_30	CV 80_20	TFIDF 80_20	CV 70_30	TFIDF 70_30
Multinomial Naive Bayes (MNB)	88.90	77.52	**88.91**	75.77	81.26	67.00	**81.46**	65.45	**78.87**	77.74	78.80	77.09
Support Vector Machine (SVM)	94.34	**95.63**	94.25	95.39	84.18	**88.41**	84.00	88.39	86.24	**87.35**	85.83	87.03
Multilayer Perceptron (MLP)	**96.10**	95.63	95.71	95.66	87.09	86.02	**87.10**	86.58	**87.75**	87.10	87.00	86.51
Decision Tree (DT)	**91.22**	91.04	90.68	75.77	77.79	77.80	78.11	**78.46**	**81.58**	81.51	81.18	80.87
Random Forest (RF)	92.92	**93.06**	92.09	92.05	83.78	84.67	83.39	**84.87**	85.73	**86.39**	84.98	86.05
Gradient Boosting (GB)	91.28	91.21	91.30	**91.33**	83.04	83.80	83.35	**83.97**	78.93	**79.72**	78.87	79.68

Figura 9. Comparación de precisión en conjuntos de datos y parámetros experimentales. Los mejores resultados están en negrita.

Fuente: Bojjireddy et al. (2020). *Machine Learning Approach to Detect Fake News, Misinformation in COVID-19 Pandemic.* (p. 577)

Mazzeo, V., Rapisarda, A., y Giuffrida, G. (2021). *Detection of fake news on CoViD-19 on Web Search Engines*. arXiv preprint arXiv:2103.11804.

- **Resumen:**

  Los motores de búsqueda web tienen gran capacidad para capturar la mayor cantidad de información y ayudar a los usuarios a identificar rápidamente los resultados más relevantes y útiles; sin embargo, no siempre son los más fiables para sus consultas de búsqueda. Por lo que este estudio tiene como objetivo detectar posibles contenidos engañosos y falsos mediante la captura y análisis de información, que fluye a través de los motores de búsqueda mediante el uso de un conjunto de datos del mundo real asociado con la pandemia COVID-19. Para ello se trabajó con los algoritmos de: Naive Bayes, Regresión Logística, SVM, SGD, y Random Forest y las técnicas de extracción de características: BoW y Tf-idf.

- **Base de datos:**

  Para la recolección de datos se aplicó web scraping a distintas páginas web que hablaban acerca de temas del COVID-19. Principalmente, los atributos seleccionados de cada noticia fueron: Url, título, descripción y fecha. El conjunto de datos final consistió en una colección de aproximadamente 3350 resultados de noticias (falsos y reales), recopilados de 2084 URL diferentes. Todas las noticias se publicaron en un intervalo de tiempo de 7 meses, entre el 20 de enero y 28 de julio de 2020.

- **Metodología:**

  La metodología consta de los siguientes pasos:

  1. Recolección de datos: Se realizó la búsqueda de noticias relacionadas al covid-19 en distintas páginas web, obteniendo un dataset de 3350 noticias, entre falsas y reales.

2. Etiquetado: Después de la recolección, se procedió a realizar el etiquetado manual de noticias verdaderas (0) y falsas (1).

3. Preprocesamiento: Se eliminaron todas las partes que eran irrelevantes, redundantes y no relacionadas con el contenido: puntuación (con algunas excepciones de símbolos, como signo de exclamación, signo de interrogación y comillas) y delimitadores adicionales; símbolos, guiones de títulos y descripciones; palabras vacías. Es notable que las fake news se diferencian mucho más de las reales por el uso excesivo de puntuación, comillas, interrogantes, palabras en total letras mayúsculas y un signo de exclamación para alertar e instar a las personas a leer las noticias. Es por ello que estos símbolos no se eliminaron en esta etapa.

4. Selección de características: En etapa de ingeniería de características, se realizó la creación, transformación, extracción y selección de características, para ello se usó el algoritmo de Bag-of-Words (BoW) y Término de frecuencia - Frecuencia de documento inverso (TF-IDF), para mapear textos limpios (títulos y descripciones) en representaciones numéricas. También se extrajeron otras características (longitud, recuento, binario) de las URL.

5. Remuestreo: Se aplicó remuestreo debido al gran desbalance de la data, pues se tenía una relación de 90:10 para las clases real y falso respectivamente. Para ello, se aplicó una validación cruzada de 5. Donde cada fold se usó una vez como validación, mientras que los folds restantes formaron el conjunto de entrenamiento. Este proceso se ejecutó repetidamente hasta que cada fold de los 5 existentes se usó como conjunto de prueba.

6. Clasificación y evaluación del modelo: Finalmente, se aplicaron diversos algoritmos de machine learning para construir el modelo de clasificación, para ello se probó con los siguientes algoritmos: Naive Bayes, Regresión Logística, SVM, SGD, y Random Forest; cada uno con la técnica de extracción de características: BoW y Tf-idf. Asimismo, se utilizaron las métricas de Precision, Accuracy, Recall, F1-score para identificar el mejor de clasificación de fake news.

- **Resultados:**

Se puede observar que los mejores modelos son aquellos que utilizan re-sampling: Naive Bayes-BoW, Regresión Logística-BoW y SVM con TF-IDF; por lo que se podría utilizar cualquiera de ellos en la identificación de una noticia falsa relacionada al COVID-19.

Re-Sampling	Classifier	Pre-Processing	Precision	Accuracy	Recall	F_1-Score
	Naïve Bayes	**BoW**	**0.64**	**0.92**	**0.78**	**0.70**
	Logistic Regression	**BoW**	**0.73**	**0.93**	**0.68**	**0.71**
	SVM	BoW	0.73	0.93	0.63	0.68
	SGD	BoW	0.70	0.92	0.62	0.66
Over	Random Forest	BoW	0.82	0.92	0.42	0.56
	Naïve Bayes	TF-IDF	0.38	0.82	0.88	0.53
	Logistic Regression	TF-IDF	0.68	0.92	0.67	0.67
	SVM	**TF-IDF**	**0.79**	**0.94**	**0.60**	**0.69**
	SGD	TF-IDF	0.60	0.91	0.64	0.62
	Random Forest	TF-IDF	0.76	0.93	0.58	0.66
	Naïve Bayes	BoW	0.23	0.64	0.91	0.37
	Logistic Regression	BoW	0.38	0.83	0.79	0.51
	SVM	BoW	0.34	0.80	0.79	0.47
	SGD	BoW	0.26	0.71	0.83	0.39
Under	**Random Forest**	**BoW**	**0.46**	**0.87**	**0.74**	**0.57**
	Naïve Bayes	TF-IDF	0.22	0.62	0.89	0.35
	Logistic Regression	TF-IDF	0.34	0.79	0.84	0.48
	SVM	TF-IDF	0.30	0.75	0.85	0.44
	SGD	TF-IDF	0.22	0.64	0.83	0.34
	Random Forest	**TF-IDF**	**0.48**	**0.88**	**0.68**	**0.57**

Figura 10. Métricas de evaluación (Precisión, Exactitud, Recall y Puntaje F1) para todos los clasificadores, considerando solo la selección de características textuales.
Fuente: Mazzeo et al. (2020). *Detection of fake news on CoViD-19 on Web Search Engines.* (p. 11)

Hayawi, K., Shahriar, S., Serhani, M., Taleb, I., y Mathew, S. (2022). *ANTi-Vax: a novel Twitter dataset for COVID-19 vaccine misinformation detection*. Public health, 203, 23-30.

- **Resumen:**

  El desarrollo e introducción de las vacunas contra el COVID-19 ha generado reticencia en la población, ello debido a la información errónea que se distribuye por las redes sociales y otras plataformas, por lo que el presente estudio busca un nuevo enfoque basado en *machine learning* para la detección de información errónea que circula en las diversas plataformas sobre el COVID-19.

- **Base de datos:**

Conformada por 15 073 tweets acerca de la vacuna del COVID-19 (extraídos con el API de Twitter y la librería Twarc) de los cuales 5 751 se etiquetaron como erróneos (*misinformation*), mientas que 9 322 se consideraron tweets generales relacionados con la vacuna (*not misinformation*).

- **Metodología:**
  - **Dataset collection:** La recolección de tweets se hizo mediante el API de Twitter y la librería Twarc. Se definieron las siguientes *keywords* para una mejor búsqueda: vacunas, pfizer, moderna, astrazeneca, sputnik y sinopharm. Solo se consideraron tweets en inglés. Asimismo, las respuestas a tweets, retweets y tweets citados no fueron considerados. Se recolectaron un total de 15 465 687 tweets en el período del 1 de diciembre de 2020 hasta el 31 de julio de 2021.
  - **Data annotation:** El etiquetado del dataset se realizó de manera manual y se recurrió a expertos médicos para validar el correcto etiquetado. Las fuentes de información confiables fueron Public Health, Healthline, the Centers of the Disease Control and Prevention (CDC) y University of Missouri Health Care. Los tweets sarcásticos y humorísticos fueron descartados para el etiquetado. El resultado de esta etapa fueron 15 073 tweets, de los cuales 5 751 fueron etiquetados como erróneos, mientas que 9 322 se consideraron tweets generales relacionados con la vacuna.
  - **Data preprocessing:**

    1.- Remoción de links, puntuación y corchetes de texto.

    2.- Conversión a minúsculas. (COVID → covid)

    3.- Remoción de *stop words*.

    4.- Forma base de las palabras aplicando *steamming*, concretamente, snowball stemmer de NLTK.
  - **Models architecture and implementation:**

    Se trabajaron con 3 categorías de los modelos de *machine learning:* XGBoost (tradicional), LSTM (deep learning) y BERT (transformer). Cada modelo fue entrenado y validado con el 75% del dataset y evaluado con el 25%.

Para XGBoost se aplicó Tf-idf como técnica para la extracción de características. Se utilizó la librería XGBoost.

Para la red LSTM, se aplicó GloVe como técnica de *embedding* para las entradas de la red (capa de *embedding)* luego, le siguió una capa bidireccional de 45 neuronas con GlobalMaxPool1D, posteriormente, 2 capas densas con función de activación ReLU (128 y 32 neuronas respectivamente), una capa de *dropout* de 0.5 y, finalmente, para la capa de clasificación con función *sigmoid.* El modelo se optimizó con la función de Adam, el cual permitió reducir el *log loss* (Binary Cross-Entropy). Se utilizó la libería Keras.

En lo que respecta al modelo BERT, se usó un modelo conocido como *bert-large-uncased*, el cual consiste de 24 capas (1024 dimensiones ocultas), 16 attention heads y 340 millones de parámetros. Se utilizó la librería Transformers.

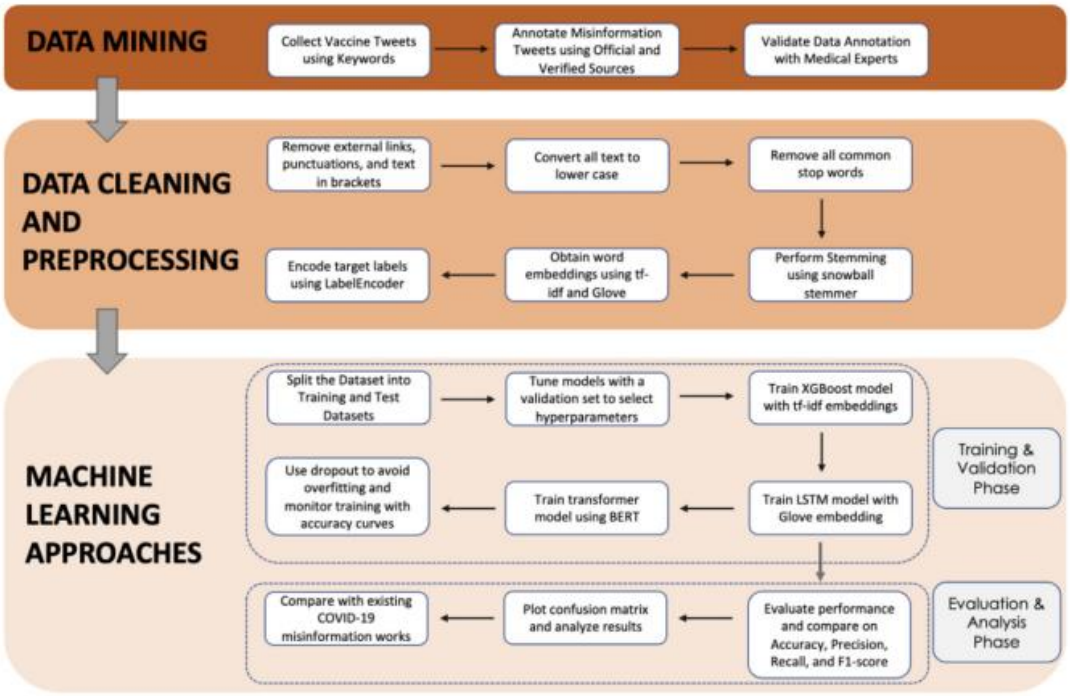


Figura 11. Marco de Investigación.

Fuente: Hayawi et al. (2022). *ANTi-Vax: a novel Twitter dataset for COVID-19 vaccine misinformation detection.* (p. 27)

- **Resultados:**

  Para la evaluación de resultados se utilizaron las métricas de accuracy, precisión, recall y F1-score. Se encontró que el mejor modelo de clasificación se obtuvo utilizando BERT, ya que brindó un accuracy de 0.98, F1-score de 0.98, precision de 0.97 y recall de 0.98. Gracias a estos resultados se puede concluir que los modelos basados en técnicas de *machine learning* son efectivos para detectar información errónea sobre las vacunas contra el COVID-19 en las redes sociales.

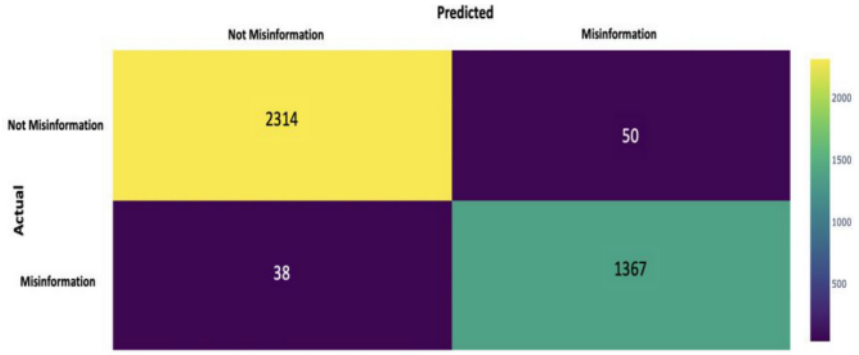


Figura 12. Matriz de confusión en el equipo de prueba usando BERT.
Fuente: Hayawi et al. (2022). *ANTi-Vax: a novel Twitter dataset for COVID-19 vaccine misinformation detection.* (p. 29)

Elhadad, M., Li, K., y Gebali, F. (2021). *COVID-19-FAKES: a Twitter (Arabic/English) dataset for detecting misleading information on COVID-19*. In International Conference on Intelligent Networking and Collaborative Systems (pp. 256-268). Springer, Cham.

- **Resumen:**

  En el contexto de pandemia, mucha información (correcta y errónea) circulaba por twitter por lo que el paper propone la construcción de un dataset que facilite futuras investigaciones respecto a la identificación de información errónea del COVID-19. Asimismo, propone modelos basados en técnicas de *machine*

learning que facilita la anotación de cada tweet. Seguido por un análisis exploratorio para identificar principales *insights.*

- **Base de datos:**

  Se trabajaron con 3 263 464 tweets en árabe e inglés recolectados en un periodo de 36 días (4 de febrero al 10 de marzo de 2020).

- **Metodología:**

  - **Tweets collections:** Para la recolección de tweets se establecieron *keywords* de búsqueda: “Coronavirus”, “Corona_virus”, “Corona-virus”, “Novel_Coronavirus”, “2019-nCoV”, “Novel-Coronavirus”, “NovelCoronavirus”, “2019_nCoV”, “nCoV”, “COVID-19”, “SARS-CoV-2”, “covid19”. Se utilizó la opción de streaming en tiempo real de la API de Twitter para la recolección, la cual se llevó a cabo desde el 4 de febrero de 2020 hasta el 10 de marzo de 2020. Se recolectaron 5 224 912 tweets en 66 idiomas diferentes, los cuales fueron almacenados en una base de datos MySQL. Para el studio solo se consideraron tweets en inglés y árabe (3 263 464 tweets).
  - **Annotation process:** Para esta etapa fue necesario recolectar información correcta del COVID-19 de diversas fuentes confiables, páginas webs y cuentas de twitter oficiales de WHO, UNICEF y UN. Sumado a ello, se utilizaron fact-checkers para contrastar la información. Asimismo, esta fase se dividió en 2 partes (Figura 13): Fase de construcción del modelo y la fase de anotación. En la primera se construyó el modelo de clasificación usando 13 algoritmos de *machine learning* (árboles de decisión, KNN, regresión logística, SVM, multinomial naive bayes, bernoulli naive bayes, perceptrón, red neuronal, random forest ensamblado, XGBoost, bagging meta-estimator, adaboost y gradient boosting) y 7 técnicas de *feature extraction* (tf, tf-idf (unigram, bigram, trigram, N-gram, carácter level) y Word embedding. Mientras que, en la segunda, se trató del etiquetado de los tweets. En ambos casos, la data de cada fase pasó por el paso de preparación, preprocesamiento y *feature engineering* (*feature selection* y *feature extraction*).

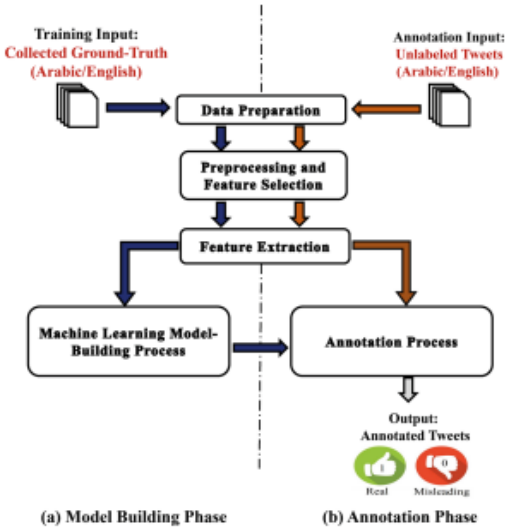


Figura 13. Sistema de anotación.

Fuente: Elhadad et al. (2021). *COVID-19-FAKES: a Twitter (Arabic/English) dataset for detecting misleading information on COVID-19.* (p. 260)

- **Exploratory Data Analysis (EDA):**

  Esta etapa permite un entendimiento más profundo de la data, es decir, encontrar *insight* en los tweets recolectados. Para ello se utilizó la libería Plotly y Bokeh de Python, y constó de los siguientes pasos:

  - Remover la columna Id.
  - Remover las filas donde no exista el texto del tweet.
  - Se realizó la limpieza de los tweets: para los tweets de inglés, se utilizaron expresiones regulares para quitar palabras que no sean de inglés y quitar caracteres simbólicos.
  - Se usó la librería TextBlob y TextBlow_ar para crear una nueva característica de sentimientos de cada tweet (Tweets' sentiments) en inglés y árabe respectivamente.
  - Se creó una nueva característica (TweetCountry) que identifica el país de cada tweet basado en su localización.
  - Se creó una nueva característica (UserCountry) que corresponde al usuario y el país al que pertenece.

- Se creó una nueva característica basado en la longitud del texto del tweet.
- Se creó una nueva característica basado en la cantidad de palabras que tiene el texto de cada tweet.
- Se crearon 2 nuevas características, una para la fecha y a otra para la hora y el día en que se publicó cada tweet.

- **Resultados:**

Si bien el estudio no menciona cuál es el mejor modelo de clasificación que utilizó para la fase de anotación, un aporte fundamental que brinda es el EDA, ya que con este análisis se pudo identificar *insights*:

- Se encontraron 3 047 026 tweets en inglés publicados por 993 320 usuarios y 276 774 tweets árabes publicados por 112 340 usuarios.
- La mayoría de tweets son opiniones neutrales con una tendencia a lo positivo. Mientras que hay muy pocos tweets que sean muy positivos o muy negativos.
- Los usuarios más activos son de los países de EE.UU. y Arabia Saudita.
- Las URL y la codificación HTML afectaban el recuento de caracteres, por lo que fue necesario una limpieza de los mismos (eliminar el ruido).
- Algunos usuarios prefieren escribir tweets largos; sin embargo, la mayoría de los tweets son cortos. (Analizando la longitud y cantidad de palabras)
- Los *stop words* encontrados tenían alta frecuencia por lo que fue necesario su eliminación puesto que afectaba el análisis.

Mahlous, A., y Al-Laith, A. (2021). ***Fake news detection in Arabic tweets during the COVID-19 pandemic*****. International Journal of Advanced Computer Science and Applications (IJACSA)**

- **Resumen:**

Debido a la pandemia, muchos posts acerca del COVID-19 circulaban por las redes sociales, estos posts fueron compartidos sin verificar su fiabilidad, por lo

que el estudio buscó afrontar este problema construyendo un clasificador de *fake news* en tweets árabes (publicados durante la pandemia) aplicando algoritmos de *machine learning*.

- **Base de datos:**

  Se trabajaron con 37 000 tweets en árabe recolectados en un periodo de 36 días (4 de febrero al 10 de marzo de 2020) mediante el API de Twitter.

- **Metodología:**

  - **Data Collection:** Se preparó una lista de *hashtags* que eran comúnmente colocadas en los tweets de acerca de COVID-19: "Corona", "Corona_virus", "New Corona", "COVID-19", "Home Quarantine", "Social Distancing", "We are all responsable", "Stay in your homes". En esta etapa se recolectaron más de 7 millones de tweets árabes relacionados al COVID-19 desde el 1 de enero hasta el 31 de mayo de 2020; sin embargo, después de aplicar algunos filtros (eliminar tweets repetidos y cortos) quedaron 5,5 millones de tweets. Las características extraídas de cada tweet fueron el id, texto, fecha y likes.
  - **Fake news keywords extraction:** Para definir la lista de *keywords* a utilizar, se tomaron como referencia 2 fuentes de información:
    - Agence France-Presse (AFP): Agencia cuya responsabilidad es manejar grandes cantidades de *fake news* en varios idiomas e indicar su error o inexactitud.
    - The Anti-Rumours Authority (No Rumours): Un proyecto creado en 2012 con la finalidad de contener los rumores y evitar que causen algún daño a la sociedad.

    Después de contrastar la desinformación que circulaba en redes sociales con estas 2 fuentes, se definió una lista de 40 *keywords* relacionados a rumores, racismo, *fake news,* curas no verificadas e información falsa, estas sirvieron como filtro de tweets relevantes para el estudio, pues, solo 37 000 tweets contenían algunas de estas *keywords,* los cuales conformaron el *corpus* del modelo.
  - **Data Pre-processing:**

  - Remover menciones, links y hashtags.
  - Remover palabras no árabes y raras.
  - Normalización del texto.
  - Remover puntuación y diacríticos árabes.
  - Remover caracteres repetidos porque crean ruido e influyen en el proceso de minado de texto.
  - Aplicar *stemming* (ISRIStemmer4) y *rooting* (Tashaphyne)

- **Corpus annotation:** La anotación (etiquetado) de cada tweet se realizó de 2 formas:
  1- Manual: 2500 tweets se anotaron manualmente, y, para agilizar el proceso, se construyó una aplicación web en la que se ingresaba el tweet y se elegía una de las 3 opciones: fake, real, next. Solo se anotaron aquellos tweets que eran claramente distinguibles entre falsos y reales, los que no lo eran, se dejaron de lado (next). Resultado de esto se obtuvieron 1 537 tweets etiquetados (835 fake y 702 real).
  2- Automática: Se construyó un clasificador basado en la data etiquetada manualmente. El resultado fue 34 529 tweets etiquetados (19 582 fake y 14 947 real)
- **Model development:** Para el desarrollo del modelo (anotación automática) se tomaron en cuenta 4 técnicas de *feature extraction:* Count Vector, Word-Level TF-IDF, N-Gram-Level TF-IDF y Character-Level TF-IDF. Asimismo, se probaron 6 algoritmos de *machine learning:* Naive Bayes (alpha=0.5), Regresión Logística (valores por default), SVM (c=1.0, kernel=linear, gamma=3), Multilayer Perceptron (función de activación ReLU, iteraciones máximas=30, learning rate=0.1), Random Forest (valores por default) y XGB (valores por default). Asimismo, se dividió la data en 80% y 20% para entrenamiento y prueba, respectivamente y cross validation de 5.

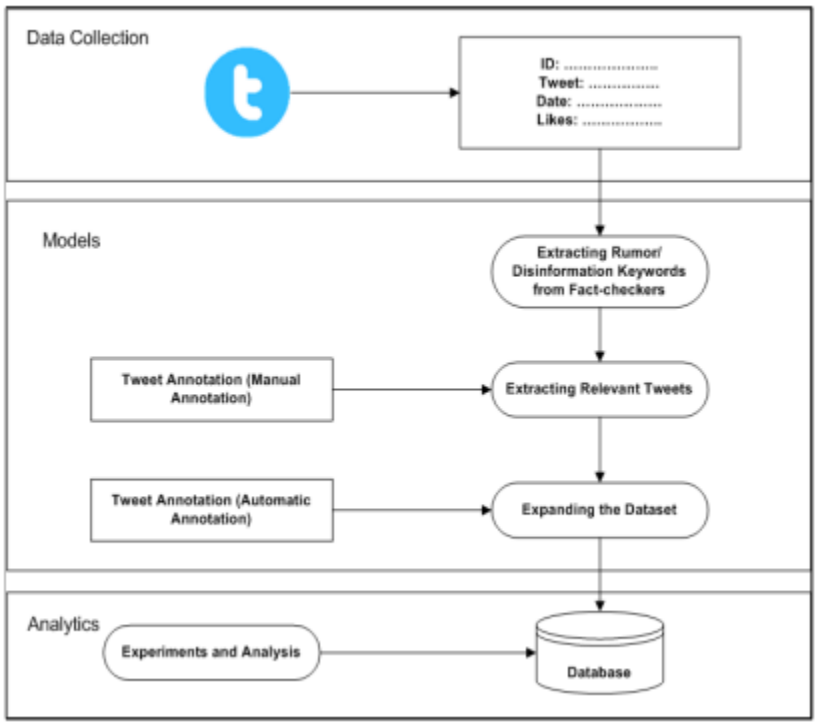


Figura 14. Arquitectura de detección de noticias falsas.

Fuente: Mahlous et al. (2021). *Fake news detection in Arabic tweets during the COVID-19 pandemic.* (p. 778)

- **Resultados:**

Para la evaluación de resultados se utilizaron las métricas de precision, recall y F1-score.

Feature		Word Count			TF-IDF (word-level)			TF-IDF (n-gram-level)			TF-IDF (character-level)		
Classifier	Measure	Raw Text	Stemming	Rooting	Raw Text	Stemming	Rooting	Raw Text	Stemming	Rooting	Raw Text	Stemming	Rooting
NB	P	77.4	78.1	75.65	82.61	80.9	78.4	80.4	85.8	82.07	83.8	83.3	77.72
	R	77.1	77.5	74.68	73.38	75.6	70.13	74	77.5	73.38	72.7	74.9	69.16
	F1	77.2	77.6	74.98	75.13	76.7	71.9	75.5	**78.9**	75.06	75.2	76.5	71.04
LR	P	84	79.9	79.8	82.1	74	81.4	87.8	76	80.9	81.3	68.7	78
	R	84	79.8	79.9	81.8	74	81.2	87.7	76	79.9	80.5	68.2	77.3
	F1	84	79.9	79.9	81.5	74	81.2	**87.8**	76	80.1	80.2	68.3	77.5
SVM	P	80.8	76.3	79.2	78.5	79.9	82.6	85.7	81.6	86.1	80.6	78.1	76.7
	R	79.9	76	79.2	78.6	79.9	82.5	85.7	81.2	85.7	80.5	77.9	76
	F1	80.1	76.1	79.2	78.4	79.9	82.4	**85.7**	81	**85.7**	80.3	77.7	76
MLP	P	83.6	78.64	78.37	78.7	76.91	76.21	80.4	78.22	79.01	86.4	77.14	77
	R	83.1	78.57	77.92	78.6	75.65	75.97	78.6	78.25	75.32	86.4	77.27	76.62
	F1	83.2	78.6	78.1	78.6	76.06	76.09	78.9	78.23	76.06	**86.4**	77.15	76.79
RF	P	81.16	80.44	79.21	74.96	74.44	78.31	77.45	75.35	77.43	79.39	73.79	75.92
	R	77.6	79.55	78.57	75	74.03	78.25	77.27	74.68	75.65	79.22	73.38	75
	F1	78.27	**79.69**	78.73	74.96	74.12	78.27	77.33	74.81	76.04	79.28	73.47	75.23
XGB	P	74.99	78.2	72.08	73.26	79.53	73.34	79.77	77.39	75.82	76.59	76.66	74.05
	R	74.03	76.95	70.78	71.75	76.95	73.05	77.6	75	75	75.97	75	73.05
	F1	74.26	77.23	71	72.11	77.44	73.11	**78**	75.46	75.13	76.13	75.35	73.21

Figura 15. Precision (P), recall (R), y F1-score (F1) (conjunto de datos anotados manualmente)

Fuente: Mahlous et al. (2021). *Fake news detection in Arabic tweets during the COVID-19 pandemic.* (p. 782)

Feature		Word Count			TF-IDF (word-level)			TF-IDF (n-gram-level)			TF-IDF (character-level)		
Classifier	Measure	Raw Text	Stemming	Rooting	Raw Text	Stemming	Rooting	Raw Text	Stemming	Rooting	Raw Text	Stemming	Rooting
NB	P	76.7	75.6	72.6	76.2	76	74	74.4	75.9	75.1	72.8	73.6	70.1
	R	76.4	75.5	72.5	76.2	74.8	69.4	74.4	76	75	72.8	73.6	70
	F1	**76.6**	75.4	72.4	76.2	75.1	70.5	74.4	75.9	74.9	72.8	73.6	70
LR	P	93.4	84.6	76.3	91.8	85.8	77.1	90.8	85.8	79.7	84.3	83.1	76.9
	R	93.3	84.6	76	91.7	85.8	77	90.7	85.7	79.6	84.2	83.1	76.9
	F1	**93.3**	84.6	76.1	91.7	85.8	77.1	90.7	85.7	79.6	84.3	83.1	76.9
SVM	P	92.1	82.9	78.3	91.2	84.2	79.8	90.6	85.4	82	89.7	83.8	76
	R	92	82.8	78	91.2	84.2	79.6	90.5	85.3	81.8	89.1	83.1	75.1
	F1	**92**	82.8	78	91.2	84.2	79.7	90.5	85.3	81.9	89.4	83.3	75.4
MLP	P	88.8	79.6	70.1	87.1	77	70.4	71.7	73.2	72.2	80.5	78	72.3
	R	88.5	79.5	70	87.1	77	70.4	71.7	73.2	72.2	80.5	78	72.3
	F1	**88.6**	79.5	70.1	87.1	77	70.4	71.7	73.2	72.2	80.5	78	72.3
RF	P	84.9	82.5	77.7	84.6	82.1	77.7	84.9	81.5	78.3	78.3	79	76.1
	R	84.7	82.3	77.5	84.7	82.1	77.6	84.9	81.6	78.3	78.3	78.9	76.1
	F1	84.7	82.4	77.6	84.6	82.1	77.6	**84.9**	81.5	78.3	78.3	78.9	76.1
XGB	P	82.8	82.1	76.2	82.8	81.7	75.8	82.3	82	76.3	80.1	80.1	76.5
	R	80.2	80.4	74.7	80.9	80.5	75.2	79.9	80.4	75.2	79.3	79.3	75.8
	F1	**80.7**	**80.7**	75.1	81.2	**80.7**	75.3	80.4	80.7	75.5	79.5	79.5	76

Figura 16. Precision (P), recall (R), y F1-score (F1) (conjunto de datos anotados automáticamente).

Fuente: Mahlous et al. (2021). *Fake news detection in Arabic tweets during the COVID-19 pandemic.* (p. 782)

Asimismo, la gráfica siguiente muestra que el mejor modelo, de acuerdo a la precision, fue la Regresión Logística con la técnica de Word Count y sin aplicar *stemming* y *rooting* puesto que ambas fallaron para mejorar los resultados de la clasificación, ello se debe a que los tweets contienen varios errores dialécticos y de lenguaje.

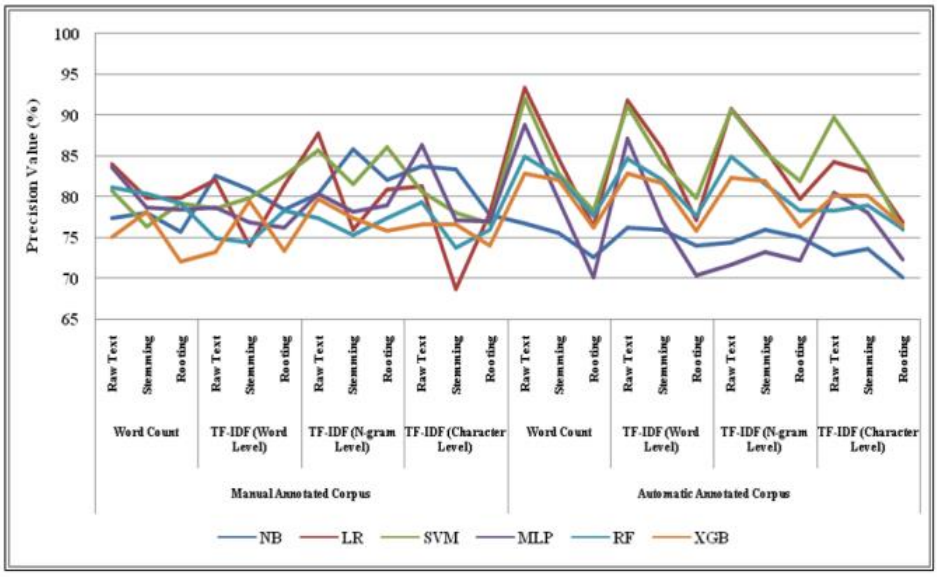


Figura 17. Resultados de la métrica Precision.

Fuente: Mahlous et al. (2021). *Fake news detection in Arabic tweets during the COVID-19 pandemic.* (p. 783)

Cui, L., y Lee, D. (2020). *CoAID: Covid-19 healthcare misinformation dataset.* arXiv preprint arXiv:2006.00885.

- **Resumen:**

  Puesto que la desinformación acerca del COVID-19 abunda en todo el mundo y esta puede causar confusión en las personas, crear conflictos en la sociedad y consecuencias en la salud, el estudio propone la construcción de un dataset con diversas desinformaciones extraídas de páginas web y redes sociales, ello con el fin de ayudar a los investigadores a combatir este problema de desinformación entre los ciudadanos.

- **Base de datos:**

  El dataset que construyó el estudio se llama CoAID, el cual cuenta con 20 097 artículos de información falsa y 281 080 de información verdadera recolectadas desde el 1 de diciembre de 2019 hasta el 1 de setiembre de 2020.

- **Metodología:**

  - **Dataset Construction:**

    1.- Hechos y desinformaciones en la web: Las *keyword* buscadas fueron COVID-19, coronavirus, neumonía, gripe, confinamiento, quedarse en casa, cuarentena y ventilador.

    a) Artículos de noticias: Se identificaron 6 fact-checkers de los cuales obtuvieron las URL de las publicaciones falsas: LeadStories, PolitiFact, FactCheck.org, CheckYourFact, AFP Fact Check and Health Feedback, mientras que, para la recolección de información verdadera, se tomaron en cuenta 9 fuentes de información confiable: Healthline, ScienceDaily, NIH, MNT, Mayo Clinic, Cleveland Clinic, WebMD, WHO y CDC. Una vez

obtenidas las urls, se utilizó la librería Newspaper3k de Python para la extracción del título, el contenido, resumen y *keywords* de cada noticia.

b) Comentarios: Se trabajaron con los comentarios en las publicaciones de la cuenta oficial de Twitter de la WHO y MNT.

2.- Participación del usuario: Mediante el API de Twitter se obtuvo datos acerca de la participación del usuario en tweets correctos como erróneos. Para ello, se buscaron los títulos de las noticias en un determinado período y se recopilaron los tweets que discutían sobre ese tema en dicho período. Los datos extraídos fueron id del usuario, tweets, respuestas, favoritos, retweets y localización. En total, se obtuvieron 10 439 tweets sobre noticias falsas y 141 652 tweets de noticias verdaderas, asimismo, 484 tweets fueron de afirmaciones falsas y 8 092 de afirmaciones verdaderas.

3.- Redes sociales: Se recolectaron publicaciones realizadas en 5 redes sociales: Facebook, Twitter, Instagram, Youtube y TikTok, las cuales se contrastaron con los sitios de chequeos para determinar su veracidad.

	Fake		True	
	Claim	News Article	Claim	News Article
# Information on Website	28	204	454	3,565
# Tweets	484	10,439	8,092	141,652
# Replies	626	7,436	12,451	114,820
# Social Platform Posts	650 (880)	-	42 (46)	-
# Total	1,788 (2,018)	18,079	21,039 (21,043)	260,037

Figura 18. Estadísticas de CoAID.

Fuente: Cui y Lee (2020). *CoAID: Covid-19 healthcare misinformation dataset.* (p. 5)

- **Dataset Analysis:** Esta etapa se dividió en 2 partes:

1.- Análisis prescriptivo: En esta fase se utilizó la librería VADER para realizar un análisis de sentimientos de los usuarios relacionados a la veracidad y falsedad del artículo. Se excluyeron los sentimientos neutrales y se encontró que los tweets relacionados a noticias falsas son más

negativos y tienen polaridad de sentimiento más fuerte que los relacionados a noticias verdaderas. Asimismo, se analizaron los 30 *hashtags* más repetidos por cada clase y no se notaron muchas diferencias, salvo que en los tweets de artículos verdaderos suelen usar *hashtags* de salud (#healtcare, #health), mientras que, en los falsos, se usan de diversos tópicos, entre ellos, los conspiracionales (#bioweapon) y curas falsas (#vitaminic).

2.- Detección de información errónea: En esta fase se procedió a la construcción del modelo de *machine learning*. Se aplicaron las siguientes técnicas:

- **SVM:** Bag of Words, kernel linear
- **Regresión Logística:** Todos los Word Embedding juntos
- **Random Forest:** Bag of Words
- **CNN:** Word Embedding fue la primera capa de convolución (Convolution1D). Seguido de una capa de max pooling
- **BiGRU:** Valores por default
- **CSI:** Doc2Vec como embedding
- **SAME:** Valores por default
- **HAN:** Valores por default
- **dEFEND:** Valores por default

Todos los modelos fueron implementados con Keras, donde se utilizó el 75% para entrenamiento y 25% para prueba. Para las dimensiones ocultas se colocó un tamaño de 100 y se inicializaron los embeddings con la técnica GloVe con tamaño del vector igual a 100. Asimismo, para los modelos de *deep learning*, se utilizó algoritmo de optimización Adam y minibatch de 50 con 10 épocas y la función de pérdida Cross-Entropy-Loss. Cada modelo se corrió 5 veces y los resultados se promediaron.

- **Resultados:**

Para la evaluación de resultados se utilizaron las métricas de AUC, precision, recall y F1-score. Los resultados mostraron que las técnicas modernas funcionaron mejor para la clasificación que los métodos simples; sin embargo, en lo que respecta a los valores de recall y F1 aún siguen siendo malos, ello probablemente al desbalanceo de la data.

	PR AUC	Precision	Recall	F1
SVM	0.3365	0.4036	0.1322	0.1986
LR	0.2871	0.4287	0.0690	0.1143
RF	0.3937	0.6056	0.0581	0.1045
CNN	0.8126	0.9653	0.1238	0.1983
BiGRU	0.2241	0.7476	0.0524	0.0930
CSI	0.3576	0.6814	0.2109	0.2283
SAME\v	0.7901	0.8922	0.2991	0.3400
HAN	0.6824	0.6965	0.4659	0.5471
dEFEND	0.7229	0.8965	0.4847	0.5814

Figura 19. Desempeño de detección de información errónea

Fuente: Cui y Lee (2020). *CoAID: Covid-19 healthcare misinformation dataset.* (p. 9)

2.1.2 Tabla resumen de artículos

Tabla 1.
Resumen de antecedentes de la investigación.

Tipo	Nro.	Antecedente	Autor (es)	País	Año
Problema	1	Detecting misleading information on covid-19	Elhadad, M., Li, K., y Gebali, F.	Canadá	2020
	2	Machine Learning to Identify Fake News for COVID-19.	Isaakidou, M., Zoulias, E., y Diomidous, M.	Grecia	2021
Propuesta	3	Detection of fake news on CoViD-19 on Web Search Engines.	Mazzeo, V., Rapisarda, A., y Giuffrida, G.	Italia	2021
	4	ANTi-Vax: a novel Twitter dataset for COVID-19 vaccine misinformation detection.	Hayawi, K., Shahriar, S., Serhani, M., Taleb, I., y Mathew, S.	Emiratos Árabes	2022
Técnica o herramienta	5	Machine Learning Approach to Detect Fake News, Misinformation in COVID-19 Pandemic.	Bojjireddy, S., Chun, A., y Geller, J.	EE.UU.	2021

	6	Fake news detection in Arabic tweets during the COVID-19 pandemic.	Mahlous, A., y Al-Laith, A.	Arabia Saudita	2021
Sector o Escenario	7	CoAID: Covid-19 healthcare misinformation dataset.	Cui, L., y Lee, D.	EE.UU.	2020

Fuente: Elaboración propia

2.2 Bases Teóricas

2.2.1 Inteligencia Artificial

En lo que respecta a Inteligencia Artificial (IA), existen diversidad de conceptos propuestos por expertos en la materia; sin embargo, nos enfocaremos en el siguiente: "El arte de desarrollar máquinas con capacidad para realizar funciones que cuando son realizadas por personas requieren de inteligencia" (Kurzweil et al, 1990). Asimismo, este experto menciona que gracias a la IA habrá una revolución en términos de conocimiento, por lo que ve el futuro como una explosión de conocimiento humano, refiriéndose a esto como una explosión en los diversos campos, como en la música, arte, cultura, ciencia, ingeniería, entre otros. (BBC, 2014). Pero no todos los autores piensan que la IA tendrá un impacto positivo en la vida de los humanos en el futuro, pues también existen grandes discusiones acerca de las posibles amenazas que esta puede presentar para la humanidad, figuras públicas como Bill Gates piensan que la IA podría representar una amenaza: "Con el paso de algunas décadas la Inteligencia Artificial será lo suficientemente fuerte como para convertirse en un desafío. [...] no entiendo cómo hay personas que no sienten preocupación" (diarioti, 2015). Elon Musk también es consciente de los peligros de la IA: "Hasta que la gente no vea a los robots matar a personas por la calle no se entenderán los peligros de la Inteligencia Artificial." (EL PAIS, 2017). Sin embargo, Mark Zuckerberg no piensa lo mismo: "En los próximos 10 años, la Inteligencia Artificial aportará muchas mejoras a nuestra calidad de vida", asimismo, añade "soy especialmente optimista en lo que respecta a la inteligencia artificial […] creo que hay quienes son pesimistas y tratan de darle bombo a todos estos escenarios apocalípticos. Simplemente, no lo entiendo" (BBC, 2017).

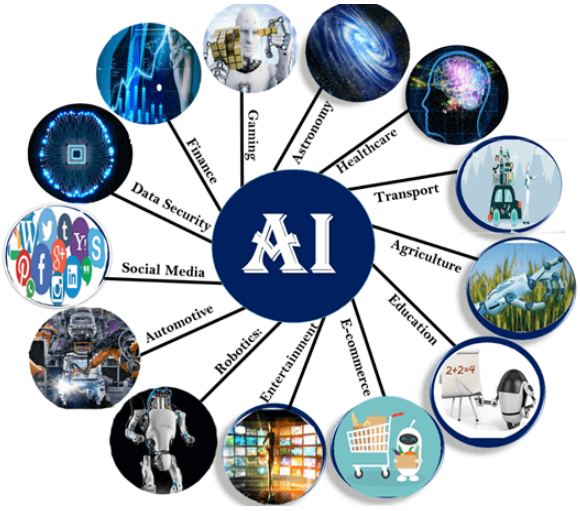


Figura 20. Aplicaciones de la inteligencia artificial.
Fuente: javaTpoint (2021). *Application of AI.* (p. 1)

2.2.1.1 Machine Learning

Mitchell (1997) sostiene que el "Machine Learning aborda la cuestión de cómo construir programas informáticos que mejoren automáticamente con la experiencia."(p. 18). Este concepto, perteneciente al campo de la inteligencia artificial, apareció en respuesta de la inteligencia artificial que solo buscaba que las máquinas puedan automatizar tareas simples imitando el comportamiento humano; sin embargo, con el pasar de los años, el concepto se fue modernizando, ya que Machine Learning no solo busca imitar el comportamiento humano, sino que también se desea que el programa pueda mejorar en base a la experiencia, esto es, que mientras más datos reciba el modelo, este aprenderá mucho mejor, lo que conllevará a resultados más exactos.

2.2.1.1.1 Aprendizaje supervisado

Harrington (2012) define este aprendizaje como el tipo de aprendizaje en el que se le pide a la máquina aprender de los datos entregados de acuerdo a la variable objetiva especificada, es decir, el algoritmo solo debe encontrar algún patrón en los datos de entrada que le conduzcan a la variable de salida. Mientras que autores como Hurwitz y Kirsch (2018) mencionan que "El aprendizaje supervisado tiene como objetivo encontrar

patrones en los datos que se pueden aplicar a un proceso de análisis" (p. 15). En este aprendizaje se presentan 2 casos con respecto a la variable objetivo, si esta es continua, estamos ante un caso de regresión, mientras que, si es nominal, estamos ante un caso de clasificación. Asimismo, en este aprendizaje encontramos problemas de *overfitting* y *underfitting* (Anexo 7), el primero de ellos está referido a que nuestro modelo se ajusta con gran precisión a los datos de entrenamiento pero no puede predecir con datos nuevos, es decir que el modelo ha aprendido a trabajar tan bien con los datos de entrenamiento que solo obtiene un gran puntaje si tiene que predecir algunos de estos datos pero que si se le muestran datos desconocidos, el modelo falla; por otro lado, el problema de *underfitting* es todo lo contrario, es decir que nuestro modelo es tan malo que no ha logrado aprender con los datos de entrenamiento y la predicción es muy mala tanto con datos de entrenamiento como con datos desconocidos. (Hurwitz y Kirsch, 2018)

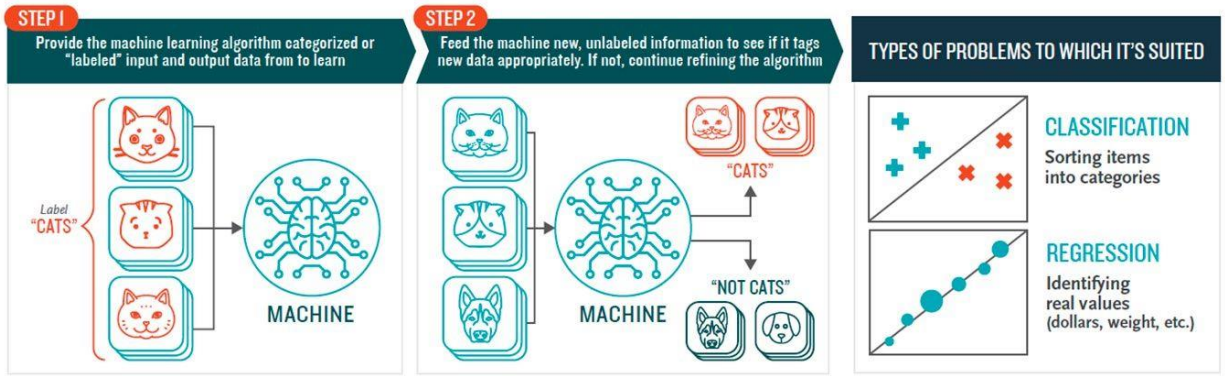


Figura 21. Esquema de aprendizaje supervisado.

Fuente: Cervan, D. (2020). *Guía básica para entender los algoritmos de Machine Learning.* (p. 1)

2.2.1.1.1.1 k-NN (k-Nearest Neighbors)

Este algoritmo de clasificación se encuentra entre los más simples y funciona de la siguiente manera:

Primero, se conoce que todos los datos tienen una etiqueta (clase), luego, se ingresa un nuevo dato desconocido (sin clase) y se compara este dato con los *k* datos más

cercanos existentes, entonces, el nuevo dato tendrá la misma etiqueta que la mayoría de esos datos cercanos (vecinos), es decir, se asigna una clase al nuevo dato mediante el voto mayoritario de los *k* datos más cercanos, cabe mencionar que *k* es un parámetro definido por el usuario y, dependiendo del valor, se puede tener un modelo mejor que otro. (Harrington, 2012)

Ventajas: Alta precisión, insensible a valores atípicos, sin suposiciones sobre los datos.

Desventajas: Computacionalmente caro, por lo que requiere mucha memoria.

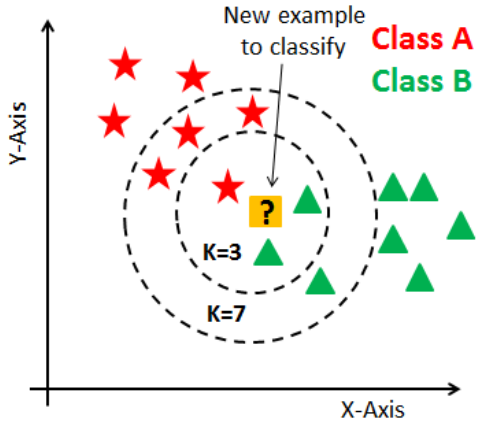


Figura 22. Algoritmo KNN.

Fuente: ICHI.PRO (2021). *K-Vecinos más cercanos (KNN).* (p. 1)

2.2.1.1.1.2 Árboles de decisión

Para Harrington (2012), es una de las técnicas de clasificación más utilizadas debido a que no es necesario conocer a fondo Machine Learning para entender el funcionamiento de este algoritmo. El árbol tiene bloques de decisión (rectángulos) y bloques de terminación (óvalos) donde se ha llegado a alguna conclusión, estos bloques también son conocidos como hojas. Las flechas derechas e izquierdas que salen de los bloques de decisión se conocen como bifurcaciones y pueden conducir a otros bloques de decisión o a un bloque de terminación dependiendo de la decisión tomada.

Ventajas: Computacionalmente barato de usar, fácil de entender los resultados aprendidos, no tiene problema con los valores faltantes, puede lidiar con características irrelevantes.

Desventajas: Propenso a caer en *overfitting*.

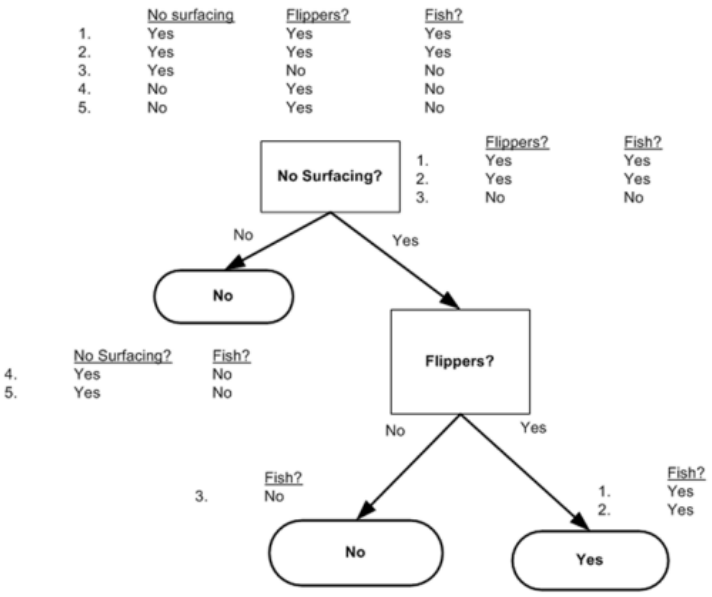


Figura 23. Rutas de datos al dividir.

Fuente: Harrington, P. (2012), *Machine Learning in Action.* (p. 46)

Pero para llevar a cabo la toma de una u otra decisión, el algoritmo utiliza el concepto de *entriopía* que se define como el valor esperado de la información. Para ello, primero se define la información con la siguiente fórmula:

$$l(x_i) = \log_2 p\,(x_i) \tag{1}$$

Donde: p(xi) es la probabilidad de elegir esta clase.

Luego, para el cálculo de la *entriopía* se necesita el valor esperado de toda la información de todos los valores posibles de nuestra clase y para hallar eso se aplica la siguiente fórmula:

$$H = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) \tag{2}$$

Donde: n es el número de clases.

2.2.1.1.1.3 Naive Bayes Classifier

Este algoritmo de clasificación pertenece al subconjunto de la teoría de decisión bayesiana, esta teoría se basa en tomar una decisión en base a un evento conocido anteriormente (Harrington, 2012), es decir, dado un punto (x,y), ¿Cuál es la probabilidad que pertenezca a la clase Ci? Para resolver esta interrogante se aplica la siguiente fórmula:

$$p(c_i|x,y) = \frac{p(x,y|c_i)p(c_i)}{p(x,y)} \quad (3)$$

Ventajas: Funciona con una pequeña cantidad de datos, maneja múltiples clases.
Desventajas: Sensible a la preparación de los datos de entrada.

2.2.1.1.1.4 Regresión Logística

Es una técnica de clasificación que es fácil de usar debido a su bajo costo computacional. Esta técnica utiliza la ecuación *sigmoid* para predecir el resultado, ya que el rango de esta ecuación está entre 0 y 1, y para determinar la clase, se basa en qué tan aproximado está el resultado con respecto a alguno de los extremos, se elegirá como clase aquel extremo más cercano. (Harrington, 2012). La ecuación de esta función es la siguiente:

$$\sigma(z) = \frac{1}{1+e^{-1}} \quad (4)$$

Ventajas: Representación del conocimiento de bajo costo computacional, fácil de implementar, fácil de interpretar.
Desventajas: Propenso a fallar, puede tener poca precisión.

2.2.1.1.1.5 Support Vector Machine (SVM)

Es uno de los algoritmos de clasificación más populares en la actualidad debido a su gran adaptabilidad para problemas de clasificación linealmente separable, semi-separable y no separables linealmente. El SVM toma el clasificador en su forma básica y, al ejecutarlo con los datos, se obtienen resultados con tasas de error bajas. Asimismo, estos algoritmos toman buenas decisiones para los puntos de datos que están fuera del conjunto de entrenamiento. (Harrington, 2012)

Ventajas: Bajo error de generalización, computacionalmente económico, resultados fáciles de interpretar.

Desventajas: Sensible a los parámetros de ajuste y la elección del kernel; de forma nativa solo maneja la clasificación binaria.

A continuación, se observa la técnica aplicada a un problema linealmente separable:

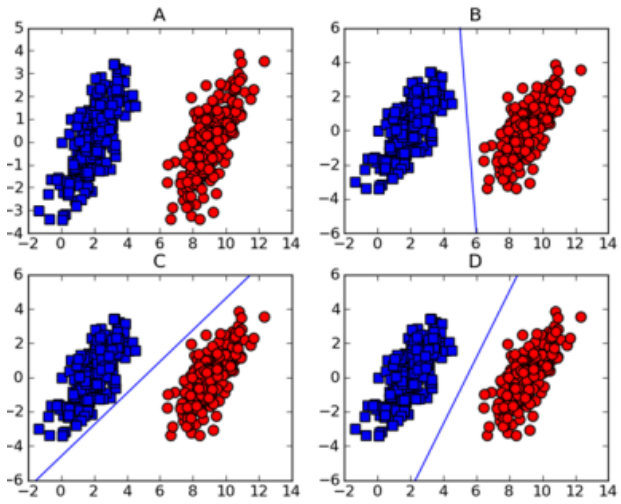


Figura 24. Los datos linealmente separables se muestran en el cuadro A. Los cuadros B, C y D muestran posibles líneas válidas que separan las dos clases de datos.

Fuente: Harrington, P. (2012), *Machine Learning in Action.* (p. 103)

Por otro lado, la siguiente imagen muestra la aplicación de la técnica en problemas no separables linealmente.

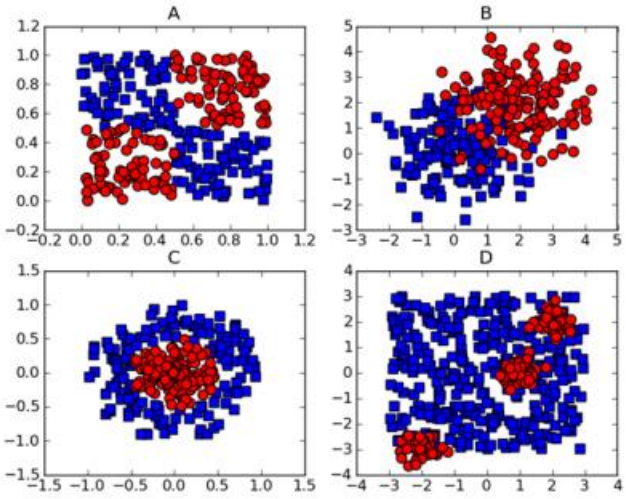


Figura 25. Cuatro ejemplos de conjuntos de datos que no se pueden separar linealmente. Fuente: Harrington, P. (2012), *Machine Learning in Action.* (p. 102)

2.2.1.1.1.6 Redes Neuronales Artificiales

Una red neuronal artificial (RNA) es una máquina diseñada para modelar la forma en que el cerebro realiza una determinada tarea o función. Dicha red cuenta con neuronas en su interior ubicadas en capas que están interconectadas. Las neuronas pertenecientes a una misma capa no se conectan entre sí, pero aquellas pertenecientes a diferentes capas sí lo hacen, de modo que trasladan la información de capa en capa (Haykin, 2009). Cada RNA se caracteriza por tener:

- *Capa de entrada:* En ella se ubican las variables de entradas (*inputs*).
- *Capas ocultas:* En estas capas se ubican aquellas neuronas que no tienen contacto con el exterior; asimismo, cuentan con una función de activación.
- *Capas de salidas:* Esta dada por los valores resultantes (*outputs*) de las diversas combinaciones lineales de las capas ocultas.

Cada capa de la RNA debe tener una función de activación: **tansig, logsig, purelin**, etc.; no es obligatorio que la misma función esté presente en todas las capas, pues se busca tener la mejor red y, para ello, se prueban todas las posibles combinaciones de funciones en las capas de la red, así pues, diferentes capas podrían tener diferentes funciones de activación o quizás, en algunas de ellas son las mismas, siempre y cuando el

modelo obtenido sea el mejor. Otro aspecto importante de una RNA es el algoritmo de aprendizaje o entrenamiento, este algoritmo permite a la red procesar los datos con los que se va a trabajar y aprender de ellos de modo que logre resolver un problema de una forma óptima. Hay numerosos algoritmos de aprendizaje, entre ellos encontramos: Backpropagation, Búsqueda exhaustiva, Algoritmo Hill Climbing, Algoritmos Genéticos, Levenberg-Marquardt, etc. Algunos tipos de redes neuronales artificiales son: Red Neuronal Monocapa (Perceptron simple), Red Neuronal Multicapa (MLP), Redes Neuronales Convolucionales (CNN), Red Neuronal Recurrente (RNN), Redes de Base Radial (RBF), Redes Generativas Adversativas (GANs),etc.

2.2.1.1.1.6.1. Funciones de transferencia de la RNA

Según Haykin (2009), las funciones de transferencia o activación son utilizadas para restringir la amplitud de la salida de la neurona, es decir, estas funciones limitan el intervalo permisible que podría tomar el valor de salida de una neurona. Normalmente, el intervalo permisible es [0; 1] o [-1; 1]. A continuación, se muestran algunas funciones de transferencia que existen:

- ***Escalón**:* Esta función solo toma 2 valores: -1 para valores de salida negativos y 1 para valores de salida positivos. Es comúnmente usada en un perceptron.

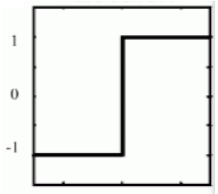


Figura 26. Gráfico de la función escalón
Fuente: Haykin, S. (2009). *Función de umbral*. (p.13)

- ***Lineal (purelin):*** Este tipo de funciones se usan como aproximadores lineales.

$$purelin(n) = n \quad (5)$$

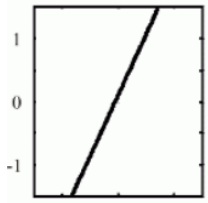


Figura 27. Gráfico de la función lineal
Fuente: Haykin, S. (2009). *Función lineal.* (p.13)

- ***Sigmoidal (logsig):*** Comúnmente se utiliza esta función en redes MLP entrenadas con el algoritmo Backpropagation, pues restringe los valores que van entre]-∞;+∞[a valores entre]0; 1[.

$$logsig(n) = \frac{1}{(1+\exp(-n))} \quad (6)$$

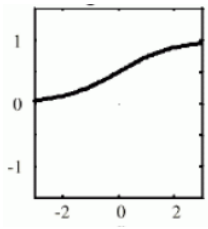


Figura 28. Gráfico de la función sigmoide
Fuente: Haykin, S. (2009). *Función sigmoidea.* (p.13)

- ***Tangente hipérbolica (tansig):*** Se usa esta función en redes neuronales donde la velocidad de procesamiento es más importante que la precisión de la función. Su intervalo va desde]-1;+1[.

$$tansig(n) = \frac{2}{(1+\exp(-2n))} - 1 \quad (7)$$

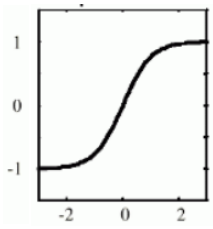


Figura 29. Gráfico de la función tangente hiperbólica

Fuente: Haykin, S. (2009). *Función tangente hiperbólica.* (p.13)

2.2.1.1.1.6.2. El perceptron de Frank Rosenblatt

El perceptron propuesto por Rosenblatt en 1958 es la forma más simple que existe de una red neuronal artificial (RNA), ya que solo cuenta con una neurona a la que llegan determinadas entradas para dar como respuesta una salida, en ella también se puede observar el bias, y, tanto las entradas como el bias tienen sus respectivos pesos los cuales son ajustables. Debido a la sencillez de la estructura, este tipo de red solo se utiliza para clasificar patrones linealmente separables (patrones ubicados en lados opuestos de un hiperplano). Este perceptron se limita a clasificación de patrones que constan de 2 clases; sin embargo, estas clases deben ser linealmente separables, de lo contrario, el perceptron no funcionará correctamente (Haykin, 2009). En la siguiente figura se observa un modelo de perceptron:

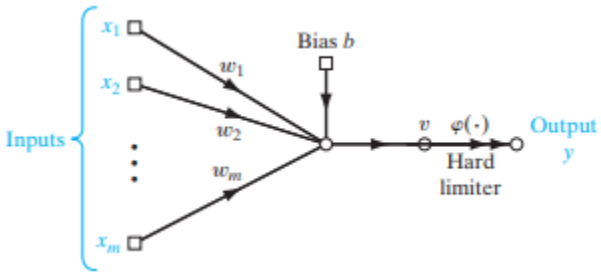


Figura 30. Modelo neuronal perceptron

Fuente: Haykin, S. (2009). *Flujo de señal gráfica del perceptron*. (p.48)

Donde:

Xi: Entradas del perceptron

Wi: Pesos del perceptron

b: Bias o sesgo

y: Salida (1 o -1)

u: Resultado de la ponderación de pesos

φ(.): Función de transferencia

Para lograr obtener la salida "y"; primero, se debe ponderar las entradas de la red con sus respectivos pesos y agregar el bias. En la siguiente ecuación se muestra de manera más detallada:

$$v = \sum_{i=1}^{m} w_i x_i + b \tag{8}$$

A este resultado, se le aplica la función de transferencia, por lo general es la función escalón, esta función solo tiene 2 posibles resultados: 1 y -1, por lo que es conveniente usarla en problemas de clasificación. Al final de ello, el resultado "y" obtenido, puede ser 1 o -1 y, de acuerdo a esto, se puede clasificar en una determinada clase:

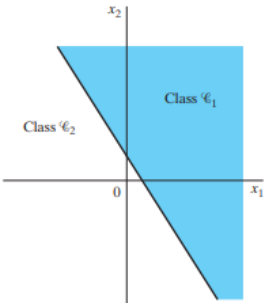


Figura 31. Clasificación en un hiperplano

Fuente: Haykin, S. (2009). *Ilustración del hiperplano como límite de decisión para un bidimensional de 2 clases de problemas de clasificación de patrones*. (p.49)

2.2.1.1.1.6.3. Modelo MultiLayer Perceptron de Paul Werbos

Como se observó que el perceptron era muy limitado, pues solo se aplicaba en problemas de clasificación, Werbos propuso una estructura de red basada en la mejora del perceptrón en 1974. Esta estructura cuenta con una o más capas ocultas, estas capas no tienen contacto con el exterior como sí lo tienen las entradas y las salidas; asimismo, cuenta con capas de salidas en donde se ubican las salidas obtenidas por la red. Dentro de cada capa están ubicadas las neuronas, y a su vez, estas están interconectadas con otras neuronas provenientes de distintas capas; asimismo, cabe mencionar que aquellas

neuronas pertenecientes a la misma capa no se conectan entre sí (Haykin, 2009). La Figura 24 muestra un modelo MLP:

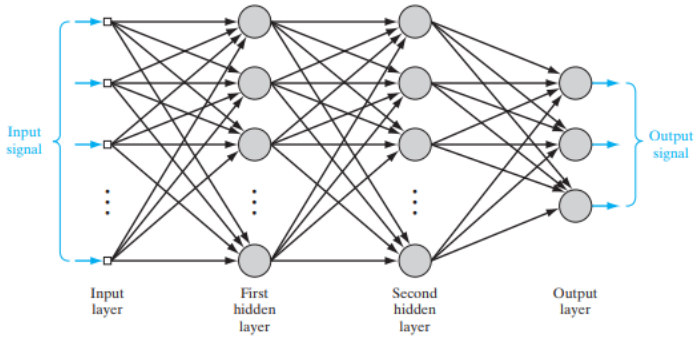


Figura 32. Modelo neuronal MultiLayer Perceptron

Fuente: Haykin, S. (2009). *Gráfico arquitectónico de un perceptron multicapa con dos capas ocultas.* (p.124)

De manera similar al perceptron, el MLP también se base en la siguiente ecuación para ponderar los pesos:

$$v_j(n) = \sum_{i=0}^{m} w_{ji}(n) y_i(n) \tag{9}$$

Donde:

Uj: Resultado de la ponderación de pesos en la neurona *j*

m: Número total de entradas

Wji: Peso que conecta la neurona *i* con la neurona *j*

yi: Salida de la neurona *i*

De igual manera, al aplicar la función de transferencia al resultado *Uj* se obtiene la salida *yj* de cada neurona.

$$y_j(n) = \varphi(v_j(n)) \tag{10}$$

Este tipo de estructura se basa en el perceptron de Rosenblatt, por lo que la ponderación de pesos es parecida pero un poco más compleja puesto que existen muchas neuronas y varias capas en donde se encuentran. Al existir más neuronas, hay mayor

número de interconexiones entre ellas, así pues, poner en práctica el uso de esta red toma más tiempo que poner en práctica el perceptron.

2.2.1.1.2 Aprendizaje no supervisado

A diferencia del aprendizaje supervisado, en este tipo de aprendizaje no se cuenta con una variable objetivo (Y) la cual predecir, solo se tienen los datos de la variable dependiente (X), es por ello que no podemos predecir la variable Y en base a X, lo que se hace en este tipo de aprendizaje es agrupar los datos de la variable X en base a ciertas características, este proceso de agrupamiento se conoce como *clustering* y los grupos obtenidos son llamados *clusters* (Harrington, 2012). Asimismo, según lo mencionado por Hurwitz y Kirsch (2018), en este aprendizaje no conocemos el contexto de los datos que serán analizados, por lo que no se pueden etiquetar dichos datos a diferencia del aprendizaje supervisado. Asimismo, este aprendizaje nos permite abordar los problemas con poca o ninguna idea de cómo deben ser nuestros resultados, por lo que es recomendable utilizarlo como primer paso antes de pasar los datos a un proceso de aprendizaje supervisado. La técnica de clusterización más conocida es *k-means.*

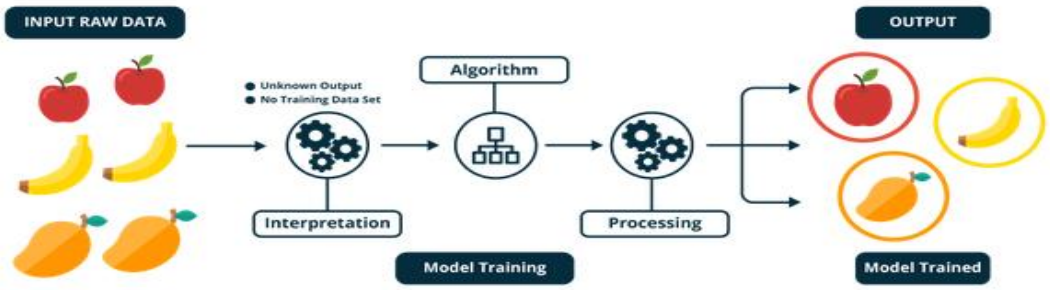


Figura 33. Aprendizaje no supervisado

Fuente: Gonzalez, F. (2019). *Machine Learning: La base que tenes que tener.* (p.1)

2.2.1.1.2.1 K-means

Para Harrington (2012), “k-means es un algoritmo que encontrará k *clusters* para un conjunto de datos dado” (p. 208). Este algoritmo agrupa los datos en base a las

distancias de cada dato con el *centroide* del *cluster*, los cuales han sido inicializados aleatoriamente (entiéndase por *centroide* como el centro de todos los datos del *cluster*) y, para saber cuántos *centroides* el usuario debe definir el valor de k, una vez definido dicho valor, el algoritmo calcula la distancia de todos y cada uno de los puntos hasta el *centroide*, luego, los *centroides* se vuelven a calcular en base a los nuevos puntos del *cluster*, este proceso se repetirá hasta que los *centroides* ya no se muevan. En caso los los *centroides* volvieran a una posición anterior, el algoritmo caería en un bucle infinito, por lo que es necesario definir un número de iteraciones máximas.

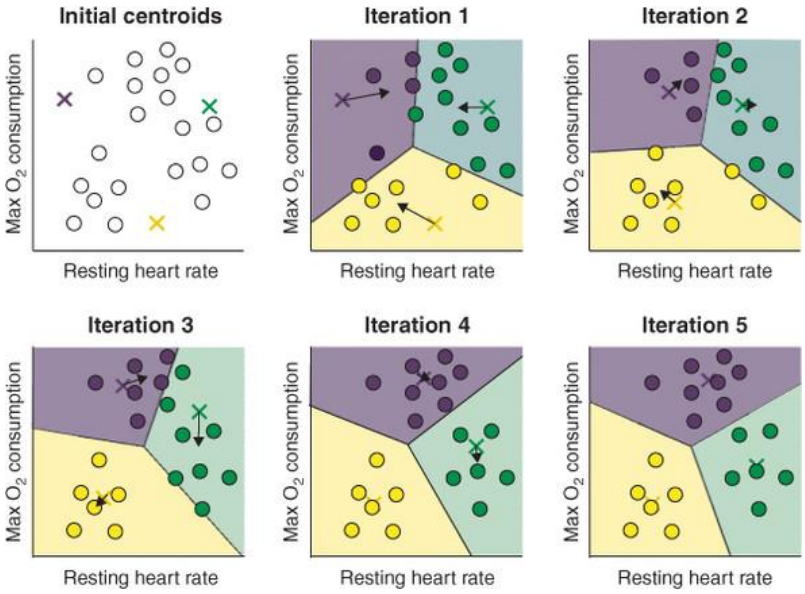


Figura 34. Cinco iteraciones de agrupamiento de k-medias. En la gráfica superior izquierda, tres centros iniciales se generan aleatoriamente en el espacio de características (cruces)
Fuente: Heifin, R. (2020). *Machine Learning with R, the tidyverse and mlr*.

2.2.1.1.3 Aprendizaje por refuerzo

Este aprendizaje es definido como un modelo de aprendizaje basado en la conducta del ser humano, es decir, que mediante la retroalimentación brindada por el usuario, el algoritmo tendrá en cuenta los errores cometidos al momento de analizar los datos y mejorará dicho análisis en la siguiente iteración, por lo tanto, una secuencia de decisiones correctas permitirá un mayor refuerzo del algoritmo pues estará resolviendo

de mejor manera el problema planteado. Según Hurwitz y Kirsch (2018) “El aprendizaje por refuerzo se diferencia de otros tipos de aprendizaje supervisado porque el sistema no está entrenado con el conjunto de datos de muestra. Más bien, el sistema aprende mediante prueba y error” (p. 17). Entre las aplicaciones más comunes de este aprendizaje tenemos la robótica y los juegos; en el caso de la robótica, este se usa para entrenar mejor a los robots, es decir, si se desea que un robot camine por un determinado terreno, este lo intentará una y otra vez, pues es probable que sufra caídas en el intento y, producto de ello, aprenderá cuál es el camino por el que no debe ir; por otro lado, en los juegos se utiliza de una manera similar, ya que se entrena al algoritmo para tomar mejores decisiones al momento de ponerlo a prueba, un ejemplo de ello son los vieojuegos creados por Atari 2600 como Pong, Breakout, Space Invaders, Seaquest, Beam Rider,etc. (Mnih et al, sf).



Figura 35. Diagrama de flujo de aprendizaje por refuerzo
Fuente: Luna, J. (2018). *Tipos de aprendizaje automático.* p. 1

2.2.2 Natural Language Processing (NLP)

Es el campo de la IA encargado de investigar la manera en la que se comunican las máquinas con las personas mediante el uso de lenguas naturales, como el español o el inglés, es decir, esta rama ayuda a las computadoras a entender, interpretar y manipula el lenguaje humano. Asimismo, Covington (2013) menciona que el NLP “es el uso de las computadoras para comprender el lenguaje humano (natural) como el inglés, el francés o el japonés” (pag. 1), también agrega que esto no significa que las máquinas puedan tener

los pensamientos y sentimientos como los humanos, sino que solo son capaz de reconocerlos y expresar dicha información en nuestro lenguaje.

2.2.2.1 Text Mining

Según Witten, Eibe y Hall (2011): "Se trata de buscar patrones en el texto: es el proceso de analizar texto para extraer información que es útil para propósitos particulares." (p. 161). Sin embargo, para Charu y ChengXiang (2012), el text mining va más allá del simple acceso a la información, sino que también busca ayudar a los usuarios a analizar, digerir la información y facilitar la toma de decisiones.

- **Preprocesamiento de la información:** Es necesario para una mejor extracción de características.
    a. *Limpieza:* Quitar información irrelevante para el modelo (imágenes, emojis, urls, etc.). Eliminar StopWord es importante en este paso, ya que estas son palabras que aparecen con mucha frecuencia pero que son insignificantes para la representación de un documento.
    b. *Tokenización:* Asignar una clave a cada palabra del texto.
    c. *Part of Speech Tagging:* Consiste en marcar cada palabra del texto como correspondiente a una parte particular del discurso, según su definición y su contexto.
    d. *Stemming:* Es el proceso de reducir las palabras a su raíz. Una raíz es la parte de una palabra que queda después de eliminar sus prefijos y sufijos (niños, niñito, niñitas -> niño)
    e. *Lemmatization:* Es parecido al stemming pero con la diferencia de que no reduce la palabra a su raíz, sino a su infinitivo (volando, volamos, volé -> volar).
    f. *N-grams:* Son secuencias de palabras cuyo sentido cobra mayor relevancia al estar juntas que separadas (no, te, vacunes -> no te vacunes). En este caso, el parámetro N define cuántas son las palabras relacionadas que se considerarán como una sola. Para ello, el modelo de N-grams

estudia la probabilidad de que dichas palabras se relacionen y vayan juntas. A continuación, se observa el ejemplo de un Trigrama:

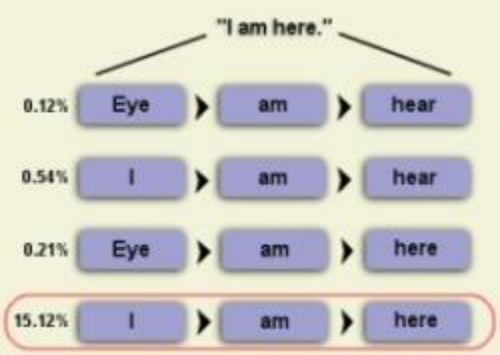


Figura 36. N-gramas de Google

Fuente: argaldo (2006). *Google y sus n-gramas.* (p. 1)

- **Extracción de características:** Es uno de los puntos clave de la minería de texto, ya que sirve como punto de partida para muchos algoritmos de *machine learning.* Extraer las entidades y sus relaciones del texto puede revelar información semántica más significativa en los datos del texto que una simple representación de una bolsa de palabras (BOW). Las técnicas más conocidas son Tf-idf y Word2Vec.

2.2.2.2 Bag of Word (BOW)

Es una manera de extraer las características del texto para usarlas en la construcción del modelado. Es fácil de entender, ya que su representación describe cuántas veces ocurre cada palabra en el texto. La desventaja de esta técnica es que su dimensionalidad es muy alta, ya que toma en cuenta cada palabra del texto, lo que se puede volver un gran problema si se trabaja con grandes cantidades de textos. (Brownlee, 2017)

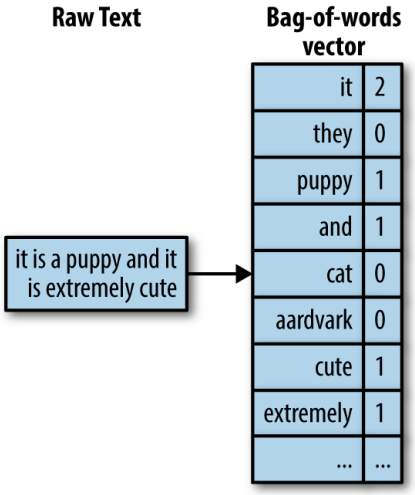


Figura 37. Bag of Words

Fuente: CHandra, R. (2020). *Does TF-IDF work differently in textbooks and sklearn routine?.* (p. 1)

2.2.2.3 Tf-idf (Term frequency - Inverse document frequency)

Es una técnica bastante buena para encontrar el documento más relevante para cierto término dentro de una colección de documentos, pues esta técnica lidia con uno de los problemas principales del BOW, la dimensionalidad, ya que el *corpus* con el que se trabaja solo considera las palabras más relevantes en la colección de documento. Para ello, mide la frecuencia en la que aparece un término o frase dentro de un determinado documento, y lo compara con el número de documentos que mencionan ese término dentro de una colección entera de documentos, de manera que deja de lado aquellas palabras irrelevantes en los documentos (palabras que se repiten en la mayoría de documentos) y solo da importancia a las palabras que aportan mayor información a lo largo del *corpus* (González, 2019). A continuación, se muestra la fórmula:

$$w_{i,j} = tf_{i,j} * \log\left(\frac{N}{df_i}\right) \quad (11)$$

Donde:

tf: Número del término en el documento

df: Número de documentos que contienen el término i

N: Cantidad total de documentos

2.2.2.4 Word2Vec

Si bien, una forma sencilla de representar un documento puede ser mediante vectores *one hot*, este se vuelve un problema cuando se trabajan con gran cantidad de datos, puesto que el costo computacional es mayor, sumado a ello, los vectores resultantes no toman en cuenta la similitud entre palabras, ya que, para esta técnica, aquellas palabras con valor de cero tienen el mismo peso, sin tomar en cuenta que alguna de ellas pueden tener más relación entre sí. Debido a ello surge word2vect (creado en 2013 por Google - Tomas Mikolov), el cual permite capturar similitudes tanto sintácticas como semánticas entre las palabras. Esta técnica utiliza una red neuronal para brindarnos una palabra vecina de la palabra ingresada, ello es posible gracias al entrenamiento de la red realizado mediante un enorme vocabulario de palabras del *corpus.* Para llevar a cabo este entrenamiento, es necesario pasar como parámetro un vector *one hot* (vector representado solo por un único valor de 1 y los demás 0). Es una técnica potente para encontrar las similaridades entre palabras. (Mikolov, 2013)

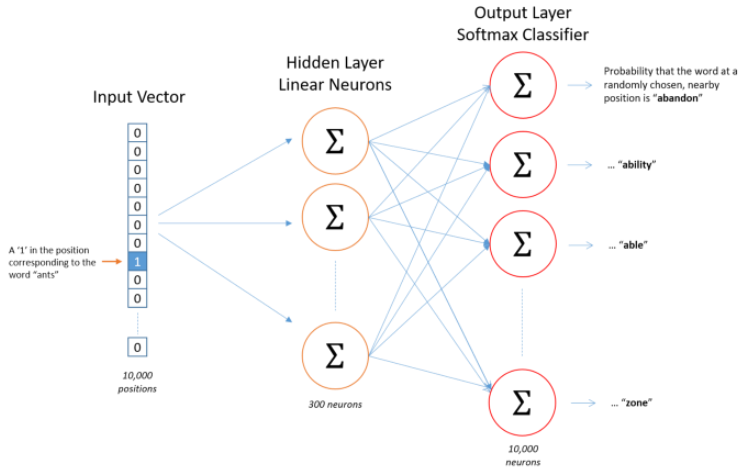


Figura 38. Representación de la arquitectura de la red neuronal
Fuente: Ruiz de Villa, G. (2018). *Introducción a Word2Vec (skip gram model)* (p. 1)

El propósito y la ventaja de word2vec es recopilar vectores de palabras que son iguales en el espacio vectorial, es decir, encontrar similitudes matemáticas (Figura 39). Word2vec genera vectores distribuidos utilizando representaciones numéricas de

elementos de palabras, características como el contexto de palabras individuales. El resultado de la red word2vec es un glosario donde cada entrada está asociada con un vector, que puede integrarse en una red neuronal profunda o simplemente requerirse para encontrar relaciones entre palabras, lo interesante de esta técnica es que, al ser vectores representado vectorialmente en un espacio, se puede realizar operaciones con los mismos encontrando nuevas palabras que se acercan bastante en lo semántico.

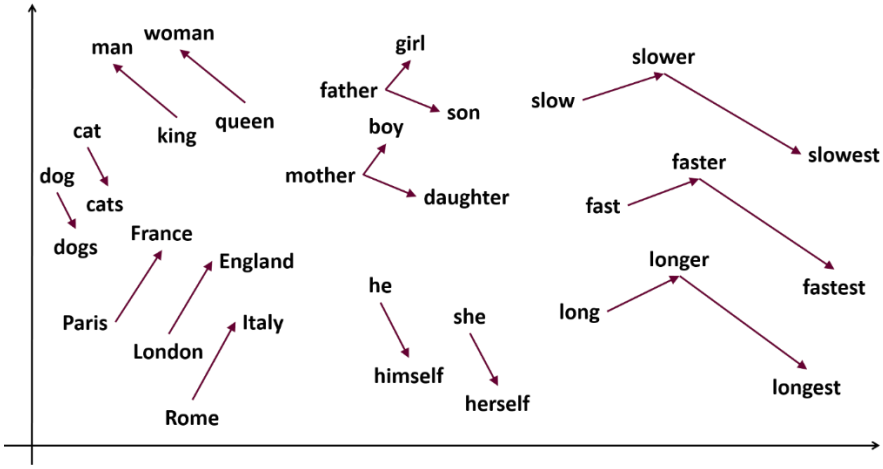


Figura 39. Word2Vec: Las palabras que comúnmente aparecen juntas están más juntas
Fuente: Medium (2018). *Word2Vec for Talent Acquisition* (p. 1)

2.2.2.4.1 Continuous Bag of Words Model (CBOW)

La función objetiva del modelo CBOW es predecir la palabra del medio cuando se le dan N/2 palabras anteriores y N/2 palabras posteriores. El mejor resultado se obtiene usando N=8. En la capa de proyección, los vectores de palabras de N palabras de contexto simplemente se promedian. No hay relevancia de la posición de la palabra para determinar el vector de palabra de la palabra del medio, de ahí el nombre **"Bag of Wods"**. El término **“Continuous”** representa el espacio vectorial D. (Mikolov, 2013)

Se pasa un vector promedio a la capa de salida seguido de un softmax jerárquico para obtener una distribución sobre V. CBOW es un modelo lineal logarítmico simple en el que el logaritmo de la salida del modelo se puede

representar por una combinación lineal de los pesos del modelo. La complejidad del entrenamiento del modelo CBOW se determinan por la siguiente ecuación:

$$Q = N \times D + D \times \log_2(V) \quad (12)$$

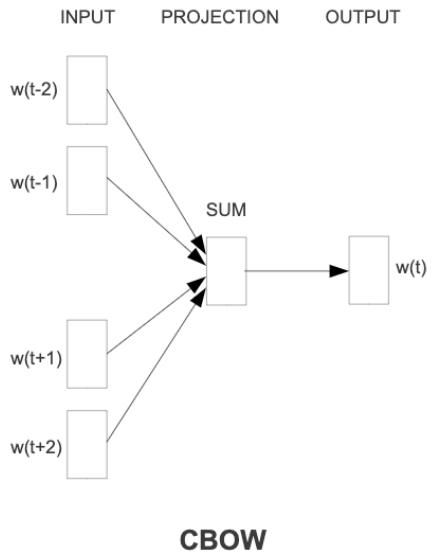


Figura 40. Arquitectura del modelo CBOW.

Fuente: Mikolov et al. (2013). *Efficient estimation of word representations in vector space*. (p. 5)

2.2.2.4.2 Continuous Skip-Gram Model

Este modelo invierte el objetivo de CBOW. Con la palabra presente, predice palabras de contexto vecinas tanto en las anteriores como el de las posteriores. Como sugiere el nombre, el modelo predice palabras de N-grams excepto la palabra actual como entrada al modelo, de ahí el nombre “Skip-Gram”. Se elige que el valor de N sea 10. Dado que las palabras distantes están menos relacionadas con la palabra actual, se muestrean menos que las palabras cercanas para producir las etiquetas de salida. Cuando N = 10, se genera un número aleatorio R del 1 al 10 con la estrategia de muestreo mencionada anteriormente, y las palabras R históricas y futuras se utilizan como etiquetas correctas para el modelo. La complejidad total del modelo está representada en la siguiente ecuación:

$$Q = N \times D + N \times D \times \log_2(V) \quad (13)$$

Tenga en cuenta que N también se multiplica por el término D × log2 (V) porque este no es un problema de clasificación de clase única para CBOW, sino un problema de clasificación de clase N. Por lo tanto, la complejidad general de este modelo es más grande que el de CBOW (Mikolov, 2013).

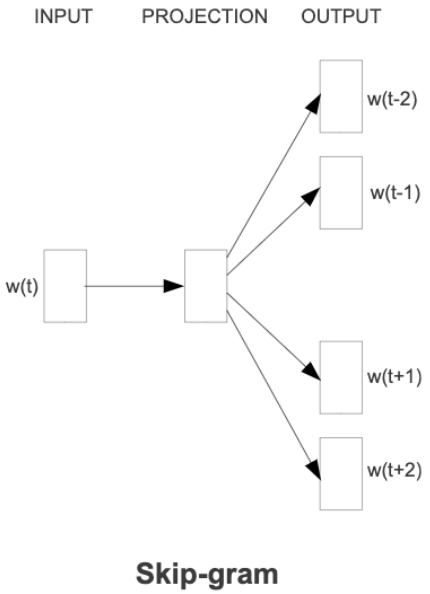


Figura 41. Arquitectura del modelo Skip-gram.

Fuente: Mikolov et al. (2013). *Efficient estimation of word representations in vector space*. (p. 5)

2.2.2.5 FastText

Esta técnica apareció para dar solución al problema principal de Word2Vec: tener en cuenta la morfología de las palabras, ya que, mediante estas, se logra un entrenamiento más eficiente, pues esta técnica considera palabras fuera del vocabulario, lo cual no hace Word2Vec, la representación vectorial de estas palabras mediante la suma de sub-palabras tiene un mejor rendimiento que la asignación de vectores nulos, y proporciona una gran ventaja cuando se trabajan con idiomas ricos en vocabulario como el español o alemán; sin embargo, pierde efectividad cuando se trata de análisis semántico y es más lento de entrenar en comparación con el skip-gram regular debido a la gran cantidad de n-gramas (Bojanowski et. al, 2017).

2.2.2.5.1 Generación de sub-palabras

Para una palabra, se generan n-gramas de caracteres de una determinada longitud (máximo 6). Para ello, primero se le agregan corchetes angulares para indicar el comienzo y fin de la palabra. Luego, se generan n-gramas de longitud n, el resultado será una lista de n-gramas que conforman la palabra. (Figura 42)



Figura 42. 3-grams de la palabra *eating*
Fuente: Chaudhary (2020). *A visual guide to FastText word embedding*. (p. 1)

Puesto que por cada palabra se generan muchos n-gramas, esto supondría un alto espacio en memoria al contar con un vocabulario inmenso, por lo que se utiliza el hash de Fowler-Noll-Vo para convertir secuencias de caracteres en valores enteros que varían desde el 1 hasta 2 100 000, gracias a esto se logra controlar el tamaño del vocabulario.

2.2.2.5.2 Skip-Gram con muestreo negativo

Esta técnica aplica los conceptos del skip-gram vistos en Word2Vec pero con ciertas mejoras. Como cada palabra se compone de n-gramas, entonces el embedding de la misma se calcula tomando una suma de vectores para los n-gramas de caracteres y la palabra completa. Para ello se aplica la siguiente ecuación de puntuación:

$$s(w,c) = \sum_{g \in \mathrm{Gw}} z_g^T v_c \qquad (14)$$

Donde:

w: Palabra

g: n-grama

Gw: Diccionario de n-gramas de la palabra

z_g: Representación vectorial de cada n-grama

v_c: Vector de contexto

Para las palabras de contexto reales, se toman sus vectores de embedding sin la necesidad de agregar los n-gramas. Luego, se recolectan muestras negativas al azar con proporción de probabilidad a la raíz cuadrada de la frecuencia del uni-gram, por cada palabra de contexto real se toman como muestra 5 palabras negativas aleatorias. Posteriormente, se realiza el producto escalar entre la palabra central y las palabras de contexto real, aplicando una función *sigmoid* al resultado para obtener valores entre 0 y 1. (Figura 43)

Finalmente, para la función de pérdida se actualizan los vectores con el optimizador SGD, con la finalidad de acercar las palabras de contexto reales a la central, a su vez que alejan de las muestras negativas.

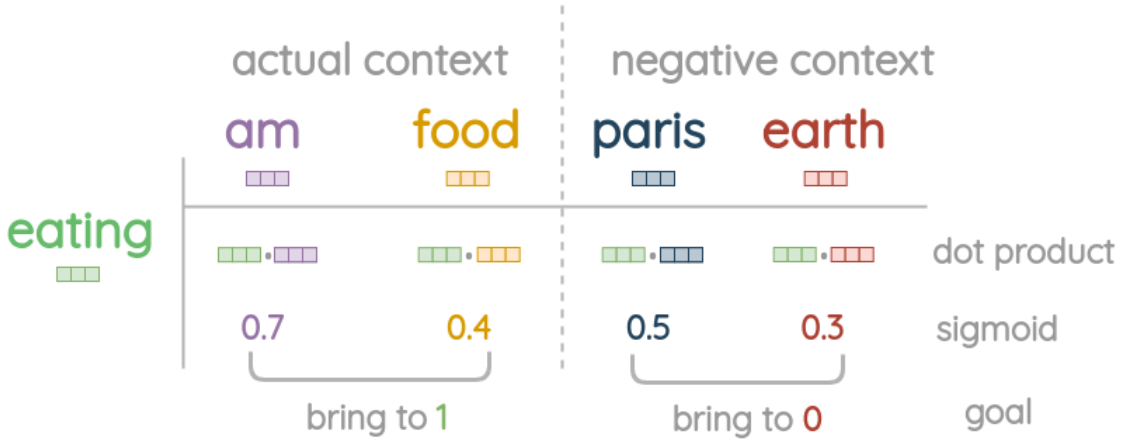


Figura 43. Skip-gram aplicado a la palabra *eating*

Fuente: Chaudhary (2020). *A visual guide to FastText word embedding*. (p. 1)

2.2.3 Información falsa

Según White (s.f.), director de la Red de Periodismo Ético (EJN), la información falsa o truncada es toda información elaborada y publicada deliberadamente con la finalidad de engañar e inducir a terceros a creer mentiras o poner en duda hechos verdaderos. Asimismo, la Organización Panamericana de la Salud (OPS, 2020) define

que la información falsa es el tipo de información cuyo propósito es engañar a los lectores, que puede llegar a perjudicar la salud física y mental de las personas, incrementar la estigmatización, amenazar los logros valiosos conseguidos en materia de salud e incitar el incumplimiento de las medidas de salud pública. Este fenómeno se amplifica a través de las redes sociales, propagándose más rápido y lejos, similar a como lo hace un virus. El concepto que se utilizará en el estudio es el propuesto por Wardle y Derakhshan (2017) que define a la información falsa o *fake news* como la combinación de tres nociones importantes: misinformation, disinformation and malinformation, esta información puede ser creada, producida o distribuida por "agentes" que tienen la intención de hacer daño, como también, por agentes que no lo tienen, por lo que es importante distinguir los mensajes verdaderos de los falsos.

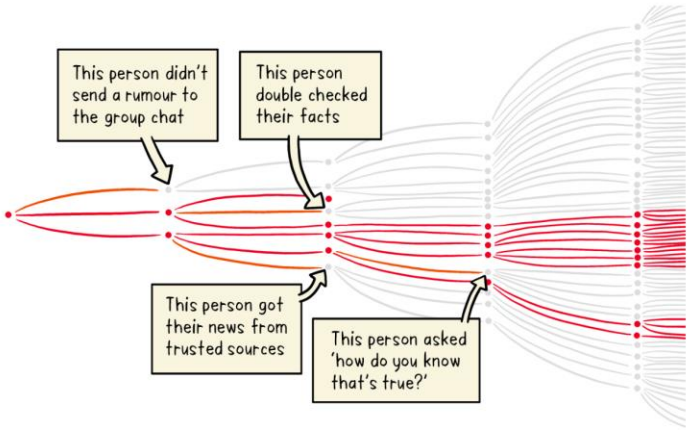


Figura 44. Ejemplo de curva de infodemia

Fuente: WHO (s.f.). *Let's flatten the infodemic curve.*

2.2.3.1 Contexto en el que surgen

Comúnmente se presentan cuando surge una infodemia, cuyo término se deriva de la unión de 'información' y 'epidemia' (epidemia de información), y hace referencia a un gran aumento del volumen de información relacionada con un tema particular, que puede volverse exponencial en un período corto de tiempo debido a un incidente concreto como la pandemia actual (OMS, 2020). En esta situación aparece una mezcla de información científica y técnica con rumores, datos manipulados, falsos expertos, información incorrecta y noticias falsas y

tendenciosas que dificultan el procesamiento y discernimiento por parte del receptor.

Asimismo, Wardle y Derakhshan (2017) argumentan que, en el contexto moderno de información, la llegada del internet, sumado al avance de la tecnología, provocaron cambios fundamentales en la manera que se produce, comunica y distribuye la información:

a) La velocidad de transmisión de la información se ha visto favorecida por el ciclo rápido de noticias y los teléfonos móviles.
b) La información se transmite en tiempo real entre pares de confianza y es mucho menos probable que se cuestione cualquier información.
c) El uso de la información, antes privada, se ha vuelto pública gracias a las redes sociales.
d) La tecnología de edición y publicación ampliamente accesible, barata y sofisticada ha hecho que la creación y distribución de contenido sea más fácil que nunca para cualquier persona.

De forma similar, García-Saisó et al. (2021) propone 3 principales factores asociados a la falta de programas de alfabetización digital que contribuyen en el desarrollo de difusión de información errónea:

1.- La dificultad de buscar, seleccionar, recomendar y diseminar críticamente datos e información confiables.
2.- La falta de criterios y herramientas para obtener información crítica en el formato y momento adecuados.
3.- El desconocimiento en el uso y pertinencia de aplicaciones digitales en salud.

2.2.3.2 Tipos

Según los autores Wardle y Derakhshan (2017), se pueden identificar 3 tipos de desórdenes de información:

Misinformation: Información falsa compartida por alguien que cree que es verdad. A menudo la inicia alguien que realmente quiere comprender un tema y se preocupa por la seguridad y el bienestar de los demás. Luego es compartido por otros que sienten lo mismo. Todos piensan que están compartiendo información valiosa, pero desafortunadamente no es así.

Disinformation: Información errónea que se comparte a sabiendas y, por lo tanto, con la intención de engañar. Esta es una mentira bien intencionada y con un propósito: lucrar o causar daño. El daño puede involucrar a una persona, a un grupo de personas, a una organización o incluso a un país. La desinformación a menudo sirve a una agenda y puede ser peligrosa.

Malinformation: Información privada que se vuelve pública como fotos compartidas en el contexto equivocado, vulnerando la privacidad de la persona. De igual manera que la *disinformation*, esta es difundida con intención de perjudicar a alguien.

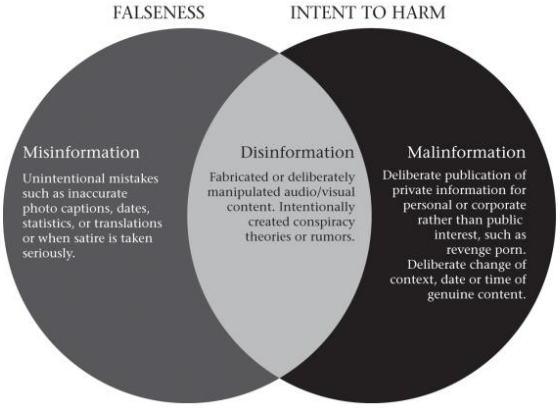


Figura 45. Tipos de desórdenes de información

Fuente: Wardle y Derakhshan (2020). *INFORMATION DISORDER: Toward an interdisciplinary framework for research and policy making.* (p. 5)

2.2.3.3 Categorías

Wardle y Derakhshan (2020) proponen 7 categorías:

1.- Sátira y parodia.- Esta forma de información se presenta de manera burlona que, a priori podría ser identificable, sin embargo, en la actualidad, cada vez más personas reciben información a través de sus redes sociales que parece ser la misma, por lo que muchos de ellos no se dan cuenta que el contenido es satírico.

2.- Conexión falsa.- Se le llama así a la información cuyos titulares, imágenes o subtítulos no son compatibles con el contenido. Un ejemplo común son aquellos titulares con *clickbait*, en la que los editores tienen que escribir cada vez más titulares para atraer clicks sin importar la concordancia con el contenido.

3.- Contenido engañoso.- Este tipo de contenido es el uso de información de manera engañosa para dar forma a un problema o individuo de una manera particular, como destacar imágenes, citas o estadísticas.

4.- Contexto falso.- Se da cuando el contenido es correcto pero aparece en un contexto equivocado. Un ejemplo puede ser la circulación de imágenes antiguas de eventos similares en el pasado.

5.- Contenido impostor.- Este contenido se vuelve un problema cada vez más común y se surge cuando la firma de periodistas aparecen en artículos que no escribieron, o el logo de una organización se adjunta a videos o imágenes que no crearon, es decir, es el tipo de información que suplanta fuentes genuinas.

6.- Contenido manipulado.- Referido a la información o imágenes genuinas manipuladas con la finalidad de engañar.

7.- Contenido fabricado.- Contenido nuevo que es predominantemente falso, diseñado especialmente para engañar y perjudicar. Puede ser contenido textual como visual (gráficos e imágenes).

2.2.3.4 Formas de detección

Chequeado.com, uno de los fact-checkers más populares en Latinoamérica, nos brinda algunos consejos para detectar la desinformación y el contenido pernicioso:

1. Desconfiar de los títulos grandes y en mayúsculas: Estos tipos de titulares están diseñados para capturar emociones y generar interés en las redes sociales, ya que se destacan más en las actualizaciones de la red.
2. Analizar la sección "Acerca de" o "About" del sitio: En la mayoría de los medios de confianza, esta sección de la página incluye información importante sobre el equipo, financiación, dirección donde opera el medio, contacto u otras características verificables. En los sitios creados especialmente para difundir contenido falso, esta sección suele faltar o estar incompleta.
3. Chequear los links y las citas: Es importante revisar las fuentes de la noticia, si los enlaces especificados como fuente no funcionan o redirigen a páginas ajenas a lo referido, entonces el artículo es sospechoso y no se debe compartir. Buscar las citas mencionadas en Google u otro buscador para comprobar que efectivamente son afirmaciones de las personas a las que se atribuyen, puede ser una buena opción.
4. Desconfiar de URLs similares a sitios conocidos: Muchas veces, la estrategia de los sitios web que difunden noticias falsas es utilizar un enlace bastante similar a otros portales de noticias famosos. Esto lo hacen con la finalidad de confundir a los lectores y difundir datos falsos.
5. Encontrar una fuente local y consultarla: Alguien en el sitio puede proporcionar información importante y no es necesario que se una a un medio con reporteros para hacerlo.
6. Realizar una búsqueda inversa de imágenes: Muchas veces el contenido engañoso utiliza fotografías fuera de contexto o que no coinciden con la imagen original. Con solo buscar estas imágenes en Google, se puede verificar fácilmente si la información presentada es verdadera.

2.3 Marco Conceptual

2.3.1 Aislamiento

El estar aislado significa estar separado de las personas, preferiblemente en un centro médico donde se pueda recibir atención médica. Se usa principalmente en personas que han dado positivo en alguna prueba de COVID-19. Se recomienda que la persona esté aislada durante 10 días desde que se presentaron los primeros síntomas o la prueba dio positivo. (OMS, 2020)

2.3.2 Alertas falsas

Son aquellas noticias manipuladas que se hacen pasar como verdaderas por organismos y entidades oficiales. (Peña et al., 2020)

2.3.3 Árboles de clasificación

Es una de las técnicas de clasificación más utilizadas debido a que no es necesario conocer, su funcionamiento se basa en las decisiones que toma en cada nodo y cada decisión tomada repercute en el resultado final. (Harrington, 2012)

2.3.4 Automedicación

Selección y uso de medicamentos con el fin de tratar enfermedades que la persona misma puede identificar. La OMS no recomienda automedicarse con algún fármaco para prevenir o curarse del COVID-19, puesto que al no tener la información suficiente de lo que se está haciendo, se puede incurrir en riesgos graves para la salud. (OMS, 2020)

2.3.5 Bag of Word (BoW)

Es una técnica para extraer las características de un texto, esta consiste en representar cuántas veces ocurre cada palabra en el texto, es decir, la frecuencia de cada palabra. (Brownlee, 2017)

2.3.6 COVID-19

Es una enfermedad infecciosa causada por el nuevo coronavirus conocido como SARS-CoV-2 cuyo origen fue en Wuhan (República Popular China), este virus afecta a

cualquier persona de cualquier edad, incluso puede llevar a la muerte si se sufre de alguna enfermedad cardiorrespiratoria o si el virus está en estado avanzado. (OMS, 2020)

2.3.7 Cuarentena

Significa que la persona debe estar separada de las demás porque ha estado expuesta al virus y es muy probable que también esté contagiada. Se aplica para cualquiera que haya estado en contacto con alguien portador del COVID-19, independientemente si esta presenta o no los síntomas. Se recomienda un periodo de 14 días. (OMS, 2020)

2.3.8 Estigma

Es un proceso construido socialmente a través del cual una persona con estigma puede experimentar discriminación y devaluación en la sociedad. (Saiful et al. 2020)

2.3.9 Información falsa

Combinación de tres nociones importantes: misinformation, disinformation and malinformation, esta información puede ser creada, producida o distribuida por "agentes" con la intención de hacer daño, como también, por "agentes" sin dicha intensión. (Wardle y Derakhshan, 2017)

2.3.10 Informaciones erróneas

Informaciones en las cuales se constata una falta de contexto de los contenidos pero que se difunden con una intencionalidad de veracidad a pesar de resultar falsas. (Peña et al., 2020)

2.3.11 KNN

Este algoritmo de clasificación se encuentra entre los más simples y clasifica las nuevas observaciones mediante la comparación de cada dato con los *k* datos más cercanos,

en el que se realiza un voto mayoritario, y, aquella clase con mayor cantidad de datos en su grupo, será la nueva clase este dato desconocido. (Harrington, 2012)

2.3.12 Mentiras

Manipulaciones intencionadas con el objetivo de producir un daño político, social, religioso o sanitario. (Peña et al., 2020)

2.3.13 Naive Bayes Classifier

Este algoritmo de clasificación pertenece al subconjunto de la teoría de decisión bayesiana, esta teoría se basa en tomar una decisión en base a un evento conocido anteriormente (Harrington, 2012)

2.3.14 Overfitting

El modelo se ajusta con gran precisión a los datos de entrenamiento pero la predicción es muy mala con datos nuevos. (Hurwitz y Kirsch, 2018)

2.3.15 Redes Neuronales Artificiales (RNA)

Una red neuronal artificial (RNA) es una máquina diseñada para modelar la forma en que el cerebro realiza una determinada tarea o función. Las neuronas pertenecientes a una misma capa no se conectan entre sí, pero aquellas pertenecientes a diferentes capas sí lo hacen, de modo que trasladan la información de capa en capa. (Haykin, 2009)

2.3.16 Regresión Logística

Es una técnica de clasificación utiliza la ecuación *sigmoid* para predecir el resultado, y, se basa en qué tan aproximado está el resultado con respecto a alguno de los extremos, se elegirá como clase aquel extremo más cercano. (Harrington, 2012).

2.3.17 Rumor

Aquella información no verificada que se puede encontrar como verdadera, fabricada o completamente falsa después de la verificación. (Saiful et al. 2020)

2.3.18 Síntomas del COVID-19

Problemas físicos productos de una enfermedad; los más frecuentes son la fiebre, tos seca y el cansancio; mientras que los menos frecuentes son la pérdida del gusto/olfato, congestión nasal, dolor de garganta/cabeza, náuseas y diarrea en una persona con COVID-19. (OMS, 2020)

2.3.19 Stemming

Es el proceso de reducir las palabras a su raíz. Una raíz es la parte de una palabra que queda después de eliminar sus prefijos y sufijos. (Bing, 2011)

2.3.20 Stopwords

Son palabras que aparecen con frecuencia y son insignificantes en un idioma que ayudan a construir oraciones pero que no representan ningún contenido de los documentos. Los artículos, preposiciones y conjunciones y algunos pronombres. (Bing, 2011)

2.3.21 Support Vector Machine (SVM)

Es uno de los algoritmos de clasificación más populares en la actualidad debido a su gran adaptabilidad para problemas de clasificación linealmente separable, semi-separable y no separables linealmente. El SVM toma el clasificador en su forma básica y, al ejecutarlo con los datos, se obtienen resultados con tasas de error bajas. (Harrington, 2012)

2.3.22 Teorías conspirativas

Son creencias explicativas sobre un individuo o grupo de personas que trabajan en secreto para alcanzar objetivos maliciosos. (Saiful et al. 2020)

2.3.23 Text Mining

Se trata de buscar patrones en el texto: es el proceso de analizar texto para extraer información que es útil para propósitos particulares. (Witten, Eibe y Hall, 2011)

2.3.24 Tf-idf

Técnica para encontrar el documento más relevante para cierto término dentro de una colección de documentos, para ello mide la frecuencia de aparición de un término o frase dentro de un determinado documento, y lo compara con el número de documentos que mencionan ese término dentro de una colección entera de documentos (González, 2019).

2.3.25 Underfitting

El modelo es tan malo que no ha logrado aprender con los datos de entrenamiento y la predicción es muy mala tanto con datos de entrenamiento como con datos desconocidos. (Hurwitz y Kirsch, 2018)

2.3.26 Word2Vec

Técnica utiliza para obtener una palabra vecina a la palabra ingresada, ello es posible gracias al entrenamiento de la red neuronal artificial, el cual se realiza con un enorme vocabulario de palabras del *corpus* (Gonzalo, 2018).

2.4 Hipótesis

2.4.1 Hipótesis General

H0: La información falsa del COVID-19 que circula en la web peruana puede ser detectada mediante técnicas de Machine Learning y Natural Language Processing.

2.4.2 Hipótesis Específicas

H1: La recolección de suficiente información del COVID-19 en la web peruana permite la construcción del dataset.

H2: Realizar el preprocesamiento influye significativamente en un mejor entrenamiento del modelo.

H3: La implementación de técnicas de Natural Language Processing favorece en la extracción de características de cada documento.

H4: La implementación de técnicas de clasificación de Machine Learning permite construir un modelo de clasificación robusto de información falsa del COVID-19.

H5: La utilización de métricas influye significativamente en la evaluación del desempeño del modelo de información falsa del COVID-19.

CAPÍTULO 3: METODOLOGÍA

3.1 Diseño, tipo y enfoque de investigación

3.1.1 Diseño

Según Hernández et al (2010), en un diseño experimental se manipulan las variables independientes con la finalidad de observar sus posibles efectos sobre las variables dependientes en una situación de control. Este tipo de diseño se utiliza cuando se busca establecer algún posible efecto de una causa que se manipula.

En el presente estudio se utilizará un diseño experimental puesto que se tendrá mayor control de las variables a utilizar, tanto independientes como la dependiente, las que se manipularán mediante varios experimentos a fin de tener la mayor cantidad de resultados posibles, los cuales se compararán y se escogerá el mejor de todos, ello significará la elección del modelo de clasificación.

3.1.2 Tipo

Para Hernández et al (2010), en una investigación aplicada, el investigador busca resolver algún problema conocido y encontrar respuestas a preguntas específicas, es decir, el énfasis de este tipo de investigación es la resolución de problemas.

Bajo este concepto, se puede decir que el estudio es una investigación aplicada, ya que se quiere dar solución al problema de la detección de información falsa relacionada al COVID-19.

3.1.2.1 Nivel

De acuerdo a Hernández et. al (2010), un alcance explicativo se enfoca en explicar la causalidad de un fenómeno y las condiciones en que se manifiesta, es decir, busca encontrar la relación entre dos o más variables.

Desde la perspectiva del presente estudio, se considera un alcance de tipo explicativo, ya que, mediante el modelo construido en base a técnicas de Machine Learning y Natural Language Processing, se busca determinar la veracidad de algún tipo de información relacionada al COVID-19, es decir, se quiere determinar si el modelo en realidad influye o no al momento de detectar falsedad de dicha información.

3.1.3 Enfoque

Según Hernández et. al (2010), un enfoque cuantitativo hace uso de datos numéricos, aplica la estadística y realiza pruebas de hipótesis, asimismo, este enfoque es secuencial, lo cual supone que no es posible saltearse los pasos, es necesario seguirlos de manera rigurosa en cada fase.

El enfoque del estudio es cuantitativo porque, para la construcción del clasificador, es necesario trabajar con datos numéricos, es por ello que mediante técnicas de vectorización de características se conseguirá convertir la data textual en data numérica, con la cual, posteriormente, se entrenará al modelo de Machine Learning. Asimismo, para llevar a cabo ello, se tendrá que seguir cada fase definida en la metodología.

3.2 Población y muestra

Tabla 2.

Población, muestra y unidad de análisis del estudio

Dimensión	Definición	Estudio
Población	Según Hernández et. al (2010), la población es el conjunto de todos los casos que concuerdan con una serie de especificaciones.	La población está dada por los 80k registros de información del COVID-19 existentes en la web peruana recolectados tanto de páginas de web como de las redes sociales.
Muestra	Según Hernández et. al (2010), el muestreo por conveniencia se realiza cuando solo se tiene acceso a cierto número de casos disponibles.	Para la determinación de la muestra se realizó un muestro por conveniencia, pues se busca que la muestra sea adecuada para un correcto entrenamiento del modelo de clasificación, ya que, si se realizara un muestreo aleatorio, el modelo podría verse afectado por el desbalance de los datos positivos y negativos, además, podría darse el caso que los registros elegidos no tengan una etiqueta clara. En total, se cuenta con un dataset de 1299 registros relacionados al COVID-19 entre verdaderos y falsos.
Unidad de análisis	Según Hernández et. al (2010), la unidad de análisis se refiere al individuo de la población.	La unidad de análisis es un registro de información relacionado al COVID-19 publicado en la web peruana.

Fuente: Elaboración propia

3.3 Operacionalización de variables

Tabla 3.
Operacionalización de variables

Variable	Definición	Medición	Fórmula

Independiente **Técnicas de Machine Learning y Natural Languaje Processing**	**Machine Learning** Según Mitchell (1997), "Se ocupa de la cuestión de cómo construir programas informáticos que mejoren automáticamente con la experiencia." **Natural Language Processing** Según Covington (2013): "Es el uso de las computadoras para comprender el lenguaje humano (natural) como el inglés, el francés o el japonés".	Accuracy	$\frac{TP+TN}{TP+FP+FN+TN}$
		Precision	$\frac{TP}{TP+FP}$
		Recall	$\frac{TP}{TP+FN}$
		F1-score	$2*\left(\frac{Precision*Recall}{Precision+Recall}\right)$
Dependiente **Información Falsa**	Combinación de tres nociones importantes: misinformation, disinformation and malinformation, esta información puede ser creada, producida o distribuida por "agentes" con la intención de hacer daño, como también, por "agentes" sin dicha intensión. (Wardle y Derakhshan, 2017)	Porcentaje de tipo de información	$\left(\frac{Registros\ falsos}{Total\ de\ registros}\right)*100$ $\left(\frac{Registros\ \ verdaderos}{Total\ de\ registros}\right)*100$

Fuente: Elaboración propia

Donde:

TP = True Positives

FP = False Positives

FN = False Negatives

TN = True Negatives

3.4 Técnicas de recolección de datos

La recolección de datos solo se realizó de una manera: *web scraping.* Esta técnica consiste en navegar por la web de manera automática y extraer información relevante para el estudio, en este caso, el título o contenido, las urls y las fechas de cadapublicación circulante en la web peruana. La ventaja de este método es que no se pierde tiempo recolectando la data de manera manual, ya que, mediante un *crawler* (araña informática), se realiza este proceso de forma automática, pues esta tecnología está programada para extraer la información de la web sin la necesidad de realizarla uno mismo, este *crawler* recorre cada página que se le indica mediante su url y obtiene la información solicitada, la que posteriormente se guarda como data estructurada (archivos excel o csv) o semiestructurada (archivos .json). Esta técnica se aplicó en la red social Twitter (tweets que circulan en Perú), las páginas de facebook del Minsa, Andina, El Comercio, La República, Perú21, El Peruano, RPP y Tv Perú, y, las páginas web del Minsa (Sección de noticias), Verificador y OjoPúblico (*Fact-checkers*).

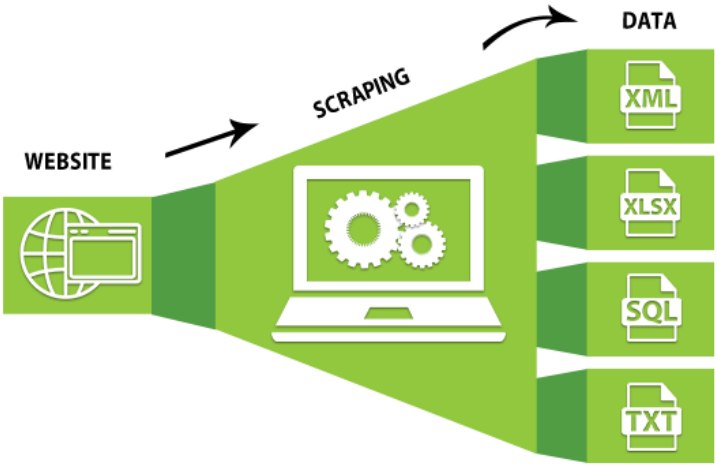


Figura 46. Funcionamiento del Web Scraping

Fuente: Agency, J. (2016). *Web Scraping Tutorial and Use Case* (p. 1)

3.5 Metodología de implementación de la investigación

La metodología empleada se basó en la combinación de 2 antecedentes fundamentales: Bojjireddy et. al (2021) y Hayawi et. al (2022). El primero de ellos brindó conocimiento acerca de las posibles fases que se podrían aplicar a lo largo del estudio; sin embargo, en este estudio se trabajó con datasets previamente etiquetados, por lo que fue necesario de un segundo antecedente que trabaje sin data etiquetada, de manera que este procedimiento se realice manualmente, entonces, se decidió incluir este antecedente a fin de identificar los pasos que siguieron para llevar a cabo el etiquetado y la validación del mismo.



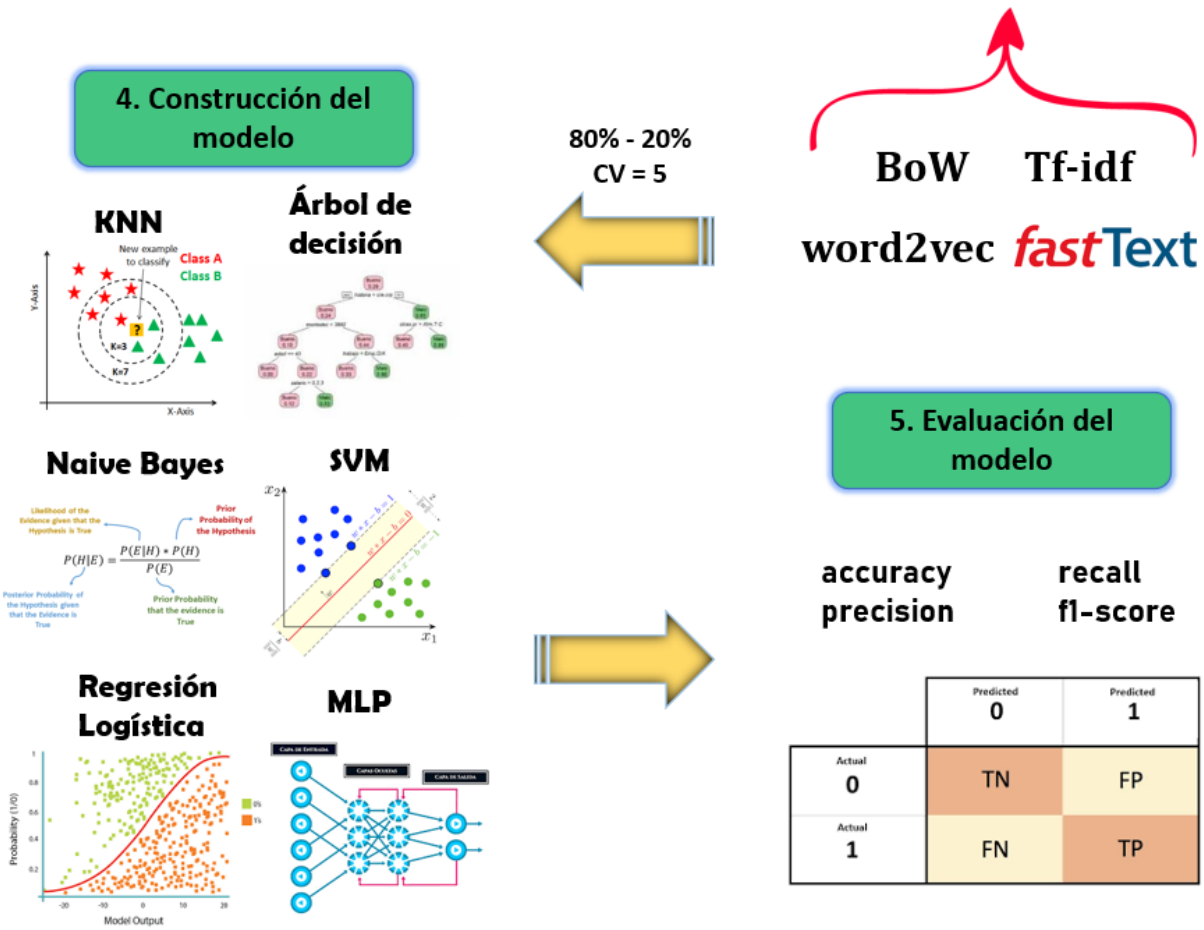


Figura 47. Metodología de la investigación

Fuente: Elaboración propia

3.5.1 Creación del dataset

Esta primera fase de la metodología describe la creación del dataset de información falsa y verdadera del COVID-19, para ello, a continuación, se presenta una tabla con las actividades y tares a realizar:

Tabla 4.
Actividades de la fase de Creación del dataset

Actividades	Descripción	Tareas
1. Identificar fuentes de datos	Identificación de fuentes de datos peruanas que publiquen información acerca del coronavirus.	Identificar las cuentas de Facebook de los medios de comunicación peruanos.
		Identificar los departamentos de los cuales se extraerán los tweets.
		Identificar las páginas web de *fact-checking* peruanas.
2. Desarrollar los algoritmos de scraping	Creación de un adecuado algoritmo de scraping por cada fuente identificada.	Crear el algoritmo de scraping para Facebook para extraer los posts y comentarios.
		Crear el algoritmo de scraping para Twitter para extraer los tweets.
		Crear el algoritmo de scraping para las páginas web con sus respectivos procedimientos de extracción de información del html.
3. Homogeneizar los datasets	Igualdad de los datasets en base a un mismo formato.	Definir las columnas que tendrá cada dataset.
		Eliminar columnas innecesarias en cada dataset.
		Añadir columnas faltantes en algún dataset.
		Unir los datasets en uno solo.
		Eliminar registros repetidos
		Eliminar registros que no traten del COVID-19.

4. Etiquetar manualmente los registros	Etiquetado manual de información verdadera o falsa.	Etiquetar cada registro contrastando el contenido con la información publicada por el MINSA.
		Comparar el registro con la información de la OMS en caso no se haya encontrado el registro en el MINSA.
		Eliminar los registros que no pertenezcan a ninguna clase.
5. Validar el etiquetado con personal del MINSA	Validación de un correcto etiquetado por personal del MINSA.	Contactarse con personal del MINSA perteneciente a la Oficina General de Comunicaciones para que valide el etiquetado.
		Realizar correcciones necesarias en base a la validación.

Fuente: Elaboración propia

Actividad 1: Identificar fuentes de datos

En esta primera actividad de la creación del dataset se describen las fuentes principales que se tomaron en cuenta para extraer información acerca del COVID-19. Puesto que se busca diversidad de fuentes, es necesario identificar qué plataformas serán útiles para llevar a cabo la investigación. Actualmente en nuestro país, existen múltiples maneras en las que podemos adquirir información acerca de los acontecimientos del COVID-19: medios de comunicación, redes sociales, páginas web, conversaciones cotidianas, etc. Asimismo, por cada medio de información, existe gran variedad de fuentes tales como canales de televisión y emisoras de radio (medios de comunicación), cuentas y/o perfiles (redes sociales), dominios (páginas web) y, amigos, familiares, y

compañeros de trabajo con quienes se interactúa diariamente. Es por ello que es necesario identificar cuáles serán las fuentes útiles para el trabajo.

Entregable:

- Fuentes seleccionadas de la cuales se extraerá información del COVID-19.

Actividad 2: Desarrollar los algoritmos de scraping

Los algoritmos de scraping nos permiten extraer información de sitios web de manera automática, lo que agiliza gran parte del proceso de recolección, es por ello que en esta segunda actividad se detalla el funcionamiento de cada algoritmo de scraping. Puesto que se seleccionaron 3 medios de información (Facebook, Twitter y páginas web), es necesario la implementación de un algoritmo para cada uno de ellos:

A) Facebook:

```
algoritmo scraping_fb

variables
    lista_de_posts, fecha_final, account

inicio
|    repetir
|    |    post1  <-  obtener post número 1 de la cuenta 'account'
|    |    agregar post1 a la lista_de_posts
|    |    post2  <-  obtener post número 2 de la cuenta 'account'
|    |    agregar post2 a la lista_de_posts
|    |    .
|    |    .
|    |    .
|    |    post_n <-  obtener post número n de la cuenta 'account'
|    |    agregar post_n a la lista_de_posts
|    hasta que se llegue a la fecha_final
fin
```

Figura 48. Pseudocódigo del algoritmo de scraping de Facebook

Fuente: Elaboración propia

Asimismo, ya que cada post cuenta con muchos comentarios, se necesita implementar un algoritmo para extraer esos comentarios:

```
variables
    lista_de_posts, lista_comentarios

inicio
|    repetir
|    |    comentario1 <-  obtener el comentario1 en el post1 de la lista_de_posts
|    |    añadir el comentario1 a la lista_comentarios
|    |    repetir
|    |    |    respuesta1  <-  obtener la respuesta1 del comentario1
|    |    |    añadir la respuesta1 a la lista_comentarios
|    |    |    .
|    |    |    .
|    |    |    .
|    |    |    respuesta_n  <-  obtener la respuesta_n del comentario1
|    |    |    añadir la respuesta_n a la lista_comentarios
|    |    hasta que se completen todas las respuestas del comentario1
|    |    .
|    |    .
|    |    .
|    |    comentario_n <-  obtener el comentario_n en el post_n de la lista_de_posts
|    |    añadir el comentario_n a la lista_comentarios
|    |    repetir
|    |    |    respuesta1  <-  obtener la respuesta1 del comentario_n
|    |    |    añadir la respuesta1 a la lista_comentarios
|    |    |    .
|    |    |    .
|    |    |    .
|    |    |    respuesta_n  <-  obtener la respuesta_n del comentario_n
|    |    |    añadir la respuesta_n a la lista_comentarios
|    |    hasta que se completen todas las respuestas del comentario_n
|    |
|    hasta que se complete toda la lista_de_posts
fin
```

Figura 49. Pseudocódigo del algoritmo de extracción de comentarios de los posts de Facebook

Fuente: Elaboración propia

B) Twitter:

```
algoritmo scraping_twitter

variables
    lista_de_tweets, palabra_clave, coordenadas, número_de_items

inicio
|    repetir
|    |    tweet1  <-  obtener tweet número 1 por palabra_clave y coordenadas
|    |    agregar tweet1 a la lista_de_tweets
|    |    tweet2  <-  obtener tweet número 2 por palabra_clave y coordenadas
|    |    agregar tweet2 a la lista_de_tweets
|    |    .
|    |    .
|    |    .
|    |    tweet_n <-  obtener tweet número n por palabra_clave y coordenadas
|    |    agregar tweet_n a la lista_de_tweets
|    hasta que se llegue al número_de_items
fin
```

Figura 50. Pseudocódigo del algoritmo de scraping de Twitter

Fuente: Elaboración propia

C) Páginas web:

Debido a que cada página web tiene una estructura diferente, es necesario crear un algoritmo de scraping por cada una de ellas; sin embargo, el algoritmo solo devuelve un html, por lo que también será necesario un procedimiento para extraer las noticias del propio html:

1.- Web del MINSA (Sección de noticias)

```
algoritmo scraping_Minsa

variables
    html, driver, url, número_final_página, time_sleep

inicio
|    acceder con el driver en la página web del MINSA mediante su url
|    colocar un delay de time_sleep segundos
|    repetir
|    |    html1  <-  obtener el hmtl de la página1
|    |    clickear a la siguiente página
|    |    colocar un delay de time_sleep segundos
|    |    html2  <-  obtener el hmtl de la página2
|    |    clickear a la siguiente página
|    |    colocar un delay de time_sleep segundos
|    |    .
|    |    .
|    |    .
|    |    html_n <-  obtener el hmtl de la página_n
|    |    clickear a la siguiente página
|    |    colocar un delay de time_sleep segundos
|    hasta que se llegue al número_final_página
fin
```

Figura 51. Pseudocódigo del algoritmo de scraping de la página web del MINSA

Fuente: Elaboración propia

```
variables
    html_parse, lista_general, noticias, fechas, links

inicio
|    parsear el html
|    crear la lista_general que agrupe el html_parse de cada noticia
|    repetir
|    |    noticia1 <-  obtener el título de la noticia1 en la lista_general
|    |    fecha1   <-  obtener la fecha de la noticia1 en la lista_general
|    |    link1    <-  obtener el link de la noticia1 en la lista_general
|    |    añadir la noticia1 a la lista de noticias
|    |    añadir la fecha1 a la lista de fechas
|    |    añadir el link1 a la lista de links
|    |    .
|    |    .
|    |    .
|    |    noticia_n <-  obtener el título de la noticia_n en la lista_general
|    |    fecha_n   <-  obtener la fecha de la noticia_n en la lista_general
|    |    link_n    <-  obtener el link de la noticia_n en la lista_general
|    |    añadir la noticia_n a la lista de noticias
|    |    añadir la fecha_n a la lista de fechas
|    |    añadir el link_n a la lista de links
|    hasta que se complete toda la lista_general
fin
```

Figura 52. Pseudocódigo del algoritmo de extracción de noticias del html del MINSA

Fuente: Elaboración propia

2.- Verificador (Factcheking)

```
algoritmo scraping_Verificador

variables
    html, driver, url, time_sleep

inicio
|    acceder con el driver en la página web de Verificador mediante su url
|    colocar un delay de time_sleep segundos
|    cerrar el contenido emergente que dificulta la navegación
|    colocar un delay de time_sleep segundos
|    mientras aparezca al botón "VER MÁS"
|    |    clickear el botón
|    |    colocar un delay de time_sleep segundos
|    hasta que el botón desaparezca
|    obtener el html de toda la página
fin
```

Figura 53. Pseudocódigo del algoritmo de scraping de la página web de Verificador

Fuente: Elaboración propia

```
variables
    html_parse, lista_general, noticias, links

inicio
|    parsear el html
|    crear la lista_general que agrupe el html_parse de cada noticia
|    repetir
|    |    noticia1 <-  obtener el título de la noticia1 en la lista_general
|    |    link1    <-  obtener el link de la noticia1 en la lista_general
|    |    añadir la noticia1 a la lista de noticias
|    |    añadir el link1 a la lista de links
|    |    .
|    |    .
|    |    .
|    |    noticia_n <-  obtener el título de la noticia_n en la lista_general
|    |    link_n    <-  obtener el link de la noticia_n en la lista_general
|    |    añadir la noticia_n a la lista de noticias
|    |    añadir el link_n a la lista de links
|    hasta que se complete toda la lista_general
fin
```

Figura 54. Pseudocódigo del algoritmo de extracción de noticias del html de Verificador

Fuente: Elaboración propia

3.- Ojo Público (Factcheking)

```
algoritmo scraping_OjoPúblico

variables
    html, driver, url, time_sleep, número_final_página

inicio
|    acceder con el driver en la página web de Ojo Público mediante su url
|    colocar un delay de time_sleep segundos
|    cerrar el contenido emergente que dificulta la navegación
|    colocar un delay de time_sleep segundos
|    repetir
|    |    colocar un delay de time_sleep segundos
|    |    html1 <- obtener el html de la página1
|    |    clickear a la siguiente página
|    |    colocar un delay de time_sleep segundos
|    |    html2 <- obtener el html de la página2
|    |    clickear a la siguiente página
|    |    .
|    |    .
|    |    .
|    |    colocar un delay de time_sleep segundos
|    |    html_n <- obtener el html de la página_n
|    |    clickear a la siguiente página
|    hasta que se llegue al número_final_página
fin
```

Figura 55. Pseudocódigo del algoritmo de scraping de la página web de Ojo Público

Fuente: Elaboración propia

```
variables
    html_parse, lista_general, noticias, fechas, links

inicio
|    parsear el html
|    crear la lista_general que agrupe el html_parse de cada noticia
|    repetir
|    |    noticia1 <-  obtener el título de la noticia1 en la lista_general
|    |    fecha1   <-  obtener la fecha de la noticia1 en la lista_general
|    |    link1    <-  obtener el link de la noticia1 en la lista_general
|    |    añadir la noticia1 a la lista de noticias
|    |    añadir la fecha1 a la lista de fechas
|    |    añadir el link1 a la lista de links
|    |    .
|    |    .
|    |    .
|    |    noticia_n <-  obtener el título de la noticia_n en la lista_general
|    |    fecha_n   <-  obtener la fecha de la noticia_n en la lista_general
|    |    link_n    <-  obtener el link de la noticia_n en la lista_general
|    |    añadir la noticia_n a la lista de noticias
|    |    añadir la fecha_n a la lista de fechas
|    |    añadir el link_n a la lista de links
|    hasta que se complete toda la lista_general
fin
```

Figura 56. Pseudocódigo del algoritmo de extracción de noticias del html de Ojo Público
Fuente: Elaboración propia

Entregable:

- Archivo csv de posts
- Archivo csv de comentarios.
- Archivo csv de tweets.
- Archivo csv de noticias del Minsa
- Archivo cvs de noticias de Verificador LR
- Archivo csv de noticias de Ojo Público

Actividad 3: Homogeneizar los datasets

Una vez recolectado los datos de mediante los algoritmos de scraping, es necesario brindar una misma estructura para cada dataset, es decir, cada dataset tiene que ser igual en lo que respecta a las columnas y formato de datos, y, principalmente, contener solo información que trate acerca del COVID-19. Entonces, en esta actividad se

describe la manera en que se estandarizarán los datasets. Para ello, será necesario eliminar las columnas que no servirán para análisis posteriores y añadir aquellas columnas faltantes, unir todos los datasets en uno solo, finalmente, eliminar registros repetidos y aquellos que no aporten información acerca del COVID-19.

Entregable:

- Dataset relacionado al COVID-19.

Actividad 4: Etiquetar manualmente los registros

Esta actividad es una de las más importantes y a su vez, más trabajosas, puesto que se trata de comparar cada registro del dataset con información verídica y actualizada a fin de determinar si dicha información es real o no. Para llevar a cabo esta actividad es necesario identificar las fuentes de información fiables, las cuales serán la base de información verdadera (MINSA, OMS y OPS) e información falsa (Verificador y Ojo Público, ambos como *factcheckers*). Asimismo, el etiquetado se dará de la siguiente manera: para el contenido falso se colocará la etiqueta de "Falso", mientras que el contenido verdadero tendrá la etiqueta de "Verdadero", y, aquel contenido que no pertenezca a ninguno de los dos, se dejará con la etiqueta "Ninguno". Cabe mencionar que, en esta actividad, solo se etiquetarán registros de los cuales se está completamente seguro de su veracidad o falsedad, y aquellos registros dudosos, satíricos y relacionados a temas religiosos se dejarán de lado. Asimismo, mencionar que aquellos registros que no se logren etiquetar se considerarán como el dataset general con el cual se entrenarán los modelos de *embedding.*

Entregable:

- Dataset etiquetado.

Actividad 5: Validar el etiquetado con personal del MINSA

Finalmente, la última actividad de esta fase trata de la validación del etiquetado con personal experto del MINSA, esto con la finalidad de obtener la aprobación de un correcto etiquetado y, si es el caso, realizar las modificaciones correspondientes mediante el *feedback* proporcionado. Para ello, es necesario identificar al área que nos brindará

apoyo, en este caso será la Oficina General de Comunicaciones, puesto que una de sus funciones principales es distribuir información al pueblo peruano, así como alertar e incentivar a no compartir contenido falso.

Entregable:

- Dataset validado.

3.5.2 Preprocesamiento

Esta segunda fase de la metodología describe el preprocesamiento realizado para cada registro del dataset. Se considera una de las fases más importantes pues dependiendo qué tan bueno sea este, el modelo de clasificación predecirá de mejor manera. A continuación, se detallan las actividades y tares a realizar:

Tabla 5.
Actividades de la fase de Preprocesamiento

Actividades	Descripción	Tareas
1. Eliminar los links, hashtags, menciones y emojis.	Eliminación de contenido irrelevante como los links, hashtags, menciones y emojis.	Crear expresión regular para eliminar links, hashtags y menciones.
		Identificar librería para eliminar emojis.
2. Limpiar el contenido	Limpieza del contenido para obtenerlo en un mismo formato a fin de una mejor vectorización de características.	Eliminar caracteres especiales propios del español.
		Pasar a minúsculas.
		Tokenizar cada elemento.
		Eliminar signos de puntuación y números.
		Eliminar stop words.
		Realizar stemmig a cada palabra.
		Crear el corpus

Fuente: Elaboración propia

Actividad 1: Eliminar los links, hashtags, menciones y emojis

Esta primera actividad describe el tratamiento de los contenidos que no serán útiles para la vectorización de características, puesto que no aportan tanto valor como para ser tomados en cuenta. Esta actividad se subdivide en 2 tareas: eliminación de links, hashtags y menciones, y, eliminación de emojis.

Se decidió agrupar los 3 primeros en una sola tarea puesto que es más sencillo de manejarlos mediante expresiones regular, a diferencia de los emojis, ya que las expresiones regulares facilitan en gran medida el tratamiento de contenido del texto, así pues, mediante estas, se pueden identificar patrones característicos del contenido que queremos eliminar: https://, #, @. Entonces, lo que se busca es crear una expresión regular que permita eliminar todos los links, hashtags y menciones que aparecen en un documento, sin alterar el resto de contenido. Por otro lado, los emojis se tratan de manera

distinta, ya que, si bien, también pueden trabajarse con expresiones regulares, esto suele ser más complicado, por ello se decide optar por una librería que facilite el tratamiento de emojis. Así que esta segunda tarea consiste en identificar la librería correcta y eliminar todos los emojis dentro del texto.

Entregable:

- Registros sin links, hashtags, menciones ni emojis.

Actividad 2: Limpiar el contenido

Esta actividad nos permitirá dejar listo el contenido para una futura vectorización de características. Para llevar a cabo esto, se comenzará con una limpieza de caracteres propios del español (ñ, ü y las tildes), puesto que ello puede ocasionar problemas al momento de vectorizar, es decir, si en el contenido se encuentran las palabras “ómicron” y ‘omicron’, en teoría, ambas representan lo mismo, por lo que los valores de sus vectores característicos deberían ser los mismo; sin embargo, puesto que una tiene tilde y la otra no, las técnicas de vectorización podrían considerarlas como palabras distintas, dando vectores diferentes. De igual manera, es necesario normalizar el contenido, es decir, pasar todo a minúsculas, con el fin de evitar el mismo problema (COVID y covid con vectores diferentes), seguido de ello, se llevará a cabo la tokenización, esto es, asignar un *token* o valor a cada elemento del texto, de manera que se pueda tratar individualmente cada *token.* Posteriormente, nos encargamos de eliminar signos de puntuación, números y *stop words* o palabras vacías, pues tampoco aportan valor al momento de la creación del modelo, particularmente los *stop words,* que suelen ser palabras con gran frecuencia (muy repetidas) pero sin mucho valor, este es el caso de artículos (el, la, los), preposiciones (de, en, para) y pronombres (mío, tuyo). Sin embargo, también será importante identificar palabras propias normalmente usadas en el contexto peruano (di, pe, pes) y añadirla a la lista de *stop words,* Una vez creada nuestra lista con todos los *stop words,* se procederá a comparar cada token del texto con esta lista, es decir, si dicho token se encuentra en la lista, será removido, de lo contrario, será conservado. Mientras que para las puntuaciones y números se hará algo similar: verificar si el token contiene caracteres alfabéticos, en caso no los contenga, se eliminará dicho token. Con

todo lo mencionado, se obtendrán solo palabras representativas por cada documento, las cuales se juntarán en una lista de documentos, formando de esta manera el *corpus* con el que se llevará a cabo la vectorización de características.

Entregable:

- Corpus de palabras preprocesadas.

3.5.3 Extracción de características

Para la tercera etapa de la metodología se procederá a extraer las características de cada registro de información del dataset aplicando técnicas de Natural Language Processing. Para ello es necesario un buen preprocesamiento, pues este facilita una mejor vectorización de características, lo que finalmente se demuestra en un mejor modelo de clasificación. A continuación, se detallan las actividades y tareas a realizar:

Tabla 6.
Actividades de la fase de Extracción de características

Actividades	Descripción	Tareas
1. Realizar extracción de características con BoW	Extracción de las características de cada registro con la técnica de BoW.	Identificar los parámetros relevantes.
		Crear 3 vectores distintos y aplicarlo al corpus.
2. Realizar extracción de características con Tf-Idf	Extracción de las características de cada registro con la técnica de Tf-Idf.	Identificar los parámetros relevantes.
		Crear 3 vectores distintos y aplicarlo al corpus.
3. Realizar extracción de características con Word2Vec	Extracción de las características de cada registro con la técnica de Word2Vec.	Entrenar el embedding con el corpus general que incluye todos los registros.
		Identificar los parámetros relevantes.
		Crear 3 vectores distintos y aplicarlo al corpus.
4. Realizar extracción de características con FastText	Extracción de las características de cada registro con la técnica de FastText.	Entrenar el embedding con el corpus general que incluye todos los registros.
		Identificar los parámetros relevantes.
		Crear 3 vectores distintos y aplicarlo al corpus.

Fuente: Elaboración propia

Actividad 1: Realizar la extracción de características con BoW

La primera actividad se basa en la creación de los vectorizadores con la técnica BoW, para lo cual es fundamental identificar los parámetros más importantes de esta técnica. Si bien en las bases teóricas no se detallan, no se tendrá en cuenta todos los existentes, sino solo los que serán útiles para crear el vectorizador.

a) Tamaño del vector (‘*max_features*’): Este es el parámetro más importante pues de este depende el tamaño del vectorizador. Como se sabe, la técnica de BoW es de las más costosas computacionalmente hablando, puesto que trabaja con todas las palabras existentes en el corpus, es por eso que gracias a este parámetro se podrá acortar el vocabulario del corpus con la finalidad de una compilación más rápida y un mejor vectorizador, ya que muchas veces incluir todas las palabras no es de mucha utilidad, especialmente cuando el procesamiento no es el adecuado, pues en el corpus pueden quedar palabras insignificantes que la técnica tomará en cuenta.

Entregable:

- Tres vectorizadores con la técnica BoW

Actividad 2: Realizar la extracción de características con Tf-Idf

La segunda actividad también se trata de la creación de los vectorizadores pero con la técnica Tf-Idf, donde los parámetros a utilizar son los siguientes:

a) Tamaño del vector (‘*max_features*’): Es el más importante pues define el tamaño del vectorizador. Como se explicó en las bases teóricas, una parte del vector se construye en base a la frecuencia de términos (Tf), la cual toma en consideración la cantidad de palabras en el vocabulario del corpus, es por eso que delimitar un número máximo permitirá trabajar con el top de palabras más frecuentes.

b) Rango de N-gramas (‘*ngram_range*’): Se probará este parámetro en el rango de 1 a 2, es decir, con unigramas (solo 1 palabra) y bigramas (2 palabras).

Entregable:

- Tres vectorizadores con la técnica Tf-Idf

Actividad 3: Realizar la extracción de características con Word2Vec

En esta actividad se buscará crear los vectorizadores; sin embargo, al tratarse de un modelo pre-entrenado, este se desarrolla de manera distinta a los anteriores, ya que no se trabaja solo con el corpus de la data etiquetada, sino también con el de la data general. Si bien el modelo está pre-entrenado, este entrenamiento es de manera general, por lo que

aún es necesario realizar un entrenamiento propio relacionado a la problemática que se quiere tratar. Es por ello que se utilizará el corpus formado del dataset general que trata acerca del covid, pues, aunque contenga registros no etiquetados, lo importante es el contenido, mas no su etiqueta. Cabe mencionar que mientras más grande sea este corpus, el entrenamiento del modelo será mejor, es decir, brindará mejores vectores de características. Para llevar a cabo este entrenamiento se considerarán los siguientes parámetros:

a) Tamaño del vector (‘*size’*): Este parámetro permite especificar cuál será el tamaño del vector que brindará el modelo después de su entrenamiento.

b) Ventana (‘*window’*): Determina la distancia máxima existente entre la palabra actual y la predicha, es decir, mientras más grande sea el valor, el modelo tratará de capturar más información acerca de otras palabras usadas en tópicos similares, por el contrario, si el valor es menor, se buscará capturar palabras similares como algunos sinónimos.

c) Frecuencia mínima del documento (‘*min_count’*): No se tomará en cuenta palabras no tan frecuentes (raras) para entrenar al modelo.

d) Tipo de algoritmo (‘*sg*’): Como se detalló en las bases teóricas, este modelo se puede entrenar de 2 formas distintas (cbow y skip-gram). Se probarán ambos casos pues puede que uno de ellos se adapte mejor al problema planteado.

Una vez definidos los parámetros, se procederá a construir el vector característico de cada registro, para ello, es necesario identificar todas las palabras que contiene el documento, pues el modelo de embedding brinda el vector característico de cada palabra, por lo que es necesario identificar cada una de ellas y promediarlas, obteniendo de esta manera el vector del registro en particular.

Entregable:

- Tres modelos Word2Vec entrenados

Actividad 4: Realizar la extracción de características con FastText

Esta actividad se desarrollará de manera similar a la anterior puesto que también se trata de un modelo pre-entrenado. Así pues, se utilizará el corpus formado del dataset

general para el entrenamiento del modelo para después obtener los vectores característicos de cada documento mediante el promedio de palabras que contiene ese documento. Los parámetros a considerar son los siguientes:

a) Tamaño del vector (‘*size’*): Este parámetro permite especificar cuál será el tamaño del vector que brindará el modelo después de su entrenamiento.

b) Ventana (‘*window’*): Determina la distancia máxima existente entre la palabra actual y la predicha.

c) Frecuencia mínima del documento (‘*min_count’*): No se tomará en cuenta palabras no tan frecuentes (raras) para entrenar al modelo.

d) Modelo (‘*model*’): Al igual que Word2Vect, este modelo puede entrenarse de 2 formas distintas (cbow y skip-gram). Se probarán ambos casos pues puede que uno de ellos se adapte mejor al problema planteado.

e) Longitud mínima del N-grama (‘*min_n*’): Permite especificar qué longitud mínima de caracteres tendrá el N-grama, este parámetro es importante pues el algoritmo iterará desde esta mínima longitud hasta la máxima, por lo que, es necesario determinar el número adecuado.

f) Longitud máxima del N-grama (‘*max_n*’): Permite especificar qué longitud máxima de caracteres tendrá el N-grama, este parámetro, es igual de importante que el anterior, pues en ello se basará el rango de los N-gramas que con el que entrenará el modelo.

Entregable:

- Tres modelos FastText entrenados

3.5.4 Construcción del modelo

La cuarta etapa explica acerca de las actividades necesarias para la construcción de los diversos modelos de clasificación empleando técnicas de Machine Learning. En esta etapa se requiere como entradas los registros vectorizados (etapa N° 3) puesto que los algoritmos solo trabajan con datos numéricos. A continuación, se detallan las actividades a realizar:

Tabla 7.

Actividades de la fase de Construcción del modelo

Actividades	Descripción	Tareas
1. Codificar la clase	Etiquetado de cada vector de manera numérica de acuerdo a su clase.	Etiquetar como 0 y 1 cada vector.
2. Particionar el dataset	Partición del dataset en train(80%) y test(20%); seguido de validación cruzada de 5-fold en el train.	Realizar primera partición de 80% para el training y 20% para el testing.
3. Aplicar técnicas de Machine Learning	Entrenamiento de los modelos con la data de train y pruebas del mismo con la data de test.	Crear la grilla para identificar los mejores parámetros de cada modelo.
		Construir un modelo de clasificación por técnica.
		Obtener los resultados del desempeño de cada modelo en una nueva data (test).

Fuente: Elaboración propia

Actividad 1: Codificar la clase

Esta actividad se realizará con la finalidad de transformar el *target* categórico (Verdadero y Falso) a uno numérico, puesto que el modelo solo funciona con datos numéricos. Asimismo, se formará un dataset solo con los vectores característicos (X) y el target numérico (Y).

Entregable:

- Dataset numérico

Actividad 2: Particionar el dataset

Es necesario llevar a cabo esta actividad para la posterior construcción del modelo de clasificación. La primera partición se realizará de manera explícita, es decir, se indicará una partición de 80% y 20% para el train y test, respectivamente. Mientras que la segunda partición se hará de manera implícita cuando se use el algoritmo de *GridSearchCV* con la data de train, en el cual se le indicará un K-fold igual a 5, esto es, del total de la data de train, la quinta parte será utilizada como data de validación en cada entrenamiento del modelo, asimismo, esta data elegida no será fija, puesto que en cada iteración, el algoritmo tomará otra quinta parte (distinta a la elegida en la iteración anterior) y la usará como data de validación, así recursivamente hasta tomar todas las quintas partes de la data de train (Figura), ello se hace con el fin de evitar obtener resultados sesgados (muy buenos o muy malos). Finalmente, la data de test no se tomará en cuenta hasta después de haber construido el modelo de clasificación, esta data servirá para evaluar el desempeño de cada modelo.

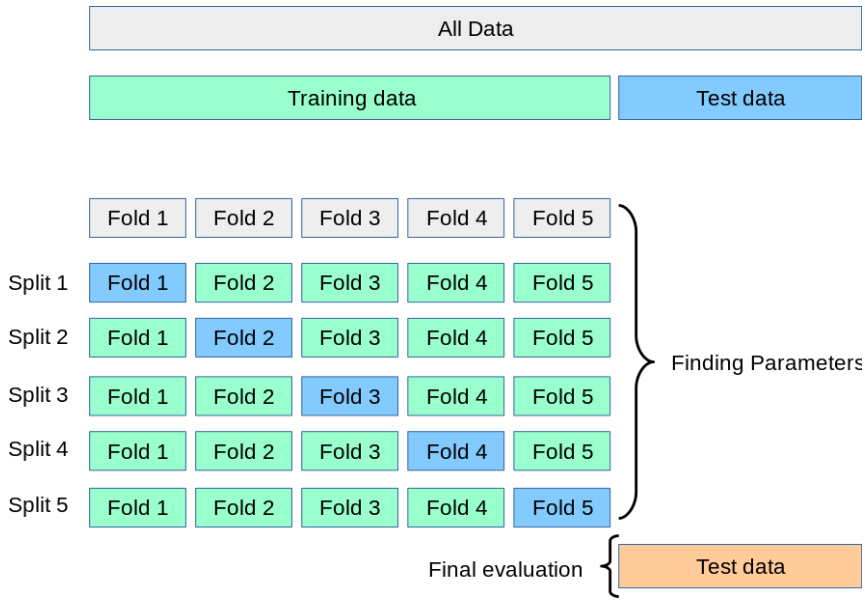


Figura 57. Validación cruzada de 5

Fuente: Runebook.dev (2020). *Validación cruzada: evaluación del rendimiento del estimador*. (p. 1)

Entregable:

- Dataset particionado en 80% y 20%

Actividad 3: Aplicar técnicas de Machine Learning

La actividad final de esta fase trata acerca de la construcción de los diversos modelos de clasificación usando técnicas de Machine Learning, para ello, es necesario identificar los parámetros clave de cada modelo:

a) KNN:

1.- Vecinos más cercanos (‘*n_neighbors*’): Representa el número de vecinos con los cuales se llevará a cabo la clasificación, es el parámetro más importante pues depende de este, el algoritmo tardará más tiempo o no, además, valores distintos pueden indicar clases distintas, es por ello es que se deben probar varios valores a fin de encontrar el óptimo.

2.- Pesos (‘*weights*’): Es otro parámetro importante, e indica cómo se debe distribuir el peso entre los valores vecinos. Si el valor es *uniform,* el algoritmo hará que los pesos se distribuyan por igual entre todos los valores vecinos, mientras que, si es *distance,* los pesos se distribuirán en función de su distancia (correlación inversa), ello implica que los vecinos más cercanos tendrán un mayor peso en el algoritmo.

3.- Distancia (‘*metric*’): Este parámetro indica el tipo de distancia a utilizar para medir la observación con cada vecino cercano.

b) Árbol de decisión

1.- Criterio (‘*criterion*’): Indica el tipo de criterio para medir la impureza de un nodo, esta función permite medir la calidad de una división.

2.- Profundidad máxima (‘*max_depth*’): Representa la profundidad máxima que puede alcanzar el árbol, es importante tener en cuenta que valores muy altos pueden ocasionar *overfitting.*

3.- Número mínimo para la partición (‘*min_sample_split*’): Representa el número mínimo de observaciones que debe tener un nodo para llevar a cabo la partición.

4.- Número mínimo de hojas (‘*min_sample_leaf*’): Representa el número mínimo de observaciones que debe tener una hoja, el cual es de gran ayuda para controlar el *overfitting*.

5.- Características (‘*max_features*’): Representa el número de características a considerar cuando se busca la mejor división

c) Naive Bayes

1.- Smoothing (‘*alpha*’): Este parámetro representa el suavizado de Laplace y es de suma importancia pues permite evitar problemas de probabilidad cero. Un valor de 0 indica que no se utilizará la técnica de Laplace.

d) SVM

1.- Costo (‘*C*’): Representa el costo de clasificaciones erróneas, un valor alto aceptará un margen más pequeño si la función de decisión clasifica mejor todos los puntos, mientras que un valor más bajo fomentará un margen de error más grande, es decir, mientras más alto sea, el algoritmo será más estricto en cuanto a evitar clasificaciones erróneas. Se debe tener cuidado que valores extremos (muy bajos o muy altos) pueden hacer que el modelo caiga en problemas de *underfitting* u *overfitting*.

2.- Gamma (‘*gamma*’): Este parámetro define hasta dónde llega la influencia de un solo ejemplo de entrenamiento, un valor bajo significa "lejos" y un valor alto significa "cerca". Es importante tener en cuenta que, si el gamma es muy grande, el radio del área de influencia de los vectores de soporte solo incluirá el propio vector de soporte y ninguna cantidad de regularización con *C* podrá evitar el *overfitting*.

3.- Kernel (‘*kernel*’): Es un parámetro igual de importante que el costo, ya que define el tipo de transformación que se le harán a los datos, esto porque ciertas dimensiones pueden ajustarse mejor que otras, lo que implica una mejor clasificación.

e) Regresión Logística:

1.- Regularización (‘*C*’): Parecido al parámetro costo de SVM, esta regularización penalizará los parámetros extremos; ya que los valores extremos en los datos de entrenamiento conducen al overfitting.

2.- Penalidad (‘*penalty*’): Especifica el tipo de penalidad que se aplicará.

3.- Solucionador (‘*solver*’): Indica las opciones para elegir el algoritmo del solucionador para la optimización. Por lo general, el solucionador predeterminado funciona muy bien en la mayoría de los casos.

f) Red Neuronal Artificial:

Este modelo es diferente a los anteriores, puesto que primero se debe crear la arquitectura de la red, para esto es necesario identificar cuántas capas escondidas habrá y cuántas neuronas tendrá cada capa, mediante esta arquitectura se definirán algunos parámetros de entrenamiento:

1.- Función de activación (‘*activation*’): Este parámetro permite colocar la función de activación que se utilizará en cada capa.

2.- Función de pérdida (‘*loss*’): La función de pérdida utilizada por el algoritmo perceptrón.

3.- Optimizador (‘*optimizer*’): El tipo de optimizador utilizado para el aprendizaje de la red.

4.- Métrica (‘*metrics*): Métrica para evaluar el desempeño de la red.

5.- Dropout (‘dropout’): Es una forma de regularización que permite evitar *overfitting* apagando cierto número de neuronas en cada época.

6.- Épocas (‘epochs’): Indica el número de épocas en el que se entrenará el modelo.

7.- Batch Size (‘batch_size’): Representa el número de muestras que se entrenan en cada época.

Como se puede observar, cada modelo tiene diversidad de parámetros clave, por lo que sería una pérdida de tiempo probar uno a uno manualmente, es por ello que es necesario crear una *grilla* en la que se especifiquen los parámetros de cada modelo para su entrenamiento. Esta grilla, es independiente de cada modelo, pues se tiene que adecuar

en base a ciertos parámetros propios de un modelo determinado. Así pues, una tarea clave será crear esta grilla para todos los modelos que se deseen construir. Para ello, el algoritmo de GridSearchCV es de gran ayuda, pues mediante este se podrá entrenar el modelo, especificando su grilla, el k-fold y la data de train. El resultado de ello serán los distintos modelos entrenados, a los cuales, posteriormente, se les pasará nueva data (test) para medir su desempeño. Estos resultados se guardarán para un posterior análisis en la siguiente etapa.

Entregable:

- Resultados de métricas de cada modelo

3.5.5 Evaluación del modelo

En la última etapa se busca evaluar el desempeño de cada modelo de clasificación construido en la etapa anterior, por ello, es necesario realizar la siguiente actividad:

Tabla 8.
Actividades de la fase de Evaluación del modelo

Actividades	Descripción	Tareas
1. Comparar todos los modelos según las métricas	Comparación de cada modelo en base a las métricas de accuracy, precision, recall y f1-score.	Encontrar el modelo con mejor accuracy, precision, recall y f1-score.

Fuente: Elaboración propia

Actividad 1: Comparar todos los modelos según las métricas

En esta única actividad, se compararán las métricas de accuracy, precision, recall y f1-score obtenidas en la fase anterior. Si bien, es probable que algunos modelos sean mejores que otros respecto a una determinada métrica, la elección se basará en el accuracy y f1-score, ya que este último identifica una buena relación de las otras 2

métricas (precision y recall). En caso existan modelos parecidos, se evaluarán las otras 2 métricas para verificar cuál es el mejor.

Entregable:

- Mejor modelo de clasificación

3.6 Metodología para la medición de resultados de la implementación

Para la medición de resultados se aplicaron los conceptos de Harrington (2012), donde se explican algunas de las métricas usadas en problemas de clasificación, las cuales pueden ser extraídas de la matriz de confusión:

		redicted	
		+1	-1
Actual	+1	True Positive (TP)	False Negative (FN)
	-1	False Positive (FP)	True Negative (TN)

Figura 58. Matriz de confusión para un problema de dos clases, con diferentes resultados etiquetados

Fuente: Harrington (2012). *Machine Learning in Action.* (p. 144)

Donde la clasificación correcta de un positivo se le conoce como True Positive (TP) y la clasificación correcta de un negativo es False Negative (FN). Mientras que la clasificación de un negativo como positivo es False Positive (FP) y lo contrario, es decir, un positivo clasificado como negativo es False Negative (FN).

a) Accuracy

Indica la fracción de predicciones correctas, tanto las positivas como las negativas. No es una buena métrica cuando se trata de datos desbalanceados.

$$\frac{TP + TN}{TP + FP + FN + TN} \tag{15}$$

b) Precision

Representa la fracción de registros que fueron positivos del grupo de predicciones y el clasificador los predijo como positivo. Una alta precision indica una menor clasificación de falsos positivos.

$$\frac{TP}{TP + FP} \tag{16}$$

c) Recall

Mide la fracción de registros reales positivos que el clasificador acertó.

$$\frac{TP}{TP + FN} \tag{17}$$

d) F1-score

Esta métrica se expresa como la media armónica entre la precision y el recall, es una medida que toma en cuenta la información proporcionada por ambas y las expresa mediante un ratio. Suele ser más útil en datos desbalanceados.

$$2 * \left(\frac{Precision * Recall}{Precision + Recall}\right) \tag{18}$$

3.7 Cronograma de actividades

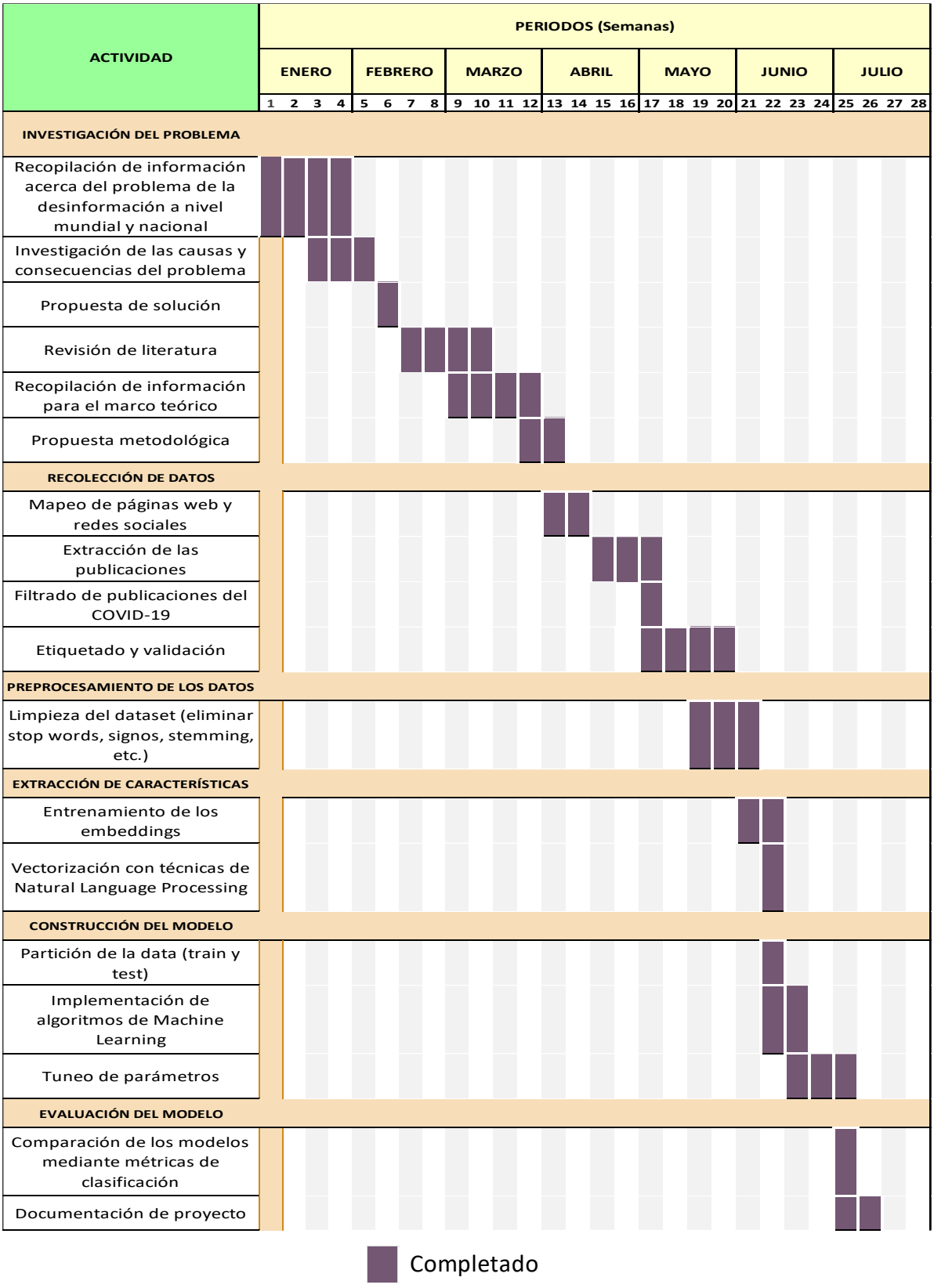


Figura 59. Cronograma de actividades

Fuente: Elaboración propia

3.8 Presupuesto

Tabla 9.
Recursos del proyecto

Recurso	Descripción	Costo (S/.)
Laptop	Dell Inspirion 15 3501 con procesador Intel Core i7	S/. 4 000
Disco SSD	Disco Solido Kingston A400 480 GB 2.5" Sata 3	S/. 240
Python	Lenguaje de programación interpretado muy utilizado en data science	-
Google Colab	Herramienta para escribir y ejecutar código de Python en la nube de Google	-
TOTAL	-	S/. 4 240

Fuente: Elaboración propia

CAPÍTULO 4: DESARROLLO DE LA SOLUCIÓN

4.1 Propuesta de solución

4.1.1 Creación del dataset

Actividad 1: Identificar fuentes de datos

En esta actividad se identificaron las fuentes de las que se extraerían los datos para el posterior armado del dataset con el cual se entrenaría el modelo.

Entregable:

- Fuentes seleccionadas de la cuales se extraerá información del COVID-19.

Primero, se eligió el tipo de plataformas con las que se trabajaría, en este caso, se optó por las redes sociales Facebook y Twitter, esto porque, según IPSOS (2021), Whatsapp, Facebook y Tiktok lideraron la frecuencia de uso en las redes sociales (ANEXO 3), por obvias razones de privacidad de mensajes no se eligió Whatsapp, de igual manera, Tiktok se descartó porque, debido a que es una red para compartir videos con un fin de entretenimiento, sería difícil encontrar información falsa del coronavirus en alguno de estos videos, es por ello que se eligió Facebook, ya que en esta plataforma se pueden encontrar publicaciones falsas así como comentarios con información falsa de COVID-19; por otro lado, si bien el uso de Twitter no es tan frecuente en el país, se pudo evidenciar que ciertos funcionarios públicos utilizan este medio para dar a conocer ciertas opiniones, sin importar si están equivocadas, por lo que también se tomó esta red como fuente de datos, así que se consideró cada departamento del país para un futuro scraping.

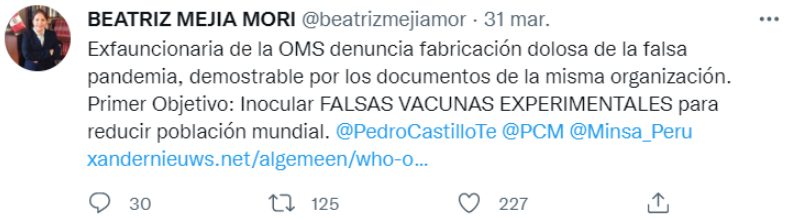


Figura 60. Ejemplo de tweet falso

Fuente: Twitter (2022)

Asimismo, otra fuente de datos identificada fueron las páginas web de algunos factcheckers peruanos (OjoPúblico y VerificadorLR) y la del propio Minsa, las cuales servirían para un posterior etiquetado, ya que, mediante ellas, se podrá determinar la veracidad o no de cada registro que trate del COVID-19. Se consideraron ambos factcheckers puesto que cuentan con el apoyo de la Global Investigative Journalism Network y la International Fact-Checking Network (respectivamente), ambas instituciones promueven el periodismo de investigación y la excelencia en la verificación de datos, lo cual es de gran ayuda en la lucha contra la desinformación.



Figura 61. Colaboración entre Global Investigative Journalism Network y Ojo Público

Fuente: Ojo Público (2022)



Figura 62. Colaboración entre International Fact-Checking Network y Verificador LR

Fuente: Verficador LR (2022)

Actividad 2: Desarrollar los algoritmos de scrapping

Se buscó trabajar con los datos más recientes, por lo que el rango de fechas a trabajar fue desde el primero de enero del presente año, hasta el 2 de mayo.

A) Facebook:

La librería *facebook-scraper* facilitó la implementación del algoritmo de scraping, ya que mediante su función *get_posts*, se pudo obtener todos los posts de una determinada cuenta. Sin embargo, para utilizar esta librería fue necesario guardar las cookies de login en la cuenta propia de Facebook. Para esto, en primer lugar, se añadió la extensión “Get cookies.txt” al navegador. (Figura 63)

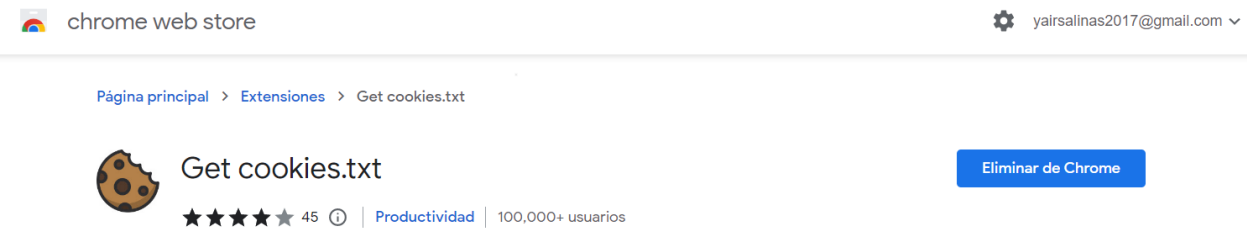
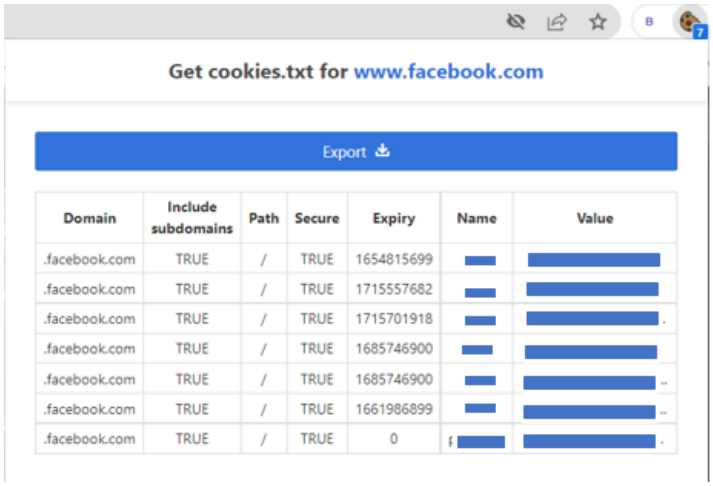


Figura 63. Extensión Get cookies.txt

Fuente: Chrome Web Store (2022)

Mediante esta extensión, al momento de iniciar sesión en Facebook, los datos de este inicio se guardaron en las cookies, como se muestra en la siguiente figura (se censuraron los valores por razones de privacidad):



Domain	Include subdomains	Path	Secure	Expiry	Name	Value
.facebook.com	TRUE	/	TRUE	1654815699		
.facebook.com	TRUE	/	TRUE	1715557682		
.facebook.com	TRUE	/	TRUE	1715701918		
.facebook.com	TRUE	/	TRUE	1685746900		
.facebook.com	TRUE	/	TRUE	1685746900		
.facebook.com	TRUE	/	TRUE	1661986899		
.facebook.com	TRUE	/	TRUE	0		

Figura 64. Datos guardados del inicio en Facebook con la extensión Get cookies.txt

Fuente: Elaboración propia

En segundo lugar, fue necesario descargar el archivo txt con los datos de las cookies, los cuales fueron parte del parámetro (‘cookies’) de la función para realizar el scraping. Esta función, además, tiene otros parámetros como el nombre de la cuenta (‘group’), la fecha final hasta la cual se realizará el scraping (‘lastest_date’) y la opción de extraer posts con comentarios (‘comments’), estos fueron los parámetros que se usaron en el scraping. Cabe mencionar que la fecha de inicio no se puede pasar como un parámetro, por defecto, el scraping inicia desde el momento actual en que se ejecuta el algoritmo. Asimismo, si bien esta librería es una herramienta potente para extraer datos

de Facebook, no es posible extraer todos los datos solicitados, puede darse el caso que no se consigan algunos posts en una determinada fecha o no se obtengan todos los comentarios de un post.

El algoritmo devolvió una lista de posts por cada cuenta, cada post era un diccionario, donde una *key* de este diccionario fue 'comments_full', lo que representó la lista de comentarios totales de ese post.

El scraping de datos resultó ser una tarea que tomó bastante tiempo (dependiendo de la cuenta de facebook se podía llegar fácilmente a las 6 horas), además, la extracción se realizó en 2 puntos distintos (15 de abril y 30 de abril) y en paralelo con otras cuentas de colab (3 cuentas propias en total) con la finalidad de agilizar el proceso. Es por ello que fue necesario almacenar los datos conforme se terminaban ciertas ejecuciones, entonces, se optó por guardar cada lista de post (resultado de una ejecución) en un archivo .npy usando la función *save* de la librería NumPy, esto con la finalidad de cargarlos más adelante para su posterior tratamiento. Como se mencionó, se scrapearon los datos en 2 fechas específicas, en la primera parte, se fijó una fecha final de 1/01/2022, mientras que, para la segunda, la fecha final fue 15/04/2022; en lo que respecta al nombre de la cuenta, esta se obtuvo entrando al perfil y se tomaron los últimos caracteres de su url, en la figura 65 se puede observar que la cuenta del Minsa sería 'minsaperu', este fue el parámetro que se le pasó al algoritmo. Finalmente, se habilitó la extracción de comentarios definiendo *True* en el parámetro 'comments'.

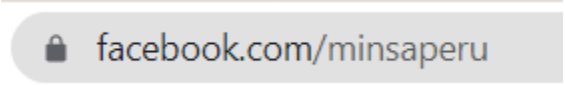


Figura 65. Cuenta de Facebook del MINSA

Fuente: Elaboración propia

Con todos los parámetros definidos, se llevó a cabo el scraping de datos. La siguiente tabla muestra los resultados del total de posts extraídos por cuenta:

Tabla 10.

Datos recopilados por cuenta de Facebook

Cuenta	Cantidad

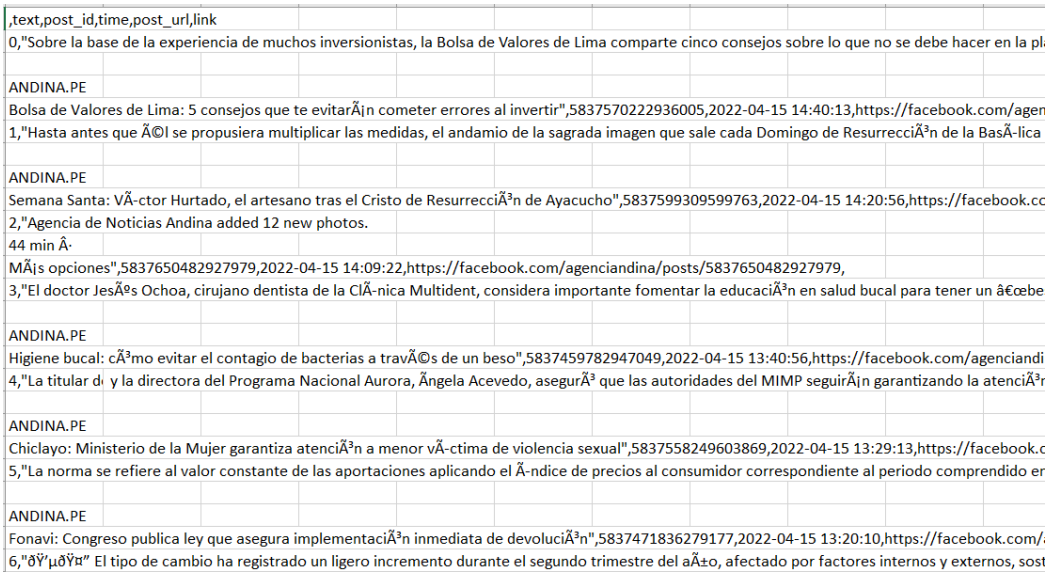
Perú 21	3 144
La República	3 122
RPP	3 106
El Comercio	3 100
El Peruano	2 536
Andina	2 531
Tv Perú	2 500
Minsa	1 471
Total	21 510

Fuente: Elaboración propia

Una vez se obtuvieron todos los posts, se procedió a la lectura de cada archivo con la función *load* de NumPy (se habilitó la carga de *pickle* en el parámetro *allow_pickle*) y se unieron todas las listas de posts en una sola, posteriormente, se comenzó la extracción de comentarios de cada post presente en esta lista, para esto fue necesario la implementación de un algoritmo que recorra cada post y obtenga la lista de todos los comentarios de dicho post, y, a su vez, recorra cada comentario para obtener la lista de respuestas a ese comentario. Al final, se obtuvieron 1 079 238 comentarios y 202 862 respuestas, puesto que estas respuestas también son comentarios, entonces se puede decir que se obtuvieron 1 282 100 comentarios en total. Finalmente, tanto los posts como los comentarios se guardaron en un archivo csv para su posterior tratamiento.

Entregable:

1. Archivo csv de posts



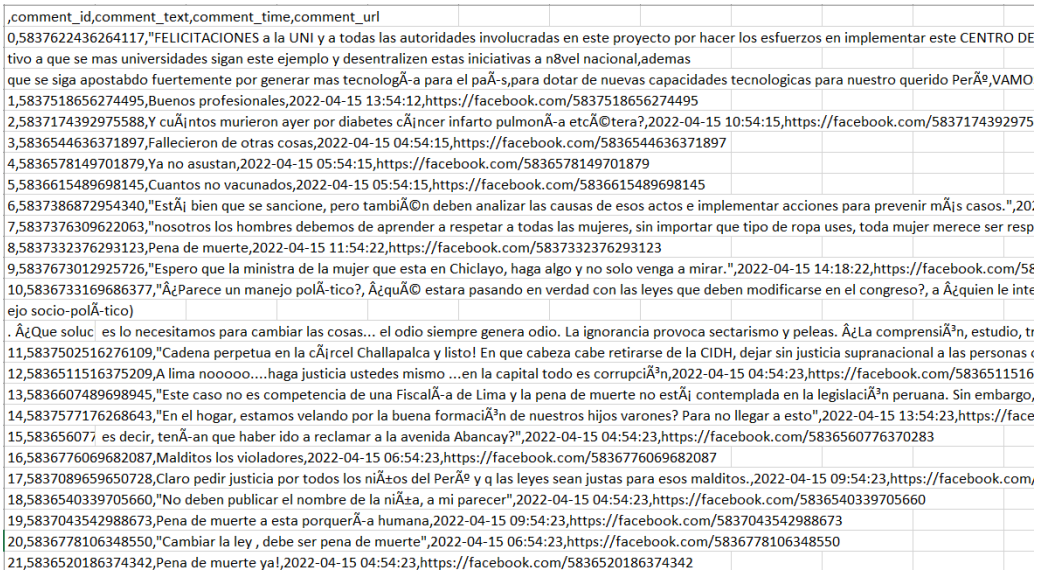
,text,post_id,time,post_url,link
0,"Sobre la base de la experiencia de muchos inversionistas, la Bolsa de Valores de Lima comparte cinco consejos sobre lo que no se debe hacer en la pl
ANDINA.PE
Bolsa de Valores de Lima: 5 consejos que te evitarÃ¡n cometer errores al invertir",5837570222936005,2022-04-15 14:40:13,https://facebook.com/agen
1,"Hasta antes que Ã©l se propusiera multiplicar las medidas, el andamio de la sagrada imagen que sale cada Domingo de ResurrecciÃ³n de la BasÃ-lica
ANDINA.PE
Semana Santa: VÃ-ctor Hurtado, el artesano tras el Cristo de ResurrecciÃ³n de Ayacucho",5837599309599763,2022-04-15 14:20:56,https://facebook.cc
2,"Agencia de Noticias Andina added 12 new photos.
44 min Â·
MÃ¡s opciones",5837650482927979,2022-04-15 14:09:22,https://facebook.com/agenciandina/posts/5837650482927979,
3,"El doctor JesÃºs Ochoa, cirujano dentista de la ClÃ-nica Multident, considera importante fomentar la educaciÃ³n en salud bucal para tener un â€œbe
ANDINA.PE
Higiene bucal: cÃ³mo evitar el contagio de bacterias a travÃ©s de un beso",5837459782947049,2022-04-15 13:40:56,https://facebook.com/agenciandi
4,"La titular d y la directora del Programa Nacional Aurora, Ángela Acevedo, asegurÃ³ que las autoridades del MIMP seguirÃ¡n garantizando la atenciÃ³r
ANDINA.PE
Chiclayo: Ministerio de la Mujer garantiza atenciÃ³n a menor vÃ-ctima de violencia sexual",5837558249603869,2022-04-15 13:29:13,https://facebook.c
5,"La norma se refiere al valor constante de las aportaciones aplicando el Ã-ndice de precios al consumidor correspondiente al periodo comprendido er
ANDINA.PE
Fonavi: Congreso publica ley que asegura implementaciÃ³n inmediata de devoluciÃ³n",5837471836279177,2022-04-15 13:20:10,https://facebook.com/
6,"ðŸ'µðŸ¤" El tipo de cambio ha registrado un ligero incremento durante el segundo trimestre del aÃ±o, afectado por factores internos y externos, sost

Figura 66. Archivo csv de posts

Fuente: Elaboración propia

Entregable:

2. Archivo csv de comentarios.



,comment_id,comment_text,comment_time,comment_url
0,5837622436264117,"FELICITACIONES a la UNI y a todas las autoridades involucradas en este proyecto por hacer los esfuerzos en implementar este CENTRO DE
tivo a que se mas universidades sigan este ejemplo y desentralizen estas iniciativas a n8vel nacional,ademas
que se siga apostabdo fuertemente por generar mas tecnologÃ-a para el paÃ-s,para dotar de nuevas capacidades tecnologicas para nuestro querido PerÃº,VAMO
1,5837518656274495,Buenos profesionales,2022-04-15 13:54:12,https://facebook.com/5837518656274495
2,5837174392975588,Y cuÃ¡ntos murieron ayer por diabetes cÃ¡ncer infarto pulmonÃ-a etcÃ©tera?,2022-04-15 10:54:15,https://facebook.com/5837174392975
3,5836544636371897,Fallecieron de otras cosas,2022-04-15 04:54:15,https://facebook.com/5836544636371897
4,5836578149701879,Ya no asustan,2022-04-15 05:54:15,https://facebook.com/5836578149701879
5,5836615489698145,Cuantos no vacunados,2022-04-15 05:54:15,https://facebook.com/5836615489698145
6,5837386872954340,"EstÃ¡ bien que se sancione, pero tambiÃ©n deben analizar las causas de esos actos e implementar acciones para prevenir mÃ¡s casos.",20
7,5837376309622063,"nosotros los hombres debemos de aprender a respetar a todas las mujeres, sin importar que tipo de ropa uses, toda mujer merece ser resp
8,5837332376293123,Pena de muerte,2022-04-15 11:54:22,https://facebook.com/5837332376293123
9,5837673012925726,"Espero que la ministra de la mujer que esta en Chiclayo, haga algo y no solo venga a mirar.",2022-04-15 14:18:22,https://facebook.com/58
10,5836733169686377,"Â¿Parece un manejo polÃ-tico?, Â¿quÃ© estara pasando en verdad con las leyes que deben modificarse en el congreso?, a Â¿quien le inte
ejo socio-polÃ-tico)
. Â¿Que soluc es lo necesitamos para cambiar las cosas... el odio siempre genera odio. La ignorancia provoca sectarismo y peleas. Â¿La comprensiÃ³n, estudio, t
11,5837502516276109,"Cadena perpetua en la cÃ¡rcel Challapalca y listo! En que cabeza cabe retirarse de la CIDH, dejar sin justicia supranacional a las personas
12,5836511516375209,A lima nooooo....haga justicia ustedes mismo ...en la capital todo es corrupciÃ³n,2022-04-15 04:54:23,https://facebook.com/5836511516
13,5836607489698945,"Este caso no es competencia de una FiscalÃ-a de Lima y la pena de muerte no estÃ¡ contemplada en la legislaciÃ³n peruana. Sin embargo,
14,5837577176268643,"En el hogar, estamos velando por la buena formaciÃ³n de nuestros hijos varones? Para no llegar a esto",2022-04-15 13:54:23,https://face
15,583656077 es decir, tenÃ-an que haber ido a reclamar a la avenida Abancay?",2022-04-15 04:54:23,https://facebook.com/5836560776370283
16,5836776069682087,Malditos los violadores,2022-04-15 06:54:23,https://facebook.com/5836776069682087
17,5837089659650728,Claro pedir justicia por todos los niÃ±os del PerÃº y q las leyes sean justas para esos malditos.,2022-04-15 09:54:23,https://facebook.com/
18,5836540339705660,"No deben publicar el nombre de la niÃ±a, a mi parecer",2022-04-15 04:54:23,https://facebook.com/5836540339705660
19,5837043542988673,Pena de muerte a esta porquerÃ-a humana,2022-04-15 09:54:23,https://facebook.com/5837043542988673
20,5836778106348550,"Cambiar la ley , debe ser pena de muerte",2022-04-15 06:54:23,https://facebook.com/5836778106348550
21,5836520186374342,Pena de muerte ya!,2022-04-15 04:54:23,https://facebook.com/5836520186374342

Figura 67. Archivo csv de comentarios

Fuente: Elaboración propia

B) Twitter:

El scraping en esta plataforma también resultó ser un poco complicado debido a la gran cantidad de datos existentes, por suerte, Twitter proporciona un api llamado *Tweepy* que facilita el proceso de scraping. Para el correcto funcionamiento de este api, fue necesario crearse una cuenta *Developer* en Twitter, ya que, mediante esta cuenta, se puede acceder a cierta cantidad de los datos que brinda Twitter de manera gratuita o de paga, dependiendo del tipo de cuenta que se elija.

Una vez iniciado el login en la cuenta, se procedió a identificar las claves y los tokens de acceso para realizar la conexión entre *Tweepy* y la cuenta *Developer*.

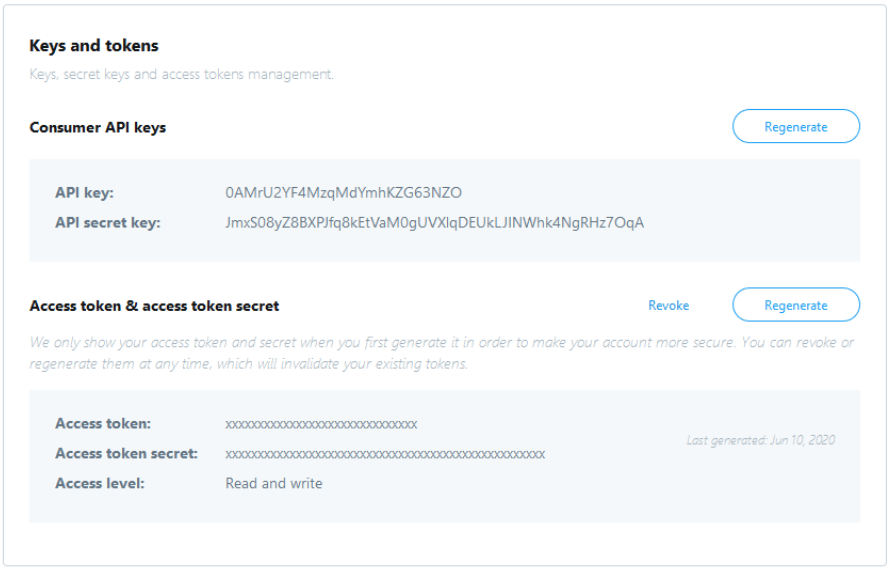


Figura 68. Credenciales de Tweepy

Fuente: Elaboración propia

Después de realizar la conexión con la función *API*, se comenzó a scrapear tweets utilizando la función *Cursor* de *tweepy*, a la cual, se le pasaron los parámetros de tipo de búsqueda (‘search’), el contenido a buscar (‘q’), el lenguaje (‘lang’), modo del tweet (‘tweet_mode’) y las coordenadas de geolocalización (‘geocode’), finalmente, fue necesario especificar cuántos tweets se extraerían por cada ejecución (‘items’).

En lo que respecta al primer parámetro, puesto que no se quería buscar cuentas en particular, sino solo tweets, se decidió optar por la función *search*, ya que esta busca específicamente tweets; para el segundo parámetro, se definieron 3 *keywords* de búsqueda relacionados al tema a tratar: ‘covid’, ‘coronavirus’, ‘vacunas’; como solo se

deseaba trabajar con tweets en español (‘es’), se filtraron los tweets de acuerdo a este idioma en el tercer parámetro; en el siguiente parámetro, se consideró un modo ‘extended’, ello con la final de no obtener el tweet truncado, sino todo su contenido; para la localización de las coordenadas geográficas, se utilizó la página web Geocode.xyz (ANEXO 4) en la cual se identificaron las coordenadas de los 24 departamentos del Perú y la Provincia Constitucional del Callao, asimismo, se tomó un radio que varió entre los 150km y 1000km alrededor de cada coordenada con la finalidad de abarcar mayor territorio peruano.

Debido a que el API de Twitter tiene cierto límite para la extracción de tweets (se utilizó la cuenta *Developer* en su versión ‘Essential’), las solicitudes se hicieron en 2 partes: la primera se llevó a cabo el 25 de abril y la 2da, el 30 del mismo mes. En lo que respecta a la primera parte, se buscaron tweets que tengan "coronavirus" como keyword, mientras que para la segunda, se buscaron "vacunas" y "covid" como keywords. Una de las principales desventajas de utilizar esta API fue que no se podían extraer tweets que pasen los 7 días de antigüedad, tampoco se podían pedir más 3200 tweets en un request y solo se permitían 25 request como máximo en un lapso de 15 minutos, en caso se excedía este límite, era necesario esperar un tiempo aproximado de 15 minutos para realizar nuevos requests (ANEXO 5), esto implicó correr el riesgo que den de baja la cuenta *Developer* por realizar muchas solicitudes a la vez (en el peor de los casos), por lo que fue necesario ser cauteloso con los tipos de solicitudes a mandar. Ello dificultó el agrupamiento del código en un solo algoritmo, ya que muchos requests podían ocasionar problemas, por lo que se decidió realizar el algoritmo de scraping para cada departamento por separado. Cabe mencionar que lo único que varió en cada algoritmo fue el área abarcada (coordenadas + radio) y el número de items solicitados (en algunos casos 2200 y en otros 3200, dependiendo del time sleep).

Finalmente, cada tweet recolectado por departamento se almacenó en una lista referente a este departamento y se guardó en un archivo .npy para ser utilizado más adelante. La tabla 11 muestra la cantidad de tweets recolectados en total por departamento.

Tabla 11.
Datos recopilados de Twitter por departamento

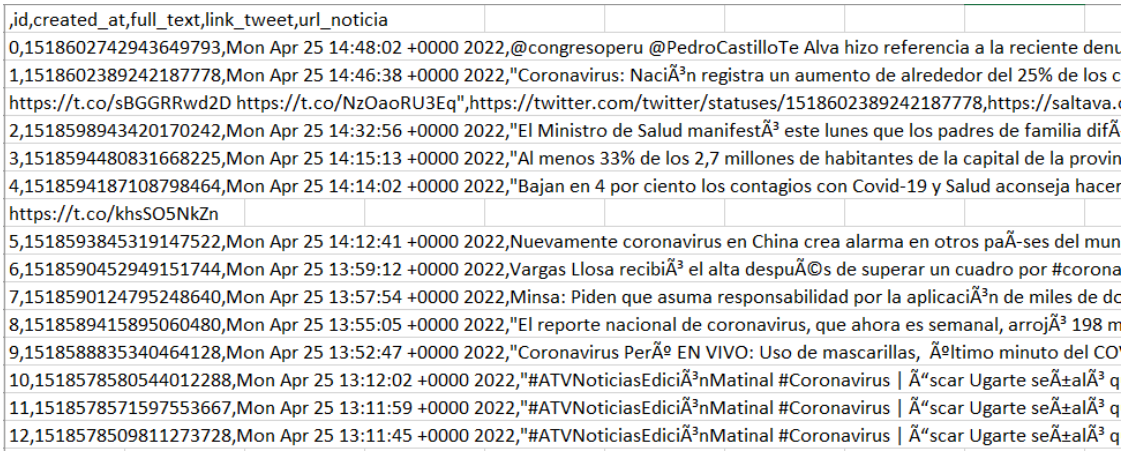
Departamento	Cantidad
Amazonas	1 810
Áncash	1 559
Apurímac	1 104
Arequipa	1 242
Ayacucho	1 050
Cajamarca	5 179
Callao	2 535
Cusco	1 823
Huancavelica	2 547
Huánuco	3 259
Ica	2 557
Junín	2 569
La Libertad	2 200
Lambayeque	2 200
Lima	3 170
Loreto	2 200
Madre de Dios	1 669
Moquegua	1 632
Pasco	1 687
Piura	1 365
Puno	1 604
San Martín	2 882
Tacna	1 635
Tumbes	2 200
Ucayali	2 200
Total	53 878

Fuente: Elaboración propia

Si bien estos tweets ya trataban temas acerca del coronavirus, aún no estaban listos para formar parte del dataset final. Toda esta data se guardó en un archivo csv.

Entregable:

3. Archivo csv de tweets.



,id,created_at,full_text,link_tweet,url_noticia
0,1518602742943649793,Mon Apr 25 14:48:02 +0000 2022,@congresoperu @PedroCastilloTe Alva hizo referencia a la reciente denu
1,1518602389242187778,Mon Apr 25 14:46:38 +0000 2022,"Coronavirus: NaciÃ³n registra un aumento de alrededor del 25% de los c
https://t.co/sBGGRRwd2D https://t.co/NzOaoRU3Eq",https://twitter.com/twitter/statuses/1518602389242187778,https://saltava.(
2,1518598943420170242,Mon Apr 25 14:32:56 +0000 2022,"El Ministro de Salud manifestÃ³ este lunes que los padres de familia difÃ
3,1518594480831668225,Mon Apr 25 14:15:13 +0000 2022,"Al menos 33% de los 2,7 millones de habitantes de la capital de la provin
4,1518594187108798464,Mon Apr 25 14:14:02 +0000 2022,"Bajan en 4 por ciento los contagios con Covid-19 y Salud aconseja hacer
https://t.co/khsSO5NkZn
5,1518593845319147522,Mon Apr 25 14:12:41 +0000 2022,Nuevamente coronavirus en China crea alarma en otros paÃ-ses del mun
6,1518590452949151744,Mon Apr 25 13:59:12 +0000 2022,Vargas Llosa recibiÃ³ el alta despuÃ©s de superar un cuadro por #corona
7,1518590124795248640,Mon Apr 25 13:57:54 +0000 2022,Minsa: Piden que asuma responsabilidad por la aplicaciÃ³n de miles de dc
8,1518589415895060480,Mon Apr 25 13:55:05 +0000 2022,"El reporte nacional de coronavirus, que ahora es semanal, arrojÃ³ 198 m
9,1518588835340464128,Mon Apr 25 13:52:47 +0000 2022,"Coronavirus PerÃº EN VIVO: Uso de mascarillas, Ãºltimo minuto del CO
10,1518578580544012288,Mon Apr 25 13:12:02 +0000 2022,"#ATVNoticiasEdiciÃ³nMatinal #Coronavirus | Ã“scar Ugarte seÃ±alÃ³ q
11,1518578571597553667,Mon Apr 25 13:11:59 +0000 2022,"#ATVNoticiasEdiciÃ³nMatinal #Coronavirus | Ã“scar Ugarte seÃ±alÃ³ q
12,1518578509811273728,Mon Apr 25 13:11:45 +0000 2022,"#ATVNoticiasEdiciÃ³nMatinal #Coronavirus | Ã“scar Ugarte seÃ±alÃ³ q

Figura 69. Archivo csv de tweets

Fuente: Elaboración propia

C) Páginas web:

En el caso de las páginas web, se implementó un algoritmo de scraping por cada una, debido a que la estructura html de las mismas era diferente, por lo que no se podía crear un solo algoritmo que agrupe a las 3; sin embargo, se utilizaron las mismas librerías: *Selenium* y *BeatifulSoup.*

Puesto que se buscó realizar un scraping dinámico y no estático, se decidió optar por la librería *Selenium* y no por *Request*, ya que mediante esta librería se pudo interactuar con las páginas (enviar consultas, cerrar ventanas emergentes, clickear,etc.), lo que facilitó en gran medida la extracción de datos. Asimismo, fue necesario la instalación de un *driver* que permita la navegación por estas páginas, como se trabajó en *colab*, la instalación del mismo no fue una tarea complicada, solo se necesitó identificar el tipo de *driver* con el que se quería trabajar (se utilizó 'Chrome').

En lo que respecta a la siguiente librería, *BeautifulSoup,* esta sirvió para parcear los htmls extraídos con *Selenium.* Se optó por esta librería puesto que, si bien, existen otras formas de extraer información del html, expresiones regulares, por ejemplo,

BeautifulSoup brinda una forma rápida de trabajo, lo cual facilitó en gran medida la extracción de información necesaria.

1.- Web del MINSA (Sección de noticias)

Para esta web, lo primero que se hizo fue encontrar el link con el cual se iniciaría el scraping, por lo cual, se fijó que la información a extraer serían las noticias del portal de noticias del Minsa. Una vez ingresado a esta sección, se copió la url y se le pasó como parámetro al *driver* mediante su método *get.* Posteriormente, se quiso seguir el mismo rango de fechas que en los anteriores scrapings, así que también se buscó obtener noticias con límite al 1 de enero de este año, por lo que se definió un número aproximado de páginas que recorrería el algoritmo, para ello, se tomó como referencia el número de publicaciones que realizaban en cada página (20 noticias por página), asimismo, se identificó que, en promedio, mensualmente publicaban alrededor de 190 noticias (lo que equivale de 9 a 10 páginas por mes). El scraping se llevó a cabo el 1 de mayo y, debido a que solo que quiso trabajar con los meses de enero hasta abril de 2022 (4 meses), se fijó un número de página igual a 38, entonces, el algoritmo de scraping tomó todas las noticias publicadas desde la página 1 hasta la número 38.

Una vez definido el número de página final, se inició con el análisis de la estructura de la web, producto de este, se concluyó que una forma fácil de avanzar página a página sería clickeando el número de la misma, también se podía hacer clickeando la opción ‘Siguiente’, pero se optó por tomar el número. Entonces, como se muestra en la figura 70, estos números tienen un texto con link, los cuales redirigen a su pestaña linkeada, es por eso que se tomó la función *find_element* del *driver* y se le pasó el valor del texto (número de la página), el cual sería clickeado para avanzar de página. Asimismo, en cada página se recolectaría el html usando la función *page_source* del mismo *driver*. Cabe mencionar que se colocó un tiempo de *sleep* igual a 3 en cada página a fin de que el algoritmo pueda esperar que se cargue el html, ya que, de lo contrario, podía darse el caso que mostrara error cuando se buscaba interactuar con algún elemento de la página que no se logró cargar.

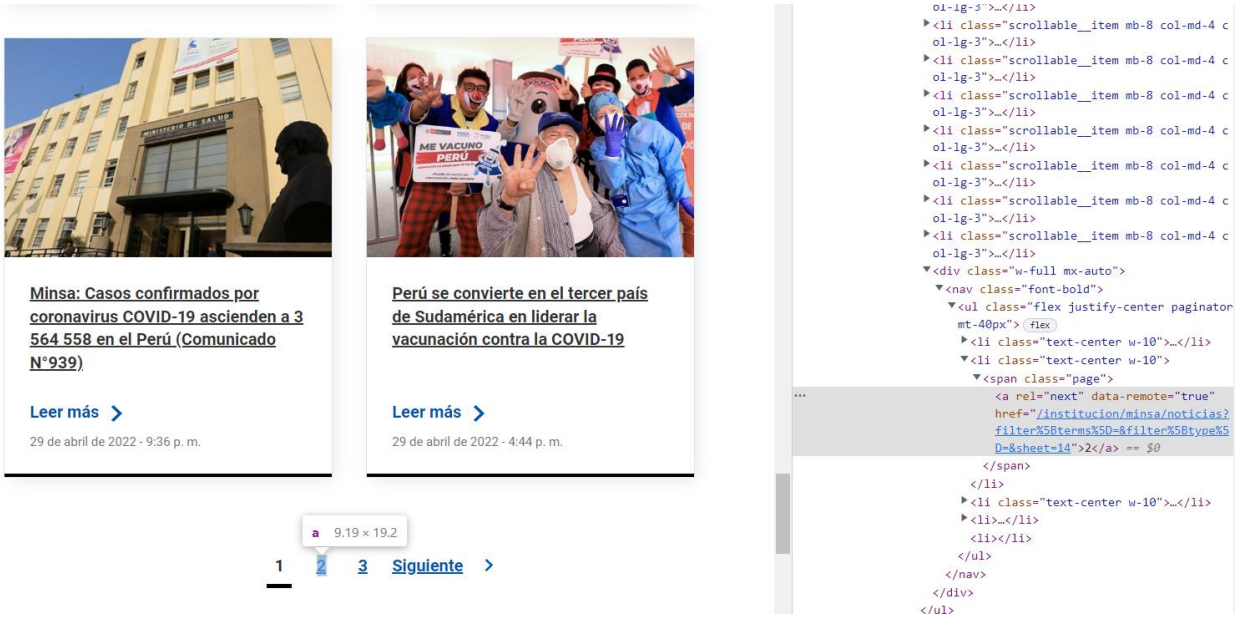


Figura 70. Inspección de la web de noticias del MINSA

Fuente: Elaboración propia

En este primer procedimiento se logró extraer el html de las 38 páginas, lo que se hizo a continuación fue parsear el html y obtener solo los atributos más importantes que conformarían este dataset: título, fecha y link; de igual manera, se analizó la estructura en busca de estos atributos. La figura 71 muestra que los 3 atributos se encontraban dentro de la etiqueta *div* cuya clase era *"card border relative bg-white flex shadow-cards min-h-14 h-full flex-col"*, así que se agrupó en una lista todos los *div* que tenían esta clase mediante la función *find_all* de *BeautifulSoup*.

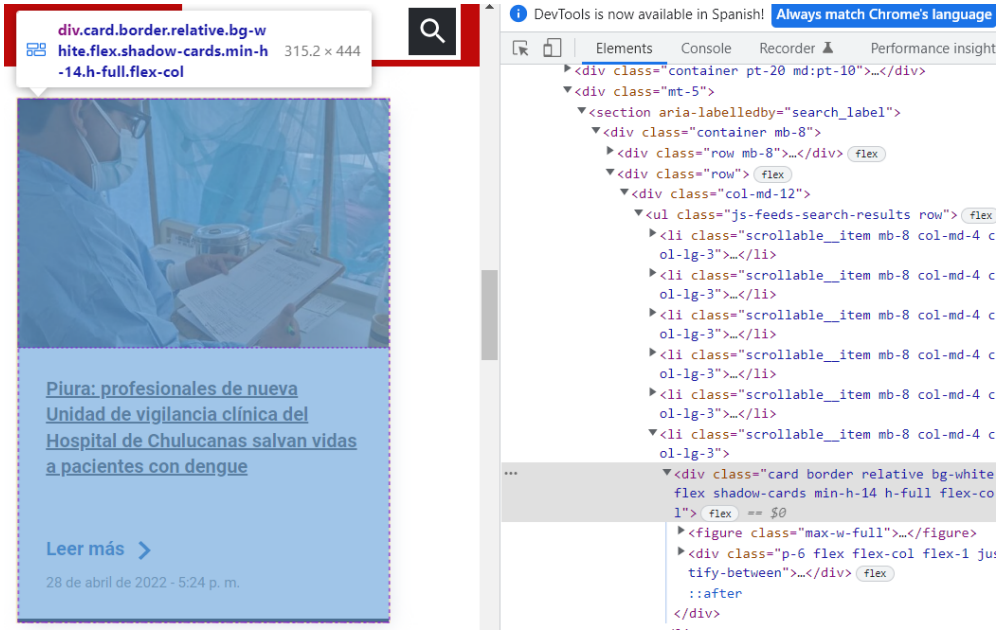


Figura 71. Atributos del div de noticias del MINSA

Fuente: Elaboración propia

Con esta lista de noticias, se procedió a la extracción del título, la fecha y el link. Se identificó que el atributo *text* contenía al título de la noticia (Figura 72); mientras que la etiqueta *time* abarcaba a la fecha (Figura 73), finalmente, para el link, este se encontró en el mismo texto de la noticia, puesto que al ser un texto linkeado, tanto el título como el link estaban en el mismo lugar, específicamente, *a* fue la etiqueta del link. (Figura 74)

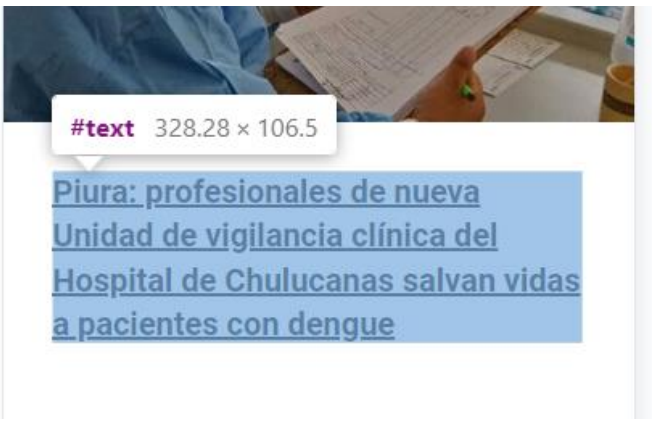


Figura 72. Título de la noticia dentro del atributo “text”

Fuente: Elaboración propia

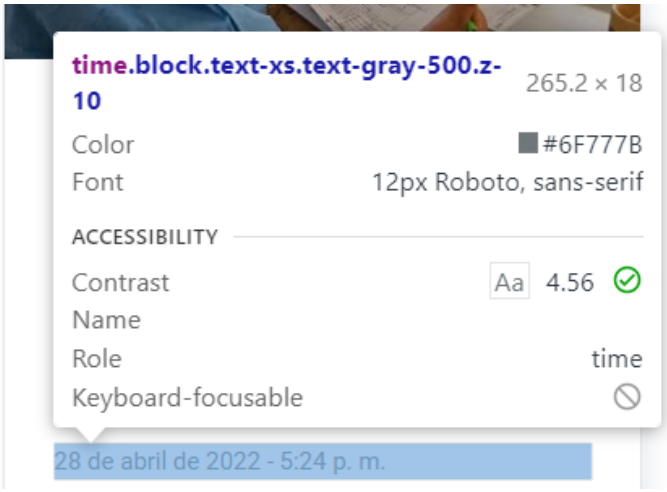


Figura 73. Fecha de la noticia dentro del atributo “time”

Fuente: Elaboración propia

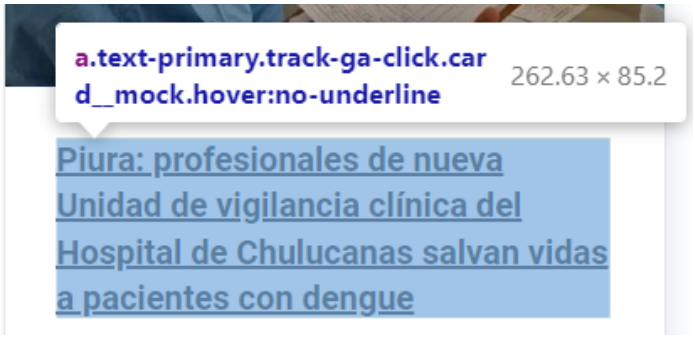


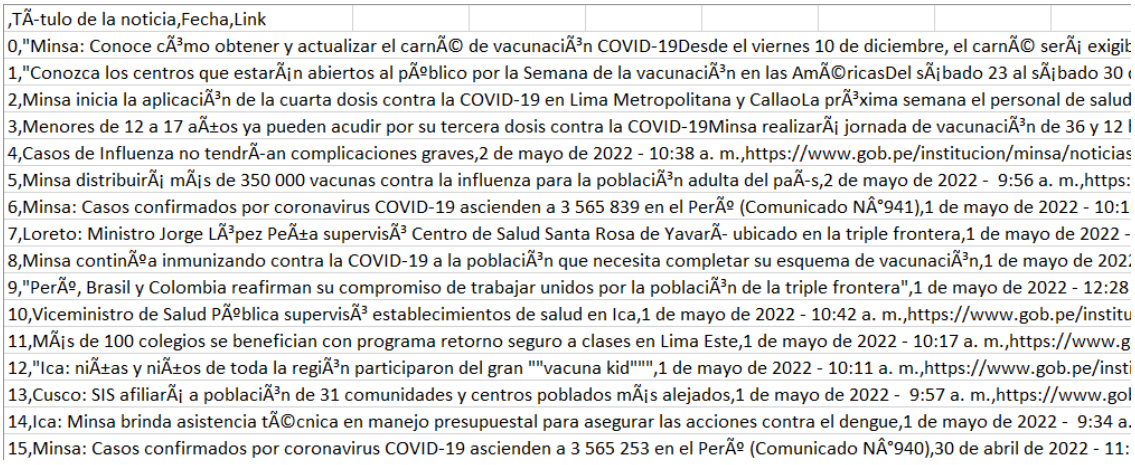
Figura 74. Link de la noticia dentro del atributo “a”

Fuente: Elaboración propia

El resultado de este scraping fueron 912 noticias en total, esto porque habían 4 noticias que pertenecían a la sección "Noticias Destacadas" (ANEXO 6) las cuales aparecían en cada página, además, cabe mencionar que el algoritmo extrajo todas las noticias en general publicadas en este periodo, sin importar si se trataba acerca del COVID-19 o no, es por eso que las noticias se guardaron en un archivo .csv para su posterior preprocesamiento.

Entregable:

4. Archivo csv noticias del Minsa



,TÃ-tulo de la noticia,Fecha,Link							
0,"Minsa: Conoce cÃ³mo obtener y actualizar el carnÃ© de vacunaciÃ³n COVID-19Desde el viernes 10 de diciembre, el carnÃ© serÃ¡ exigib							
1,"Conozca los centros que estarÃ¡n abiertos al pÃºblico por la Semana de la vacunaciÃ³n en las AmÃ©ricasDel sÃ¡bado 23 al sÃ¡bado 30							
2,Minsa inicia la aplicaciÃ³n de la cuarta dosis contra la COVID-19 en Lima Metropolitana y CallaoLa prÃ³xima semana el personal de salud							
3,Menores de 12 a 17 aÃ±os ya pueden acudir por su tercera dosis contra la COVID-19Minsa realizarÃ¡ jornada de vacunaciÃ³n de 36 y 12 l							
4,Casos de Influenza no tendrÃ-an complicaciones graves,2 de mayo de 2022 - 10:38 a. m.,https://www.gob.pe/institucion/minsa/noticias							
5,Minsa distribuirÃ¡ mÃ¡s de 350 000 vacunas contra la influenza para la poblaciÃ³n adulta del paÃ-s,2 de mayo de 2022 - 9:56 a. m.,https:							
6,Minsa: Casos confirmados por coronavirus COVID-19 ascienden a 3 565 839 en el PerÃº (Comunicado NÂ°941),1 de mayo de 2022 - 10:1							
7,Loreto: Ministro Jorge LÃ³pez PeÃ±a supervisÃ³ Centro de Salud Santa Rosa de YavarÃ- ubicado en la triple frontera,1 de mayo de 2022 -							
8,Minsa continÃºa inmunizando contra la COVID-19 a la poblaciÃ³n que necesita completar su esquema de vacunaciÃ³n,1 de mayo de 202							
9,"PerÃº, Brasil y Colombia reafirman su compromiso de trabajar unidos por la poblaciÃ³n de la triple frontera",1 de mayo de 2022 - 12:28							
10,Viceministro de Salud PÃºblica supervisÃ³ establecimientos de salud en Ica,1 de mayo de 2022 - 10:42 a. m.,https://www.gob.pe/institu							
11,MÃ¡s de 100 colegios se benefician con programa retorno seguro a clases en Lima Este,1 de mayo de 2022 - 10:17 a. m.,https://www.g							
12,"Ica: niÃ±as y niÃ±os de toda la regiÃ³n participaron del gran ""vacuna kid""",1 de mayo de 2022 - 10:11 a. m.,https://www.gob.pe/insti							
13,Cusco: SIS afiliarÃ¡ a poblaciÃ³n de 31 comunidades y centros poblados mÃ¡s alejados,1 de mayo de 2022 - 9:57 a. m.,https://www.gol							
14,Ica: Minsa brinda asistencia tÃ©cnica en manejo presupuestal para asegurar las acciones contra el dengue,1 de mayo de 2022 - 9:34 a.							
15,Minsa: Casos confirmados por coronavirus COVID-19 ascienden a 3 565 253 en el PerÃº (Comunicado NÂ°940),30 de abril de 2022 - 11:							

Figura 75. Archivo csv noticias del Minsa

Fuente: Elaboración propia

2.- Verificador (Factcheking)

El scraping se realizó el 2 de mayo y, de igual manera que en la página anterior, se identificó que la url principal con la cual se llevaría a cabo el procedimiento sería el de la sección “NOTICIAS”, la cual se pasó como parámetro al método *get.* A diferencia de la web del Minsa, en esta página se mostró una ventana emergente cuando se ingresaba con el *driver* (ANEXO 7), lo cual dificultaba la navegación en la misma, es por ello que fue necesario cerrar dicha ventana, para ello se identificó que la etiqueta *button* del botón ‘Acepto’ tenía la clase *“css-47sehv”*, mediante el *driver* se logró clickear este botón y desaparecer la ventana.

En esta web pasaba algo particular: no existían números de páginas, solo un botón “VER MÁS”, como se muestra en la figura 76. Esto resultó ser un problema de cierto modo puesto que no se sabía hasta qué punto se scrapearían las noticias. Es por ello que se decidió optar por un bucle *while* en el que se dé click al botón hasta que no haya nada más noticias por mostrar. Este botón se encontraba en la etiqueta *span* y la clase *“loadMore box-verificador__titles”*, ambos parámetros se pasaron a la función *“find_element_by_css_selector”* del *driver.*



Figura 76. Botón "VER MÁS" de Verificador

Fuente: Elaboración propia

Una vez mostradas todas las noticias, se obtuvo el html de la pestaña mediante la función *page_source*. Luego, se siguió el mismo procedimiento que en la web anterior. Primero, se parseó el html, después, se creó la lista de noticias, en este caso, el atributo que agrupaba cada noticia se encontró en la etiqueta *div* con la clase *boxVerificador* (Figura 77). Se pasaron ambos valores en la función *find_all* para obtener la lista de noticias.



Figura 77. Noticia dentro de la clase "boxVerificador"

Fuente: Elaboración propia

A continuación, se identificaron los atributos claves para el dataset: título y link; en esta parte no se encontró el atributo de fecha, por lo que este atributo no se tomó en cuenta para la extracción. De igual manera que el caso anterior, tanto el título como el link se encontraron en el mismo lugar, dentro de una etiqueta *a* (Figura 78), el título tenía un atributo *text*, mientras que el atributo del link fue *href*. Finalmente, las 133 noticias extraídas se guardaron en un archivo csv.



Figura 78. Título y link de la noticia dentro del atributo “a”

Fuente: Elaboración propia

Entregable:

5. Archivo csv noticias de Verificador LR

,TÃ-tulo de la noticia,Link
0,"No, La RepÃºblica no ha difundido supuesta cita de AnÃ-bal Torres sobre entrega de pasaportes",https://larepublica.pe//verifica
1,Es impreciso seÃ±alar que China llamÃ³ a George Soros â€œterrorista globalâ€,https://larepublica.pe//verificador/2022/04/29/e
2,"No, foto de un ataque ruso en Ucrania no corresponde a una â€œexplosiÃ³n de gasâ€ en 2018",https://larepublica.pe//verificad
3,"No, imÃ¡genes no exponen â€œcambio de banderaâ€ en una planta de MariÃºpol en 2022",https://larepublica.pe//verificador/
4,"No, excomandos de ChavÃ-n de HuÃ¡ntar no llevaron una foto de Vladimiro Montesinos a ceremonia",https://larepublica.pe//v
5,La OMS emi un estudio de Pfizer sugiere que la vacuna contra el Covid podrÃ-a ser al culpable,https://larepublica.pe/verificador
6,Pfizer afirmÃ³ que la vacuna contra la COVID-19 â€œpodrÃ-a ser culpableâ€ del brote de hepatitis,https://larepublica.pe//verific
7,La vacuna Pfizer estÃ¡ asociada con los riesgos de miocarditis en niÃ±os ,https://larepublica.pe//verificador/2022/04/27/post-sc
8,"No, la Corte finlandesa no â€œpermiteâ€ el abuso sexual de menores de edad",https://larepublica.pe//verificador/2022/04/27,
9,No hay registros de que Christine Lagarde dijera que â€œla democracia ha llegado a sus lÃ-mitesâ€,https://larepublica.pe//verifi
10,"No, estudiante peruano de medicina no forma parte de proyecto para restaurar la visiÃ³n",https://larepublica.pe//verificador/
11,"No, imÃ¡genes de accidente aÃ©reo no muestran un reciente evento en Italia, como indica viral",https://larepublica.pe//verifi
12,"No, la cantante Karol G no falleciÃ³ ni sufriÃ³ accidente de aviÃ³n",https://larepublica.pe//verificador/2022/04/24/no-la-canta
13,"Es falso que imÃ¡genes de accidente aÃ©reo sean recientes, como indica viral",https://larepublica.pe//verificador/2022/04/2
14,ÃšLTIMO MINUTO! 15 JUGADORES DE TENIS NO PUEDEN TERMINAR EL DE MIAMI EE-UU POR CULPA DE LA VACUNA,https://l
15,Deportistas abandonaron torneo de tenis por sÃ-ntomas de la vacuna ,https://larepublica.pe//verificador/2022/04/23/es-enga

Figura 79. Archivo csv noticias de Verificador LR

Fuente: Elaboración propia

3.- Ojo Público (Factcheking)

En esta última web también se desarrolló el scraping el 2 de mayo. El link a utilizar sería el de la sección "Factchecking". Parecido al caso anterior, se mostraba una ventana emergente cuando se ingresaba a la web (ANEXO 8), por lo que fue necesario cerrarla dando click en el botón "Aceptar y cerrar", para ello, se encontró que el botón pertenecía a la etiqueta *button* con la clase *"decline-button eu-cookie-compliance-default-button"*; sin embargo, una vez cerrada esta ventana (ANEXO 9), aparecía otra, de igual manera, se identificó que para cerrarla, se tenía que clickear el ícono de 'X', este se encontraba en la etiqueta *i* con clase *"fas fa-times"*.

Cerradas ambas ventanas ya se estaba listo para navegar en la web y extraer las noticias. Primero, se definió hasta qué página se realizaría el scraping, como cada página contenía 5 noticias, entonces, se identificó que con las primeras 20 se llegaría hasta las noticias publicadas a fines de diciembre e inicio de enero del 2022. Como en esta web sí existían números de páginas, el desplazamiento de página a página fue más sencillo, esta vez se decidió optar por el botón que avanzaba a la siguiente página cada vez que se clickeaba, este se encontraba en la etiqueta *i* con la clase *"fas fa-chevron-right"* (Figura 80), ambos valores se pasaron como parámetros de la función "*find_element_by_css_selector"* del *driver*.



Figura 80. Página siguiente con la clase "fas fa-chevron-right"
Fuente: Elaboración propia

Se extrajo el html de cada página con la función *page_source* y se colocó un tiempo de *sleep* igual a 3, a fin de no encontrarse con errores en cada extracción. Luego de obtener todos los htmls, se parseó el resultado usando la librería *BeautifulSoup* y se identificó que cada noticia estaba en la etiqueta *article* con la clase *"more-news-*

article search-article" (Figura 81), por lo que se procedió a extraer todas las noticias con la función *find_all*.



Figura 81. Noticia dentro de la etiqueta "article"

Fuente: Elaboración propia

Luego, se buscaron los atributos del dataset en esta lista: título, fecha y link. El título y el link nuevamente se encontraron junto dentro de la etiqueta *a* (Figura 82), siendo el atributo *text* para el título y *href* para el link; por otro lado, la fecha se encontró dentro de la etiqueta *span* (Figura 83).



Figura 82. Título y link de la noticia de Ojo Público dentro de la etiqueta "a"

Fuente: Elaboración propia



Figura 83. Fecha de la noticia de Ojo Público dentro de la etiqueta "span"

Fuente: Elaboración propia

En total se consiguieron 100 noticias, las cuales se almacenaron en un archivo csv para su posterior tratamiento.

Entregable:

5. Archivo csv de noticias de Ojo Público

,TÃ-tulo de la noticia,Fecha,Link
0,Las vacunas contra la covid-19 causan enfermedad a la piel.,28 / 04 / 2022,https://ojo-publico.com/3458/es-falso-que-vacu
1,Luz Pacheco Zerga sostuvo ante la comisiÃ³n especial del Congreso que en Argentina una mujer se casÃ³ con una pared.,20 /
2,"Ama Llulla revisÃ³ el tema, consultÃ³ bibliografÃ-a especializada, conversÃ³ con lingÃ¼istas y preparÃ³ este explicador para
3,Congresista de Avanza PaÃ-s asegurÃ³ que el actual Congreso es el mÃ¡s fragmentado de los Ãºltimos 20 aÃ±os por tener 10
4,Por WhatsApp circula esta grabaciÃ³n que pertenece a otro paÃ-s y que no se condice con el mecanismo de asistencia del C
5,"En redes sociales han estado circulando fotos de las protestas en Lima y otras ciudades. Algunas, como esta donde se ve a
6,Circulan imÃ¡genes en redes sociales con un mensaje acerca de que bus de transporte pÃºblico fue incendiado cerca a un ce
7,"Eduardo Ortega, jefe de equipo del Despacho Ministerial del Minsa dijo que la vacunaciÃ³n con tercera dosis supera el 70 %
8,La abogada y docente de la Universidad de Piura asegurÃ³ que en Argentina se aceptÃ³ el matrimonio entre una mujer y una
9,"Esto es parte parte de los esfuerzos por promover la capacitaciÃ³n en verificaciÃ³n que son impulsados desde OjoLab, OjoE
10,Un mensaje viral de redes sociales asegura que un francotirador voluntario que apoya a Ucrania falleciÃ³ tras ataque de la
11,La parlamentaria de PerÃº Libre asegurÃ³ que los actuales tribunos fueron elegidos por un Congreso con mayorÃ-a fujimor
12,Congresista Chirinos dijo que el presidente Castillo no debiÃ³ usar banda presidencial durante su intervenciÃ³n por la mociÃ
13,Ama Llulla revisÃ³ la evoluciÃ³n de la normativa para designar a altos funcionarios y explica a continuaciÃ³n los cambios qu
14,"El parlamentario Alejandro Aguinaga dijo que en su gestiÃ³n en el Minsa se erradicÃ³ la polio. Sin embargo, esto no es del
15,La crisis en PetroperÃº ocasionÃ³ que se considere como una empresa de 'bonos basura'. Â¿QuÃ© quiere decir esto y cÃ³r

Figura 84. Archivo de noticias de Ojo Público

Fuente: Elaboración propia

Actividad 3: Homogeneizar los dataset

Con todos los datos recopilados de la web, era necesario establecerlos en un mismo formato a fin de obtener un dataset general. Lo primero que se hizo fue identificar cuáles serían las columnas más importantes con las que se trabajaría, en base a los antecedentes, se notó que para la construcción del modelo de clasificación utilizan

principalmente el contenido y la clase, asimismo, la fecha se podría usar para tener una referencia del momento en que se llevó a cabo la publicación y, finalmente, el link de la publicación también sería importante para una posterior extracción del contenido que sirva como corpus de entrenamiento de los embeddings, así que estos fueron los atributos seleccionados.

Con respecto al dataset de posts, se notó que existían demasiados atributos (Figura 85), eran 53 en total; sin embargo, las únicas importantes para el dataset fueron *'text'*, *'time'* y *'link'*, por lo que solo se quedó con esas.

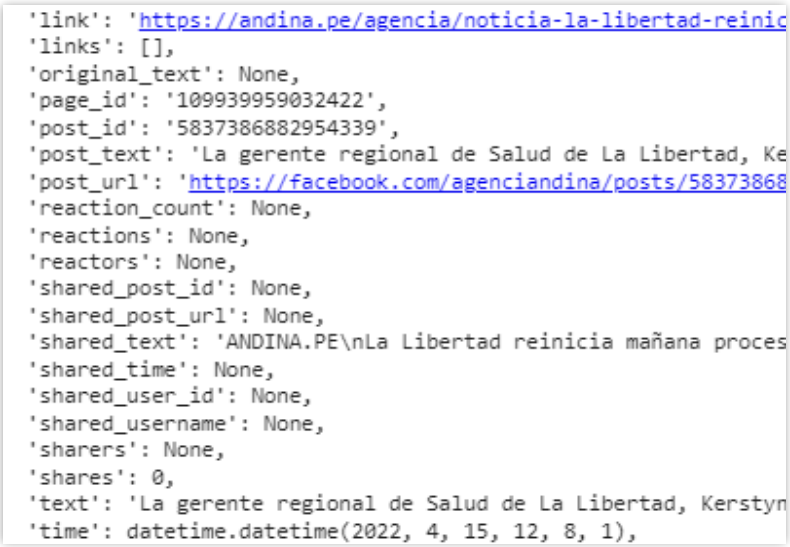


Figura 85. Ejemplo de atributos de un post

Fuente: Elaboración propia

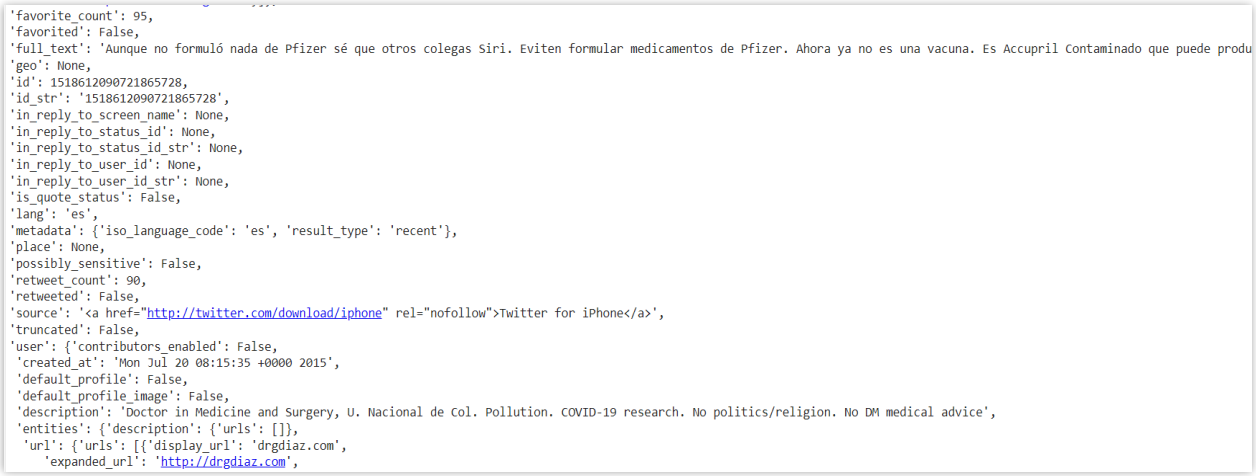
De igual manera, los comentarios tenían 12 atributos (Figura 86), de los cuales solo fue necesario quedarse con *'comment_text'*, *'comment_time'* y *'comment_url'*.



Figura 86. Atributos de un comentario

Fuente: Elaboración propia

Por otro lado, los tweets tenían 30 atributos (Figura 87), pero los importantes fueron *'full_text', 'created_at'* y *'expanded_url'*. Solo se seleccionaron estos atributos y se descartaron los demás.



'favorite_count': 95,
'favorited': False,
'full_text': 'Aunque no formuló nada de Pfizer sé que otros colegas Siri. Eviten formular medicamentos de Pfizer. Ahora ya no es una vacuna. Es Accupril Contaminado que puede produ
'geo': None,
'id': 1518612090721865728,
'id_str': '1518612090721865728',
'in_reply_to_screen_name': None,
'in_reply_to_status_id': None,
'in_reply_to_status_id_str': None,
'in_reply_to_user_id': None,
'in_reply_to_user_id_str': None,
'is_quote_status': False,
'lang': 'es',
'metadata': {'iso_language_code': 'es', 'result_type': 'recent'},
'place': None,
'possibly_sensitive': False,
'retweet_count': 90,
'retweeted': False,
'source': 'Twitter for iPhone',
'truncated': False,
'user': {'contributors_enabled': False,
'created_at': 'Mon Jul 20 08:15:35 +0000 2015',
'default_profile': False,
'default_profile_image': False,
'description': 'Doctor in Medicine and Surgery, U. Nacional de Col. Pollution. COVID-19 research. No politics/religion. No DM medical advice',
'entities': {'description': {'urls': []},
'url': {'urls': [{'display_url': 'drgdiaz.com',
'expanded_url': 'http://drgdiaz.com',

Figura 87. Ejemplo de atributos de un tweet

Fuente: Elaboración propia

En lo que resta a los dataset de las páginas web, con estos no hubo inconveniente puesto que al momento del scraping se decidió qué atributos se recopilarían, por lo que todas ya tenía la columna de '*Título de la noticia*', *'Fecha'* y *'Link'*, a excepción del dataset de verificador que no contaba con la fecha.

Lo que siguió a continuación fue el cambio de nombre de las columnas, en este paso se establecieron los nombres finales de cada columna del dataset: *'Contenido'*, *'Fecha'* y *'Link'*. Las columnas *'text', 'comment_text', 'full_text'* y *'Título de la noticia*' se renombraron como *'Contenido'*. Mientras que *'time', 'comment_time'* y *'created_at'* se renombraron como *'Fecha'*. Finalmente, las columnas *'link', 'comment_url'* y *'expanded_url'* pasaron a ser *'Link'*.

Con todas las columnas en un mismo formato se procedió a la unión en un dataset general mediante la función *concat* de la librería Pandas, obteniendo un dataset de 1 171 551 registros. Posteriormente, se eliminaron los registros que no trataban del coronavirus, para ello se construyó una función *filter_news* cuyo parámetro fue el contenido del

registro. La función se encargó de comparar cada palabra definida como *keyword* (Figura 88) con cada uno de los registros, si el registro tenía alguna de estas *keywords*, se retornaba el mismo contenido, caso contrario, se retornaba un valor nulo.

```
keywords = ['covid','coronavirus','pandemia','vacuna','sars','ómicron','deltacron','molecular','pcr','oms'
           ,'presencial','mascarilla','camas uci','variante','dosis','contagio','contagiado','cuarentena']
```

Figura 88. Lista de keywords

Fuente: Elaboración propia

Seguidamente, se eliminaron todos los valores nulos y/o duplicados, y se añadió la columna *'Clase'* con los valores de *'Ninguna'*, se inicializaron estos valores porque sería la etiqueta final en caso algún registro no sea ni *'Verdadero'* ni *'Falso'*. Resultado de ello fue un dataset de 80 515 registros, el cual se guardó en un archivo Excel para facilitar el etiquetado.

Entregable:

- Dataset relacionado al COVID-19.

	Clase	Contenido	Fecha	Link
0	Ninguna	Minsa: Conoce cómo obtener y actualizar el carné de vacunación COVID-19D	4 de diciembre	https://www.gob.pe/institucion/minsa/noticias/
1	Ninguna	Conozca los centros que estarán abiertos al público por la Semana de la vacu	22 de abril de	https://www.gob.pe/institucion/minsa/noticias/
2	Ninguna	Minsa inicia la aplicación de la cuarta dosis contra la COVID-19 en Lima Metr	2 de abril de 2	https://www.gob.pe/institucion/minsa/noticias/
3	Ninguna	Menores de 12 a 17 años ya pueden acudir por su tercera dosis contra la COV	25 de marzo d	https://www.gob.pe/institucion/minsa/noticias/
4	Ninguna	Minsa distribuirá más de 350 000 vacunas contra la influenza para la població	2 de mayo de	https://www.gob.pe/institucion/minsa/noticias/
5	Ninguna	Minsa: Casos confirmados por coronavirus COVID-19 ascienden a 3 565 839 e	1 de mayo de	https://www.gob.pe/institucion/minsa/noticias/
6	Ninguna	Minsa continúa inmunizando contra la COVID-19 a la población que necesita	1 de mayo de	https://www.gob.pe/institucion/minsa/noticias/
7	Ninguna	Ica: niñas y niños de toda la región participaron del gran "vacuna kid"	1 de mayo de	https://www.gob.pe/institucion/minsa/noticias/
8	Ninguna	Minsa: Casos confirmados por coronavirus COVID-19 ascienden a 3 565 253 e	30 de abril de	https://www.gob.pe/institucion/minsa/noticias/
9	Ninguna	Minsa: El país se encuentra abastecido de vacunas para cubrir demanda de la	30 de abril de	https://www.gob.pe/institucion/minsa/noticias/
10	Ninguna	Minsa: Solo en cinco regiones el uso de mascarilla será opcional en espacios	30 de abril de	https://www.gob.pe/institucion/minsa/noticias/
11	Ninguna	Puno: Diresa establece como meta aplicar totalidad de dosis pediátricas cont	30 de abril de	https://www.gob.pe/institucion/minsa/noticias/
12	Ninguna	1 de mayo: uso facultativo de mascarilla será en regiones cuya población may	30 de abril de	https://www.gob.pe/institucion/minsa/noticias/
13	Ninguna	Minsa: Casos confirmados por coronavirus COVID-19 ascienden a 3 564 558	29 de abril de	https://www.gob.pe/institucion/minsa/noticias/
14	Ninguna	Perú se convierte en el tercer país de Sudamérica en liderar la vacunación co	29 de abril de	https://www.gob.pe/institucion/minsa/noticias/
15	Ninguna	Arequipa: Minsa impulsó la vacunación pediátrica contra la COVID-19 en la I.I	29 de abril de	https://www.gob.pe/institucion/minsa/noticias/
16	Ninguna	Semana de la vacunación en las Américas: Se aplican 18 tipos de vacunas par	29 de abril de	https://www.gob.pe/institucion/minsa/noticias/

Figura 89. Dataset homogeneizado

Fuente: Elaboración propia

Actividad 4: Etiquetar manualmente los registros

El etiquetado se realizó en base al mismo procedimiento llevado a cabo en antecedente de Hayawi et. al (2022), es decir, primero se realizó el etiquetado manual y

luego se validó dicho etiquetado con expertos en el tema. En esta actividad se llevó a cabo solo el etiquetado manual, la validación se desarrolló en la siguiente actividad.

Para realizar el etiquetado manual se identificaron las fuentes de información fiables, estas fueron el MINSA (ente máximo de salud a nivel nacional), la OPS (organización de salud a nivel americano) y la OMS (ente máximo de salud a nivel mundial); y los factchekers VerificadorLR y OjoPúblico (ambos con apoyo de agencias internacionales de investigación).

Luego, se procedió a contrastar la información del registro con cada fuente, para lo cual, se requirió la ayuda de un familiar que trabaja en el rubro de la salud, esto con el fin de tener una guía al momento de llevar a cabo este procedimiento. El proceso tomó alrededor de 3 meses, trabajando en un promedio de 2 horas al día, y constó en lo siguiente:

Por cada registro a comparar, se cuestionó a priori si este trataba con información acerca del COVID o solo era un registro en el que se mencionaba este tema de manera sarcástica o con tendencias religiosas (Figura 90), si este era el caso, automáticamente se descartaba, manteniéndose la etiqueta original de '*Ninguna*'.

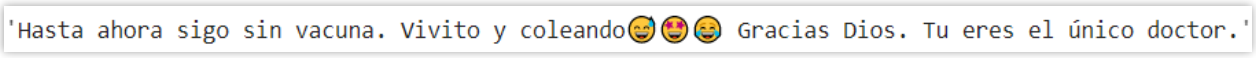


Figura 90. Registro con tendencias religiosas

Fuente: Elaboración propia

Por otro lado, si este no era el caso, se buscaba si en algunos de los factchekers ya se había publicado alguna noticia similar, si ya se encontraba publicada, el registro se etiquetaba con la misma denominación dada por el checker ("*Falso*" o "*Verdadero*"); sin embargo, si la denominación no era ninguna de estas (Figura 91), se dejaba con la etiqueta "*Ninguna*".



Figura 91. Noticia registrada como "engañoso"

Fuente: Elaboración propia

Asimismo, si la información no se encontraba publicada en alguno de estos sitios web, se buscaba en las publicaciones del MINSA (Twitter, Facebook y web). En este punto fue necesario utilizar los campos de fechas para tener una referencia del día en el que se publicó la noticia, de manera que fuese más fácil buscar esa información en los medios del MINSA. Se siguió una idea similar de etiquetado, es decir, si el registro contradecía alguna de estas publicaciones, se colocaba como "*Falso*", si era acorde a las mismas, sería "*Verdadero*", y, si no se encontraba en alguna de las publicaciones, se procedía a buscarlo en las webs de la OMS y la OPS, finalmente, si tampoco se encontraba información similar en estos sitios, se descartaba el registro ("*Ninguna*").

Antes de pasar al proceso de validación, se dio una revisión final con el objetivo de descubrir si existía algún error ortográfico en el etiquetado. El resultado de esta etapa fueron 2022 registros etiquetados (1011 por clase), los cuales se guardaron en un nuevo archivo Excel.

Entregable:

- Dataset etiquetado.

Clase	Contenido	Fecha	Link
Verdadero	Minsa acerca pruebas moleculares y vacunación contra la COVID-19 a la p	7 de enero de	https://www.gob.pe/institucion/minsa/
Verdadero	Actualmente existen 122 laboratorios facultados por el INS para el diagn	6 de enero de	https://www.gob.pe/institucion/minsa/
Verdadero	Minsa habilita Puntos de Descarte de COVID-19 (Comunicado N°804)	6 de enero de	https://www.gob.pe/institucion/minsa/
Verdadero	Minsa refuerza detección de COVID-19 en Surquillo ante el incremento de	5 de enero de	https://www.gob.pe/institucion/minsa/
Verdadero	Minsa cuenta con stock de más de 7.8 millones de pruebas antígenas par	5 de enero de	https://www.gob.pe/institucion/minsa/
Verdadero	Minsa incrementa centros de descarte para la COVID-19	5 de enero de	https://www.gob.pe/institucion/minsa/
Verdadero	Minsa promueve medidas de prevención contra la COVID-19 en Plaza Sar	3 de enero de	https://www.gob.pe/institucion/minsa/
Falso	Las vacunas contra la covid-19 causan enfermedad a la piel.	28 / 04 / 2022	https://ojo-publico.com/3458/es-falso
Falso	LAS "VACUNAS COVID-19" PROVOCAN "SIDA" Y HACEN QUE LAS FUNCIO	14 / 03 / 2022	https://ojo-publico.com/3378/es-falso
Falso	Las funciones del sistema inmune de los vacunados contra la covid-19 dis	16 / 03 / 2022	https://ojo-publico.com/3378/es-falso
Falso	Un estudio científico indica que la vacuna de ARNm de Pfizer altera el AD	10 / 03 / 2022	https://ojo-publico.com/3365/es-falso
Falso	Un video muestra que un niño falleció a causa de la vacuna contra la covi	08 / 01 / 2022	https://ojo-publico.com/3255/es-falso
Falso	Los CDC han admitido que las pruebas PCR ya no sirven para detectar la c	31 / 12 / 2021	https://ojo-publico.com/3253/es-falso
Falso	Los Centros de Control de Enfermedades (CDC) finalmente están retirand	31 / 12 / 2021	https://ojo-publico.com/3253/es-falso

Figura 92. Dataset etiquetado

Fuente: Elaboración propia

Actividad 5: Validar etiquetado con personal del MINSA

Siguiendo los mismos pasos del antecedente mencionado en la etapa anterior, lo que continuó fue la validación del etiquetado manual con algún profesional conocedor del tema. Debido a que los registros tienen información acerca del coronavirus, fue pertinente contactarse con algún personal del organismo de salud con mayor rango en el país: el Ministerio de Salud. La profesional que brindó apoyo en esta etapa fue la Srta. Margarita Arias Vargas, quien ocupó el cargo de Coordinadora de Comunicaciones del MINSA, y participa en la difusión de información hacia el pueblo peruano a través de los distintos medios como las redes sociales o la misma página web. La declaración de validación (Figura 93) fue firmada por ella una vez terminada la revisión el 30 de julio.

El dataset final con el que se llevó a cabo el modelamiento constó de 2022 registros etiquetados y validados.

Entregable:

- Dataset validado.



DECLARACIÓN DE VALIDACIÓN DEL ETIQUETADO DEL DATASET

Yo **Margarita Arias Vargas** con el cargo de **Coordinadora de Comunicaciones en el Ministerio de Salud (MINSA)**, declaro haber validado el correcto etiquetado de los registros de COVID-19 del estudiante **Yair Andrey Salinas Bolaños** para el trabajo de Tesis 2 titulado "Propuesta de modelo para la detección de información falsa del COVID-19 mediante técnicas de Machine Learning y Natural Language Processing".

Lima, 30 de junio del 2022

Lic. M. Arias.

Firma

DNI: 10225624

Figura 93. Documento de validación del etiquetado

Fuente: Elaboración propia

4.1.2 Preprocesamiento

Actividad 1: Eliminar los links, hashtags, menciones y emojis.

Con el dataset de registros validados, se procedió a realizar la limpieza de cada registro a fin de una mejor vectorización de características. En esta primera actividad, se creó la expresión regular para eliminar los links, hashtags y menciones, para lo cual se utilizó la librería *re* (regular expresions o regex).

Se pudo identificar que el inicio de cualquier link suele ser con los caracteres "https//", de igual manera, tanto los hashtags como las menciones suelen iniciar con los caracteres de "#" y "@", respectivamente. Entonces, la expresión regular consideró estos casos para la eliminación de los mismos mediante la función *sub*, la cual permite sustituir cualquier expresión regular por algún otro valor, en este caso, ese valor fue un vacío, lo

que indicaba que la expresión se había eliminado. La figura 94 muestra la expresión creada para la identificación de los 3 casos.

```
'http[s]?://\S+|#\S*|@\S*'
```

Figura 94. Expresión regular para eliminar links, hashtags y menciones

Fuente: Elaboración propia

La primera de ellas (http[s]?://\S+) hace referencia a que se encontrará cualquier link que inicie necesariamente con los caracteres 'http', sin ser el carácter 's' determinante, es decir, después de esta cadena de caracteres puede darse el caso que siga la letra 's' como también no, esto con la finalidad de abarcar links con protocolo seguro (https) y links sin este protocolo (http). Seguidamente, es necesario que el link contenga los caracteres ':' y '//', ya que estos son quienes separan el protocolo del resto de sus partes (host, dominio, puerto, etc.), finalmente, la última parte ('\S'), indica que se buscará cualquier carácter que no sea un espacio, esto se repetirá muchas veces ('+') hasta encontrar el espacio, lo cual indica el final de la url.

La segunda parte (#\S*) se encargó de identificar expresiones que empiecen con el carácter '#', seguido de caracteres no espaciados, los cuales se repetirán varias veces hasta encontrar el espacio. Finalmente, la última parte (@\S*) es bastante similar a la anterior solo que esta buscó identificar menciones de twitter las cuales se hacen mediante el carácter '@'. Estas 3 expresiones se unieron mediante la condicional *or* (|).

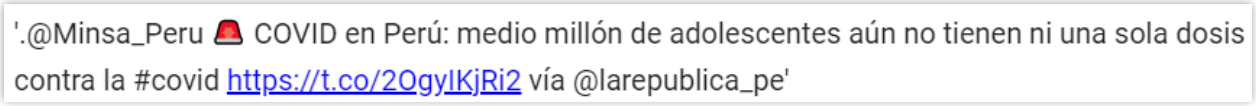


Figura 95. Ejemplo de registro original del dataset

Fuente: Elaboración propia

A continuación, se muestra el ejemplo de un registro (Figura 96) procesado con estas expresiones:

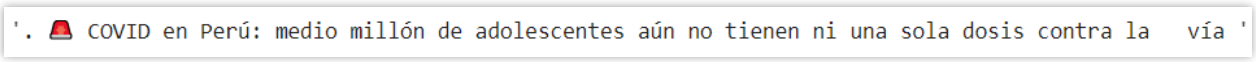


Figura 96. Ejemplo de registro sin links, hashtags ni menciones

Fuente: Elaboración propia

Seguidamente, se realizó la eliminación de emojis. Para ello se utilizó la función *replace_emoji* de la librería *emoji* (librería especializada para el tratamiento de emojis), de manera similar que *sub*, esta función permite reemplazar los emojis presentes en un registro por cualquier otro carácter, en este caso, se decidió que no haya carácter de reemplazo, lo que suponía su eliminación. La figura 97 muestra el resultado final de esta actividad.

```
'.  COVID en Perú: medio millón de adolescentes aún no tienen ni una sola dosis contra la   vía '
```

Figura 97. Ejemplo de registro sin links, hashtags, menciones ni emojis

Fuente: Elaboración propia

Entregable:

- Registros sin links, hashtags, menciones ni emojis.

🔴📲 Infórmate en #TiempoDeSalud:\n\n✅ Vacuna mo...	Infórmate en Vacuna moderna cuenta con...
Yesica Tirado Becerra La dentista contagiada p...	Yesica Tirado Becerra La dentista contagiada p...
¡El barrido de la vacunación continúa llegando...	¡El barrido de la vacunación continúa llegando...
Esas vacunas no sirven, no protegen en nada. N...	Esas vacunas no sirven, no protegen en nada. N...
Mario Vargas Llosa, aislado en su casa de Madr...	Mario Vargas Llosa, aislado en su casa de Madr...
🚨🚀 Minsa aplicó el doble de la cantidad establ...	Minsa aplicó el doble de la cantidad establ...

Figura 98. Registros sin links, hashtags, menciones ni emojis

Fuente: Elaboración propia

Actividad 2: Limpiar el contenido

Se realizó una limpieza más detallada del contenido en esta etapa. Primero, se llevó a cabo la normalización del registro, esto supuso llevar los registros a un mismo formato, para ello se utilizó la librería *unicode* mediante su función *normalize* con el tipo de normalización *'NFKD'* y la codificación *'ascii',* ignorando la eliminación de caracteres fuera de este formato, es decir, aquellas palabras que contenían caracteres

propios del español como ‘ñ’, ‘ü’ o las tildes, convirtieron su valor (letra) en el formato ASCII de caracteres imprimibles (ANEXO 12), de manera que se quedó con la palabra completa. La figura 99 muestra el resultado de esta primera parte.

```
'.  COVID en Peru: medio millon de adolescentes aun no tienen ni una sola dosis contra la   via '
```

Figura 99. Registro en formato ASCII

Fuente: Elaboración propia

Luego, para la siguiente normalización se pasó a minúsculas todos los caracteres en mayúscula, esto mediante el método *lower*, propio de los strings de Python.

```
'.  covid en peru: medio millon de adolescentes aun no tienen ni una sola dosis contra la   via '
```

Figura 100. Registro normalizado

Fuente: Elaboración propia

Una vez normalizado el contenido, se llevó a cabo la tokenización. La librería utilizada fue *nltk* (Natural Language Toolkit), la cual cuenta con la función *wordpunct_tokenize* que permitió obtener una lista de tokens de las palabras y de los signos de puntuación del contenido.

```
['.',
 'covid',
 'en',
 'peru',
 ':',
 'medio',
 'millon',
 'de',
 'adolescentes',
 'aun',
 'no',
 'tienen',
 'ni',
 'una',
 'sola',
 'dosis',
 'contra',
 'la',
 'via']
```

Figura 101. Tokens del registro

Fuente: Elaboración propia

Con los tokens listos, se realizó la eliminación de los signos de puntuación y números, para ello, solo se consideraron los tokens alfabéticos con el método *isalpha* (propio de los strings), también se decidió mantener solo aquellas palabras con longitud mayor a 2 puesto que fue el límite evidenciado en el que las palabras importantes aparecían.

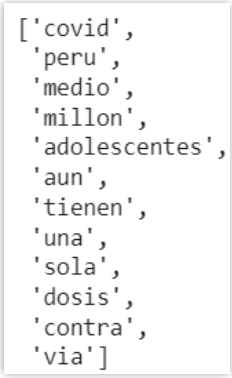


Figura 102. Registro sin signos de puntuación ni números

Fuente: Elaboración propia

Una vez que el registro contenía solo palabras, se procedió con la eliminación de *stop words*. Para llevar a cabo esta tarea se trabajaron con 3 fuentes de stop words. La primera proveniente del paquete *corpus* de la librería *nltk*, este paquete brindó una lista de 313 stop words del español; la segunda, de la página web COUNTWORDSFREE (ANEXO 11) que provee stop words de distintos idiomas y en 3 formatos (json, xml, txt), de esta se obtuvieron 443 palabras vacías en español: y, para la tercera, se agregaron modismos propios del país como ‘pe’, ‘ps’, ‘pues’, etc. Se unieron las 3 fuentes y se eliminaron las palabras repetidas, obteniendo un total de 626 palabras vacías. La figura 103 muestra algunas stop words.

```
['estaremos',
 'porque',
 'yo',
 'esta',
 'tuvieran',
 'has',
 'les',
 'tenías',
 'al',
 'tras',
 'primera',
 'hubo',
 'consideró',
 'tuvieseis',
 'eres',
 'consigo',
 'vez',
 'quien',
```

Figura 103. Ejemplos de stop words

Fuente: Elaboración propia

Se comparó cada palabra del contenido con la lista final de stop words, si existía alguna stop word presente, esta se eliminaba. Cabe mencionar que también se normalizó la lista pasando al formato ASCII para que la comparación entre las palabras no se vea afectada por las tildes. En la siguiente figura se puede observar el registro sin stop words:

```
['covid', 'peru', 'medio', 'millon', 'adolescentes', 'dosis', 'via']
```

Figura 104. Registro sin stop words

Fuente: Elaboración propia

Finalmente, se creó el corpus para la extracción en la etapa siguiente. En este caso, fueron necesarios 2 corpus, el primero en formato string para las técnicas de BoW y Tf-Idf (Contenido_Preprocesado), mientras que el segundo en forma de lista de palabras para los modelos de embedding w2c y fastText (Contenido_Preprocesado_Embedding).

Entregable:

- Corpus de palabras preprocesadas.

Contenido	Contenido_Preprocesado	Contenido_Preprocesado_Embedding
Hospital Villa El Salvador refuerza vacunación...	hospital villa salvador refuerza vacunacion di...	[hospital, villa, salvador, refuerza, vacunaci...
Hay q ser muy ignorante después de toda la inf...	ignorante informacion ponerse dosis	[ignorante, informacion, ponerse, dosis]
Toxicidad del grafeno: El arma perfecta para e...	toxicidad grafeno arma perfecta enfermar matar...	[toxicidad, grafeno, arma, perfecta, enfermar,...
Desde el Cusco, durante el VI Consejo de Minis...	cusco consejo ministros descentralizado minist...	[cusco, consejo, ministros, descentralizado, m...
La vacunación contra el #COVID19 en estudiante...	vacunacion estudiantes obligatoria retornar cl...	[vacunacion, estudiantes, obligatoria, retorna...

Figura 105. Corpus de palabras preprocesadas

Fuente: Elaboración propia

4.1.3 Extracción de características

Actividad 1: Realizar la extracción de características con BoW

Se importó la técnica *CountVectorizer* de la librería *sklearn* (scikit-learn) y se crearon 3 vectorizadores. La siguiente tabla muestra los vectorizadores con el único parámetro utilizado.

Tabla 12.
Parámetros de CountVectorizer

		Parámetro
		max_features
Vectorizadores	cv_1	100
	cv_2	200
	cv_3	300

Fuente: Elaboración propia

Luego se vectorizó cada contenido del corpus aplicando *fit_transform* y se obtuvo la matriz de características por cada uno: 2022x100, 2022x200 y 2022x300. A continuación se muestra el vector de características para el registro de la figura 104 pero en formato string (“covid peru medio millon adolescentes dosis via”).

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Figura 106. Vector de características con BoW

Fuente: Elaboración propia

Entregable:

- Tres vectorizadores con la técnica BoW

```
cv_1 = CountVectorizer(max_features=100)
cv_2 = CountVectorizer(max_features=200)
cv_3 = CountVectorizer(max_features=300)
```

Figura 107. Vectorizadores con BoW

Fuente: Elaboración propia

Actividad 2: Realizar la extracción de características con Tf-Idf

Se importó la técnica *TfidfVectorizer* de la librería *sklearn* (scikit-learn) y se crearon 3 vectorizadores. La siguiente tabla muestra los vectorizadores con sus parámetros.

Tabla 13.
Parámetros de TfidfVectorizer

		Parámetros	
		max_features	ngram_range
Vectorizadores	tf_idf_1	100	1,2
	tf_idf_2	200	1,2
	tf_idf_3	300	1,2

Fuente: Elaboración propia

De igual manera que en la técnica anterior, se vectorizaron todos los registros del corpus, obteniendo las siguientes dimensiones de matriz de características: 2022x100, 2022x200 y 2022x300. A continuación, se muestra el ejemplo de vector de características para el registro de la figura 104, de igual manera que el caso anterior, el registro está como string.

```
array([0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.2876747 , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.39935456, 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.71175751, 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.5011591 , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ,
       0.        , 0.        , 0.        , 0.        , 0.        ])
```

Figura 108. Vector de características con Tf-Ifd

Fuente: Elaboración propia

Entregable:

- Tres vectorizadores con la técnica Tf-Idf

```
tf_idf_1 = TfidfVectorizer(max_features=100,ngram_range=(1,2))
tf_idf_2 = TfidfVectorizer(max_features=200,ngram_range=(1,2))
tf_idf_3 = TfidfVectorizer(max_features=300,ngram_range=(1,2))
```

Figura 109. Vectorizadores con Tf-Idf

Fuente: Elaboración propia

Actividad 3: Realizar la extracción de características con Word2Vect

El corpus para el entrenamiento de este modelo se obtuvo del dataset general; sin embargo, no se tomaron todos los registros (80k) puesto que gran parte de los mismos eran comentarios, mas no publicaciones, es por eso que solo se tomaron los posts de Facebook, las noticias de las páginas web y las publicaciones en Twitter, ya que estas, al estar bien redactadas, brindan un buen contexto para el modelo, lo cual no era posible incluyendo los comentarios.

Principalmente se trabajó con los links de cada publicación, de los cuales se extrajo el contenido de toda la notica usando la librería *newspaper3k* con su módulo *Article*, el cual cuenta el método *download* que permite descargar diversas partes de un artículo mediante su url (autores, fecha, contenido, links de imágenes/videos, etc.), esta forma de scraping también demoró puesto que se colocó un *request_timeout* de 3 segundos, ello con la finalidad de que no

ocurran problemas con el cargado de la página. Sin embargo, no fue posible la extracción del contenido de todos los registros, en estos casos solo se retornaban valores vacíos.

Asimismo, algunos links redireccionaban a páginas en otros idiomas, lo cual no favoreció para el corpus, es por eso que se utilizó la librería *langdetect* (language-detection) para detectar si el contenido estaba en español o no. Se implementó una función que retornaba el mismo contenido en caso esté escrito en español, y un valor nulo en caso contrario.

Debido a que el corpus contenía registros con valores nulos, fue necesario la eliminación de los mismos, ya que estos no servirían para el entrenamiento, de igual manera, se eliminaron valores duplicados. El resultado final fue un dataset de 11 631 filas y una sola columna, la cual era el contenido.

Este dataset pasó por el mismo preprocesamiento anteriormente explicado, de manera que el resultado del mismo se expresó en una lista de palabras, donde cada palabra representó el token de entrenamiento.

Para llevar a cabo el entrenamiento se utilizó la librería *Gensim* con el modelo *Word2Vec*. Este entrenamiento se realizó en 100 épocas pues se evidenció que la similaridad de las palabras era mejor con este número. Además, los valores probados variaron desde el 1 hasta el 5 para los parámetros *size* y *min_count*, mientras que *sg* se probó con 0 (cbow) y 1 (skip-gram). La siguiente tabla muestra los mejores modelos entrenados:

Tabla 14.
Parámetros de Word2Vec

		Parámetros			
		Size	Window	min_count	sg
Vectorizadores	w2v_1	100	5	5	0
	w2v_2	200	5	5	0
	w2v_3	300	1,2	5	1

Fuente: Elaboración propia

El primer y segundo vectorizador contaron con 19 572 palabras en el vocabulario, mientras que el tercero, 22 502. En la figura 110 se observan las similaridades más altas con la palabra ‘coronavirus’ del mejor modelo (w2v_1), las cuales resultaron no ser tan altas debido al corpus pequeño con el que se entrenó.

```
w2v_1.wv.most_similar('coronavirus', topn=5)
```

```
[('oxigeno', 0.5150542855262756),
 ('eliminada', 0.4252941608428955),
 ('evolucionando', 0.38515961170196533),
 ('segmento', 0.3460485637187958),
 ('terapeuticas', 0.3449375629425049)]
```

Figura 110. Similaridades del modelo w2v con la palabra “coronavirus”

Fuente: Elaboración propia

Estos modelos se guardaron con la función *save* propia del mismo modelo. Posteriormente se cargaron los modelos con la función *load* para obtener los vectores de características para cada registro. Puesto que cada palabra contó con su propio vector, fue necesario promediar todos los vectores de las palabras que aparecían dentro de un registro, el resultado de ello fue el vector final de dicho registro.

Así pues, el vector para el registro de la Figura 104, sería el promedio de los vectores de las palabras ‘covid’, ‘peru’, ‘medio’, ‘millon’, ‘adolescentes’, ‘dosis’ y ‘via’, obteniendo el siguiente resultado:

```
array([ 2.42498441e-04,  1.51077786e-03,  2.30308739e-03,  6.63569153e-05,
       -1.59767995e-04,  1.08520011e-03, -1.63188635e-03, -3.58906551e-03,
        4.06246632e-03,  1.26200460e-03,  1.49032066e-03,  9.95074748e-04,
       -2.55153773e-05,  2.06247042e-03, -4.99453908e-03, -3.03405849e-03,
       -1.65947049e-03,  1.07478921e-03,  3.66736902e-03, -5.07986464e-04,
       -2.30743340e-03,  6.68632623e-04, -4.67902573e-04, -2.52283039e-03,
        3.71841947e-03,  6.31094910e-04,  6.88319502e-04,  3.46340152e-04,
       -2.29753111e-03, -6.04520028e-04,  1.42836780e-03,  2.24628649e-03,
        1.41118164e-03, -2.07901164e-03,  2.50790222e-03, -5.02358482e-04,
       -6.97117823e-04,  1.30315148e-03, -4.34943882e-04,  5.96537313e-04,
       -1.48575101e-03,  3.58216534e-03,  2.79169157e-03, -2.38945839e-04,
       -2.73160753e-03, -3.34457989e-04, -3.00086639e-03,  8.70280375e-04,
        3.57851922e-03,  5.73117239e-03,  1.69295690e-03,  2.07347539e-03,
       -2.47603445e-03,  7.57098664e-04, -4.08855791e-04, -1.67688041e-03,
       -6.34633063e-04, -5.38801833e-04, -6.96748102e-05,  2.36425735e-03,
        9.69330315e-04,  3.02675853e-05, -1.36443484e-03, -2.14611297e-03,
       -2.93112127e-03, -2.87571177e-03, -3.97233386e-03,  6.13199314e-03,
        5.12195146e-03, -1.78781059e-03, -2.70070857e-03,  1.21276209e-03,
        3.22001106e-05,  8.54998012e-04, -2.06695753e-03,  2.49279605e-04,
       -7.60366311e-05, -1.20286446e-03, -9.31199524e-04,  5.68114594e-03,
       -2.73767486e-03,  3.16548510e-03, -8.02353083e-04,  1.57787814e-03,
        1.48300093e-03,  1.80767372e-03,  4.67663631e-04,  2.89342628e-04,
        2.25539552e-03,  4.16699657e-03, -1.12410740e-03, -2.04463420e-03,
       -1.05999492e-03, -3.13897617e-04, -5.45738847e-04, -5.90405776e-04,
        1.20168384e-04,  2.63182237e-03, -3.24365543e-03,  5.09910635e-04],
      dtype=float32)
```

Figura 111. Vector de características con w2v

Fuente: Elaboración propia

Finalmente, la matriz de características estuvo data por cada uno de estos vectores pero en todos los registros del corpus, es decir, se obtuvieron 3 matrices: 2022x100, 2022x200 y 2022x300.

Entregable:

- Tres modelos Word2Vec entrenados



Figura 112. Vectorizadores con w2v

Fuente: Elaboración propia

Actividad 4: Realizar la extracción de características con FastText

El entrenamiento de este modelo se realizó con el mismo corpus de w2v (11 631) y de manera similar que el anterior, solo que se utilizó el modelo *FastText* de la librería *Gensim.* Adicionalmente, se consideraron 2 parámetros más propios de este modelo: la longitud mínima y máxima del n-grama de cada palabra.

Este entrenamiento se realizó en 100 épocas, ya que se conseguía un gran porcentaje de similaridad con este número de épocas. La siguiente tabla muestra los mejores modelos entrenados:

Tabla 15.
Parámetros de FastText

		Parámetros					
		Size	Window	min_count	Sg	min_n	max_n
Vectorizadores	fastText_1	100	4	5	0	2	5
	fastText_2	200	4	3	0	2	4
	fastText_3	300	4	4	0	2	6

Fuente: Elaboración propia

El primer vectorizador contó con 19 586 palabras en el vocabulario, mientras que el segundo, 26 583, y el tercero, 22 526. En la figura 113 se observan las similaridades más altas con la palabra ‘coronavirus’ del mejor modelo (fastText_1), en este caso, se pudo observar mayor similaridad que con el modelo w2v, ya que fastText se basa principalmente en relaciones sintácticas.

```
fastText_1.wv.most_similar('coronavirus',topn=5)

[('betacoronavirus', 0.8173174262046814),
 ('rotavirus', 0.6305937767028809),
 ('corona', 0.5824894309043884),
 ('coronaria', 0.5522018671035767),
 ('covirus', 0.5242584347724915)]
```

Figura 113. Similaridades del modelo fastText con la palabra “coronavirus”

Fuente: Elaboración propia

Se guardaron los modelos (*save*) para su posterior uso (*load*). Después se procedió a obtener los vectores de características para cada registro, puesto que cada palabra está compuesta por distintos n-gramas, el vector de dicha palabra sería el resultado del promedio de estos n-gramas; sin embargo, no fue necesario realizar el cálculo ya que el modelo lo hizo internamente y entregó el vector final de cada palabra, lo que sí se calculó fue el promedio de los vectores de las palabras que aparecían dentro de un registro para obtener el vector característico de ese registro. Siguiendo el ejemplo de la Figura 104, su vector característico fue el siguiente:

```
array([ 5.31840604e-04,  3.77363584e-04,  1.09690147e-04, -3.50399408e-04,
        3.56984732e-04,  6.35770557e-04, -6.23146072e-04,  6.93982001e-04,
       -4.49783402e-04, -9.12336924e-04,  1.05546205e-04,  9.47631488e-04,
       -1.39161435e-04,  1.43787795e-04, -1.16677920e-05, -7.24326019e-05,
       -2.58869783e-04, -1.52532040e-04,  4.43491066e-04, -1.78753726e-05,
        4.52763139e-04, -1.74555229e-04,  1.24728671e-04, -9.14570701e-04,
       -2.55949999e-04, -7.87723460e-04, -1.02714272e-04, -6.25023735e-04,
        9.95584880e-04,  4.43155965e-04,  6.52737916e-04,  2.90410917e-05,
       -2.03029515e-04, -5.08932688e-04,  1.14024129e-04,  3.29172326e-04,
       -3.97433090e-04, -2.54138547e-04, -1.57079383e-04, -1.15104311e-04,
       -5.68139774e-04, -3.34363984e-04,  7.01431942e-04,  7.16701616e-05,
       -3.56855198e-05,  1.17918127e-03, -3.49753769e-04, -1.14680035e-04,
        3.41204199e-04,  1.83828801e-04, -7.35354784e-04,  1.59963762e-04,
        1.03760154e-04,  3.84308019e-04,  3.37918464e-04,  1.29041262e-04,
       -2.07262510e-05,  3.72710580e-04,  8.60682339e-05,  2.61889742e-04,
        5.13452280e-04, -3.72396258e-04,  3.22880922e-04,  4.99296293e-04,
       -8.41662404e-04, -1.80284158e-04,  1.38064017e-04, -1.15100481e-03,
        7.05321960e-04,  1.12689049e-04, -5.05961289e-05,  5.32407837e-04,
        4.70202212e-04, -6.01230829e-04,  5.44420254e-05, -7.90735241e-04,
        8.34710605e-04, -9.29753587e-04, -5.20522881e-05,  1.52436772e-03,
        8.50535929e-04,  1.20779514e-04, -7.91566999e-05,  1.40417877e-04,
       -2.25206037e-04,  3.73864226e-04,  2.62544869e-04, -1.72954256e-04,
       -1.44204576e-04, -7.62167387e-04,  5.55334627e-05,  4.97058172e-05,
       -8.94352415e-05,  5.62404930e-05, -3.02999601e-04, -3.24965949e-04,
       -6.02887070e-04,  6.73700299e-04, -1.05971016e-03,  5.54584491e-04],
      dtype=float32)
```

Figura 114. Vector de características con fastText

Fuente: Elaboración propia

De igual manera que en los casos anteriores, se obtuvieron 3 matrices de características para el corpus: 2022x100, 2022x200 y 2022x300.

Entregable:

- Tres modelos FastText entrenados

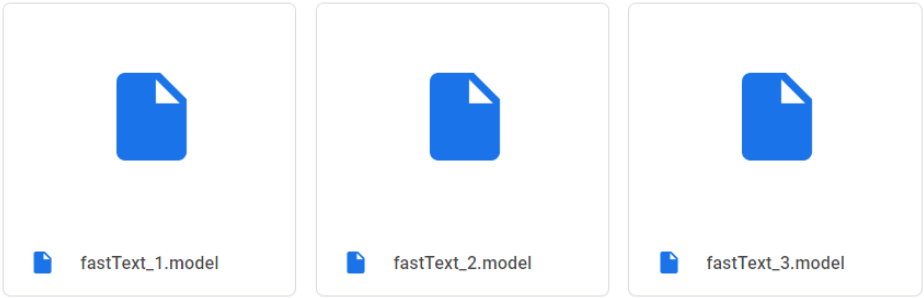


Figura 115. Vectorizadores con fastText

Fuente: Elaboración propia

4.1.4 Construcción del modelo

Actividad 1: Codificar la clase

Lo que se hizo en esta actividad fue colocar el target numérico a cada uno de los vectores característicos, ya que, mediante ellos, se pudo evaluar el desempeño del modelo. En total se trabajó con 1011 datos para cada clase: Verdadero y Falso.

Para llevar a cabo esto, fue necesario utilizar un *LabelEncoder* de la librería *sklearn*, el cual permitió codificar etiquetas del target categórico en valores numéricos. La codificación resultante fue 0 para los registros falsos y 1 para los verdaderos. Este nuevo label se unió con los vectores numéricos, formando el dataset final previo a los algoritmos de Machine Learning.

Entregable:

- Dataset numérico

cv_1	tf_idf_1	w2v_1	fastText_1	Clase
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.334...	[-0.00027666941362743575, -0.00020650010264944...	[-0.0008982636500149965, 0.0002436189242871478...	1
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...	[0.0002662890765350312, -4.742498276755214e-05...	[-0.00042443323764018714, -0.00013718445552513...	0
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.241...	[0.00114224920980632 3, -0.0015124749625101686,...	[-0.00019906948728021234, -0.00036443574936129...	0
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...	[-0.0009945885006648798, 0.0004905016653032766...	[-0.0001844444195739925, -8.00703201093711e-05...	1
[0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 1, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.8013355925890...	[-0.0004556013154797256, 0.001098821172490716,...	[-0.0002862054097931832, -5.2819545089732856e-...	1

Figura 116. Dataset numérico con las técnicas *cv*_1, tf_*idf*_1, w2v_1 y fastText_*1*

Fuente: Elaboración propia

Actividad 2: Particionar el dataset

Antes de realizar la partición de la data, se hizo un *suffle* de la misma con el fin de evitar sesgos. Se utilizó la función *train_test_split* del paquete *model_selection* de la librería *sklearn* y se le pasó como parámetros la data independiente (X) y la data dependiente (Y). Se definió un porcentaje de 20% para la data de test (*test_size*) y el porcentaje restante sería considerado data de train. El resultado de ello fueron 4 particiones: 1617x12 para la data *X_train*, 1617x1 para *Y_train*, 405x12 para *X_test* y 405x1 para *Y_test*.

Entregable:

- Dataset particionado en 80% y 20%

2001	[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, ...	[0.0, 0.0, 0.0, 0.671796116513577, 0.0, 0.0, 0...
965	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...
742	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, ...
1114	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.36407034...
2021	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...

1617 rows × 13 columns

Figura 117. Porción de la data train (*X_train*, *Y_train*)

Fuente: Elaboración propia

Actividad 3: Aplicar técnicas de Machine Learning

Una vez conseguida la data de train y test, se procedió a entrenar los modelos de *Machine Learning.* Para cada modelo se creó una grilla (*grid*), la cual se pasó como parámetro al algoritmo *GridSearchCV* con una validación cruzada (*cv*) de 5. Se entrenaron los modelos con la data de train perteneciente tanto a los vectorizadores (*X_train*) como al target (*Y_train*), considerando el tipo de vectorizador para cada modelo. Se utilizó el método *fit* para llevar a cabo este entrenamiento. Por otro lado, luego de obtener los mejores modelos por técnica de vectorización, se procedió a evaluar el desempeño de cada uno con el método *predict* para la data de test (*X_test*), los resultados de estos se compararon con el target de test (*Y_test*) y con ello se obtuvieron las métricas de accuracy, precision, recall y F1-score.

Entregable:

- Resultados de métricas de cada modelo

a) KNN

La figura 118 muestra los parámetros probados en la grilla de este algoritmo con los cuales se entrenaron un total de 84 modelos por cada vectorizador. Se utilizó la técnica *KNeighborsClassifier* del paquete *neighbors* de *sklearn.*

```
grid = {'n_neighbors' : list(range(3,30,2)),
        'weights' : ['uniform','distance'],
        'metric'  : ['minkowski','euclidean','manhattan']}
```

Figura 118. Grilla de KNN

Fuente: Elaboración propia

La tabla 16 muestra los mejores parámetros obtenidos por cada modelo con su respectiva técnica de vectorización:

Tabla 16.
Mejores vectorizadores con KNN

		knn_grid		
		n_neighbors	Weights	metric
Vectorizador	cv_1	7	*Distance*	*minkowski*
	cv_2	3	*Distance*	*minkowski*
	cv_3	3	*Distance*	*minkowski*
	tf_idf_1	5	*Distance*	*minkowski*
	tf_idf_2	11	*Distance*	*minkowski*
	tf_idf_3	25	*Distance*	*minkowski*
	w2v_1	3	*Distance*	*minkowski*
	w2v_2	3	*Distance*	*minkowski*
	w2v_3	3	*Distance*	*manhattan*
	fastText_1	3	*Distance*	*manhattan*
	fastText_2	5	*Distance*	*minkowski*
	fastText_3	29	*Distance*	*Minkowski*

Fuente: Elaboración propia

La tabla 17 muestra los resultados de las métricas obtenidos por cada modelo entrenado:

Tabla 17.
Resultados del modelo KNN

		Métricas			
		Accuracy	Precision	Recall	F1-score
KNN	cv_1	0.80	0.77	0.88	0.83
	cv_2	0.80	0.76	0.90	0.82
	cv_3	0.78	0.74	0.91	0.81
	tf_idf_1	0.83	0.83	0.87	0.85

	tf_idf_2	0.83	0.82	0.87	0.84
	tf_idf_3	0.86	0.85	0.90	0.87
	w2v_1	0.69	0.70	0.73	0.71
	w2v_2	0.74	0.72	0.81	0.76
	w2v_3	0.69	0.67	0.79	0.73
	fastText_1	0.71	0.78	0.63	0.70
	fastText_2	0.72	0.75	0.70	0.72
	fastText_3	0.70	0.80	0.58	0.67

Fuente: Elaboración propia

b) Árbol de decisión

La figura 119 muestra los parámetros probados en la grilla de este algoritmo con los cuales se entrenaron un total de 6720 modelos por cada vectorizador. Se utilizó la técnica *DecisionTreeClassifier* del paquete *tree* de *sklearn.*

```
grid = {'criterion':['gini','entropy'],
        'max_depth': np.arange(3, 10),
        'min_samples_leaf': np.arange(5, 20),
        'min_samples_split': np.arange(2, 10),
        'max_features': ['auto', 'sqrt', 'log2', None]}
```

Figura 119. Grilla de Árbol de Decisión

Fuente: Elaboración propia

La tabla 18 muestra los mejores parámetros obtenidos por cada modelo con su respectiva técnica de vectorización:

Tabla 18.
Mejores vectorizadores con Árbol de Decisión

		dt_grid				
		criterion	max_depth	min_samples_leaf	min_samples_split	max_features
Vecto	cv_1	*gini*	9	10	2	*None*
	cv_2	*gini*	9	10	2	*None*

	cv_3	*gini*	9	10	2	*None*
	tf_idf_1	*entropy*	9	16	2	*None*
	tf_idf_2	*gini*	8	6	2	*None*
	tf_idf_3	*gini*	9	11	2	*None*
	w2v_1	*gini*	8	8	2	*None*
	w2v_2	*entropy*	9	13	2	*Auto*
	w2v_3	*gini*	5	6	2	*None*
	fastText_1	*entropy*	9	17	2	*None*
	fastText_2	*gini*	8	19	2	*Auto*
	fastText_3	*entropy*	8	9	2	*None*

Fuente: Elaboración propia

La tabla 19 muestra los resultados de las métricas obtenidos por cada modelo entrenado:

Tabla 19.
Resultados del modelo Árbol de Decisión

		Métricas			
		Accuracy	Precision	Recall	F1-score
Decision Tree	cv_1	0.76	0.74	0.85	0.79
	cv_2	0.76	0.74	0.85	0.79
	cv_3	0.76	0.74	0.85	0.79
	tf_idf_1	0.76	0.75	0.83	0.78
	tf_idf_2	0.76	0.74	0.84	0.79
	tf_idf_3	0.77	0.76	0.81	0.79
	w2v_1	0.60	0.63	0.60	0.62
	w2v_2	0.63	0.67	0.59	0.63
	w2v_3	0.66	0.74	0.56	0.64
	fastText_1	0.69	0.70	0.71	0.71
	fastText_2	0.70	0.71	0.73	0.72
	fastText_3	0.62	0.63	0.70	0.66

Fuente: Elaboración propia

c) Naive Bayes

La figura 120 muestra los parámetros probados en la grilla de este algoritmo con los cuales se entrenaron un total de 6 modelos por cada vectorizador. Se utilizó la técnica *MultinomialNB* del paquete *naive_bayes* de *sklearn.*

```
grid = {'alpha': (1, 0.1, 0.01, 0.001, 0.0001, 0.00001)}
```

Figura 120. Grilla de Naive Bayes

Fuente: Elaboración propia

La tabla 20 muestra los mejores parámetros obtenidos por cada modelo con su respectiva técnica de vectorización:

Tabla 20.
Mejores vectorizadores con Naive Bayes

		nb_grid
		Alpha
Vectorizador	cv_1	0.1
	cv_2	0.1
	cv_3	0.01
	tf_idf_1	0.00001
	tf_idf_2	0.1
	tf_idf_3	0.1
	w2v_1	0.1
	w2v_2	1
	w2v_3	0.1
	fastText_1	1
	fastText_2	1
	fastText_3	0.1

Fuente: Elaboración propia

La tabla 21 muestra los resultados de las métricas obtenidos por cada modelo entrenado:

Tabla 21.
Resultados del modelo Naive Bayes

		Métricas			
		Accuracy	Precision	Recall	F1-score
Naive Bayes	cv_1	0.82	0.86	0.80	0.83
	cv_2	0.84	0.87	0.82	0.84
	cv_3	0.85	0.88	0.83	0.85
	tf_idf_1	0.81	0.82	0.83	0.82
	tf_idf_2	0.83	0.84	0.84	0.84
	tf_idf_3	0.86	0.87	0.87	0.87
	w2v_1	0.68	0.85	0.49	0.62
	w2v_2	0.68	0.88	0.47	0.61
	w2v_3	0.66	0.88	0.43	0.57
	fastText_1	0.73	0.80	0.65	0.72
	fastText_2	0.72	0.81	0.61	0.70
	fastText_3	0.70	0.81	0.58	0.67

Fuente: Elaboración propia

d) SVM

La figura 121 muestra los parámetros probados en la grilla de este algoritmo con los cuales se entrenaron un total de 100 modelos por cada vectorizador. Se utilizó la técnica *SVC* del paquete *svm* de *sklearn.*

```
grid = {'C': [0.1, 0.5, 1.0, 10.0, 100.0],
        'gamma': [1, 0.1, 0.01, 0.001, 0.0001],
        'kernel': ['rbf', 'linear','poly','sigmoid']}
```

Figura 121. Grilla de SVM

Fuente: Elaboración propia

La tabla 22 muestra los mejores parámetros obtenidos por cada modelo con su respectiva técnica de vectorización:

Tabla 22.
Mejores vectorizadores con SVM

		svm_grid		
		C	gamma	Kernel
Vectorizador	cv_1	1	0.1	*Rbf*
	cv_2	0.5	1	*Linear*
	cv_3	0.5	1	*Linear*
	tf_idf_1	1	1	*Rbf*
	tf_idf_2	0.5	1	*Rbf*
	tf_idf_3	0.5	1	*Rbf*
	w2v_1	1	1	*Rbf*
	w2v_2	10	0.1	*Rbf*
	w2v_3	10	0.1	*Rbf*
	fastText_1	1	1	*Rbf*
	fastText_2	10	0.1	*Rbf*
	fastText_3	10	0.1	*Rbf*

Fuente: Elaboración propia

La tabla 23 muestra los resultados de las métricas obtenidos por cada modelo entrenado:

Tabla 23.
Resultados del modelo SVM

		Métricas			
		Accuracy	Precision	Recall	F1-score
SVM	cv_1	0.84	0.83	0.88	0.85
	cv_2	0.82	0.80	0.86	0.83
	cv_3	0.85	0.85	0.86	0.86
	tf_idf_1	0.86	0.88	0.84	0.86
	tf_idf_2	0.86	0.88	0.86	0.87

	tf_idf_3	0.87	0.88	0.87	0.88
	w2v_1	0.73	0.68	0.91	0.78
	w2v_2	0.80	0.83	0.78	0.80
	w2v_3	0.79	0.91	0.68	0.78
	fastText_1	0.62	0.58	1	0.74
	fastText_2	0.84	0.82	0.89	0.85
	fastText_3	0.84	0.92	0.77	0.84

Fuente: Elaboración propia

e) Regresión Logística

La figura 122 muestra los parámetros probados en la grilla de este algoritmo con los cuales se entrenaron un total de 48 modelos por cada vectorizador. Se utilizó la técnica *LogisticRegression* del paquete *linear_model* de *sklearn.*

```
grid = {'solver': ['newton-cg', 'lbfgs', 'sag', 'saga'],
        'penalty':['none', 'l2'],
        'C':[0.001, 0.01, 0.1, 1, 10, 100]}
```

Figura 122. Grilla de Regresión Logística

Fuente: Elaboración propia

La tabla 24 muestra los mejores parámetros obtenidos por cada modelo con su respectiva técnica de vectorización:

Tabla 24.
Mejores vectorizadores con Regresión Logística

		lr_grid		
		C	penalty	Solver
Vectorizador	cv_1	100	*l2*	*saga*
	cv_2	100	*l2*	*sag*
	cv_3	10	*l2*	*sag*
	tf_idf_1	10	*l2*	*newton-cg*

	tf_idf_2	1	*l2*	*newton-cg*
	tf_idf_3	10	*l2*	*newton-cg*
	w2v_1	0.001	*none*	*newton-cg*
	w2v_2	0.001	*none*	*lbfgs*
	w2v_3	0.001	*none*	*lbfgs*
	fastText_1	0.001	*none*	*lbfgs*
	fastText_2	0.001	*none*	*lbfgs*
	fastText_3	0.001	*none*	*lbfgs*

Fuente: Elaboración propia

La tabla 25 muestra los resultados de las métricas obtenidos por cada modelo entrenado:

Tabla 25.
Resultados del modelo Regresión Logística

		Métricas			
		Accuracy	Precision	Recall	F1-score
Logistic Regression	cv_1	0.84	0.84	0.86	0.85
	cv_2	0.83	0.83	0.85	0.84
	cv_3	0.86	0.86	0.88	0.86
	tf_idf_1	0.85	0.87	0.84	0.86
	tf_idf_2	0.85	0.86	0.85	0.85
	tf_idf_3	0.86	0.88	0.87	0.87
	w2v_1	0.69	0.72	0.67	0.69
	w2v_2	0.74	0.77	0.72	0.74
	w2v_3	0.77	0.81	0.74	0.77
	fastText_1	0.76	0.78	0.74	0.76
	fastText_2	0.82	0.85	0.80	0.82
	fastText_3	0.80	0.83	0.78	0.80

Fuente: Elaboración propia

f) Multilayer Perceptron

El diseño de la arquitectura de la red fue un poco complicado, puesto que en los antecedentes no especificaban claramente la arquitectura utilizada, se tuvieron que realizar varios experimentos hasta encontrar algunas aceptables, las cuales se utilizaron para el entrenamiento con los distintos vectorizadores. Se utilizó *MLPClassifier* del paquete *neural_network* de *sklearn.* En total, se entrenaron 48 modelos por cada vectorizador con 100 épocas.

```
grid = {"hidden_layer_sizes" : [(10,), (50,), (100,), (10, 10), (50, 50), (100, 10), (200, 20), (300,30)],
        "activation": ['logistic', 'tanh', 'relu'],
        "solver": ['sgd', 'adam']}

model_mlp = MLPClassifier(max_iter=100,random_state=2022)
```

Figura 123. Grilla de MLP

Fuente: Elaboración propia

La tabla 26 muestra los resultados de las métricas obtenidos por cada modelo entrenado:

Tabla 26.
Mejores arquitecturas del MLP

	Vectorizer	Layers/Neurons	Activation	Epochs	Optimizer	Accuracy
MLP	cv_1	3: 10, 10, 1	*sigmoid, sigmoid, softmax*	100	*adam*	0.84
	cv_2	2: 100, 1	*sigmoid, softmax*	100	*adam*	0.82
	cv_3	2: 10, 1	*relu, softmax*	100	*adam*	0.85
	tf_idf_1	3: 50, 50, 1	*sigmoid, sigmoid, softmax*	100	*adam*	0.84
	tf_idf_2	3: 10, 10, 1	*relu, relu, softmax*	100	*adam*	0.83
	tf_idf_3	2: 100, 1	*tanh, sigmoid, softmax*	100	*adam*	0.84
	w2v_1	3: 100, 100, 1	*relu, relu, softmax*	100	*adam*	0.69

	w2v_2	3: 50, 50, 1	*relu, relu, softmax*	100	*adam*	0.74
	w2v_3	3: 50, 50, 1	*tanh, tanh, softmax*	100	*adam*	0.76
	fastText_1	3: 300, 30, 1	*tanh, tanh, softmax*	100	*adam*	0.74
	fastText_2	3: 300, 30, 1	*tanh, tanh, softmax*	100	*adam*	0.79
	fastText_3	3: 300, 30, 1	*tanh, tanh, softmax*	100	*adam*	0.74

Fuente: Elaboración propia

4.1.5 Evaluación del modelo

En la última fase de la metodología se evaluaron todos los modelos construidos para conocer aquel con mayor desempeño. En primer lugar, debido a que se probaron 12 vectorizadores distintos por cada modelo, se tuvo que identificar la mejor combinación entre el modelo de Machine Learning y la técnica de vectorización de NLP. Esta primera comparación se hizo tomando en cuenta la métrica de *accuracy*. Los resultados se dieron sobre una data de test compuesta por 405 registros (214 falsos y 191 verdaderos).

a) KNN:

El mejor modelo con esta técnica se obtuvo con el vectorizador *tf_idf_3,* el cual tiene como parámetros: n_neighbors (25), weights (*distance*) y metric (*minkowski*). Se consideró la mejor combinación puesto que el *accuracy* con valor 86,17% fue superior al resto. Además, según se observa en la matriz de confusión (Figura 124), el modelo logra diferenciar los registros falsos de los verdaderos, es decir, brinda mayor contraste a los datos clasificados correctamente.

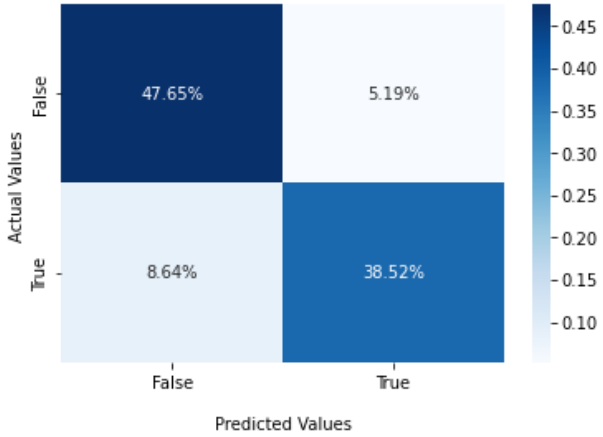


Figura 124. Matriz de confusión del mejor modelo de KNN

Fuente: Elaboración propia

b) Árbol de decisión:

El mejor modelo con esta técnica se obtuvo con el vectorizador *tf_idf_3* con los parámetros de criterion (*gini*), max_depth (9), max_features (*None*), min_samples_leaf (11) y min_samples_split (2). Se consideró la mejor combinación puesto que el *accuracy* con valor 76,53% fue superior al resto. Además, según se observa en la matriz de confusión (Figura 125), el modelo brinda mayor contraste a los datos clasificados correctamente.

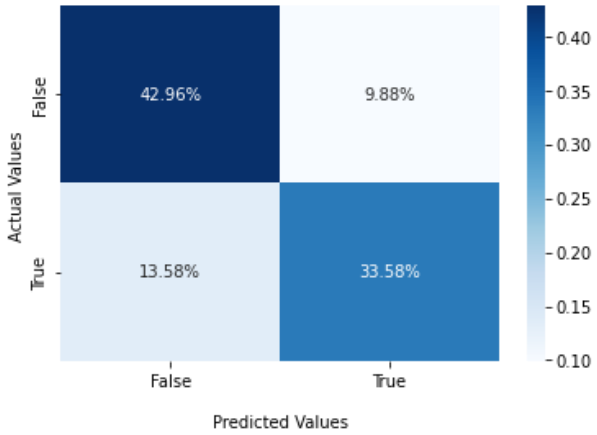


Figura 125. Matriz de confusión del mejor modelo de Árbol de Decisión

Fuente: Elaboración propia

c) Naive Bayes:

El mejor modelo con esta técnica se obtuvo con el vectorizador *tf_idf_3* con el único parámetro de alpha (0.1). Se consideró la mejor combinación puesto que el *accuracy* con valor 86,17% fue superior al resto. Además, según se observa en la matriz de confusión (Figura 126), el modelo brinda mayor contraste a los datos clasificados correctamente.

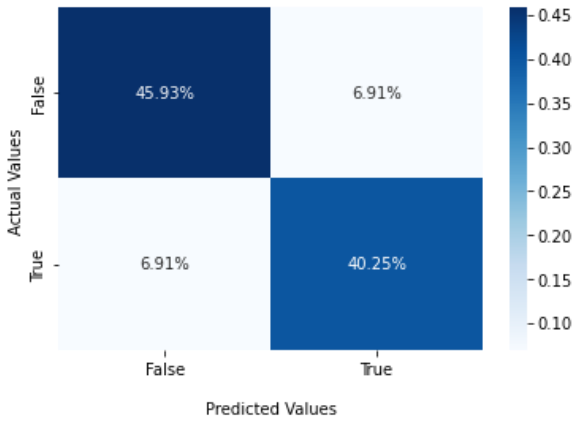


Figura 126. Matriz de confusión del mejor modelo de Naive Bayes

Fuente: Elaboración propia

d) SVM:

El mejor modelo con esta técnica se obtuvo con el vectorizador *tf_idf_3* con los parámetros: C (0.5), gamma (1) y kernel (*rbf*). Se consideró la mejor combinación puesto que el *accuracy* con valor 87,41% fue superior al resto. Además, según se observa en la matriz de confusión (Figura 127), el modelo brinda mayor contraste a los datos clasificados correctamente.

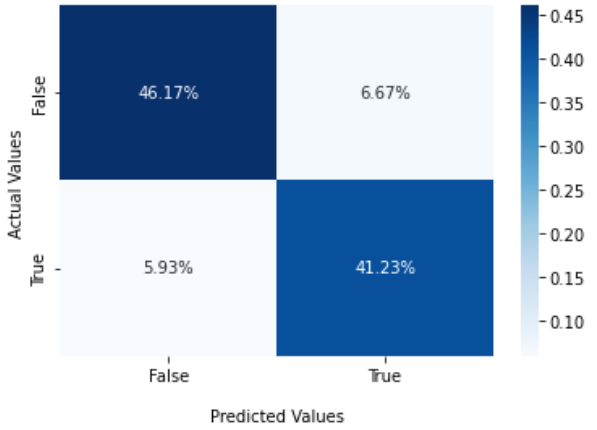


Figura 127. Matriz de confusión del mejor modelo de SVM

Fuente: Elaboración propia

e) Regresión Logística:

El mejor modelo con esta técnica se obtuvo con el vectorizador *tf_idf_3* con los parámetros: C (10), penalty (*l2*) y solver (*newton-cg*). Se consideró la mejor combinación ya que el *accuracy* con valor 86,67% fue superior al resto, a excepción del *cv_3*, pero el *f1-score* sí lo fue con respecto a este. Además, según se observa en la matriz de confusión (Figura 128), el modelo brinda mayor contraste a los datos clasificados correctamente.

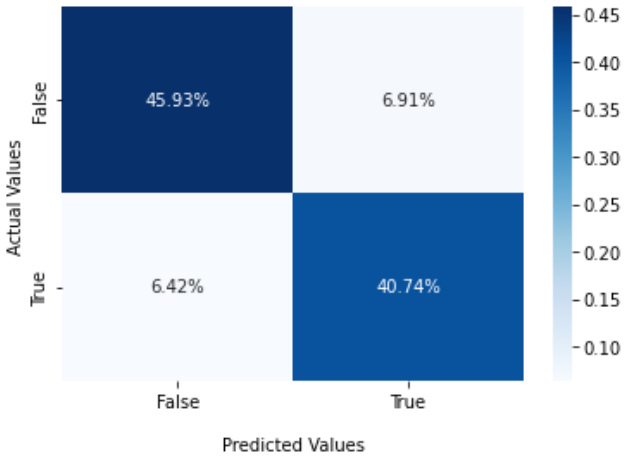


Figura 128. Matriz de confusión del mejor modelo de Regresión Logística

Fuente: Elaboración propia

f) Multilayer Perceptron:

El mejor modelo con esta técnica se obtuvo con el vectorizador *cv_3* con una arquitectura compuesta por 2 capas: una capa oculta con 10 neuronas y una capa de salida con 1 neurona, utilizando la función de activación *relu* y *softmax*. Se consideró la mejor combinación ya que el *accuracy* con valor 85,19% fue superior al resto. Además, según se observa en la matriz de confusión (Figura 129), el modelo brinda mayor contraste a los datos clasificados correctamente.

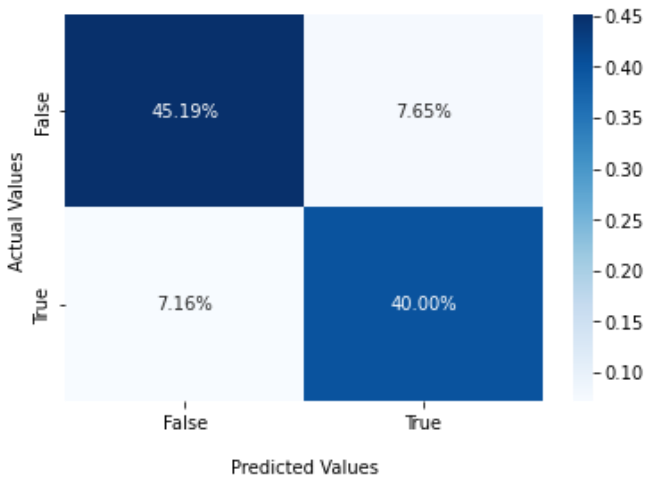


Figura 129. Matriz de confusión del mejor modelo de MLP

Fuente: Elaboración propia

4.2 Medición de la solución

La siguiente tabla muestra la comparación entre los mejores modelos, y los resultados de cada uno. Se puede observar que el modelo superior a los demás fue SVM con *tf_idf_3*.

Tabla 27.
Comparación de las mejores métricas de cada modelo

Modelo	Parámetros	Accuracy	Precision	Recall	F1-score

KNN (*tf_idf_3)*	n_neighbors: 25 weights: *distance* metric: *minkowski*	86.17%	84.65%	90.19%	87.33%
Árbol de decisión (*tf_idf_3)*	criterion: *gini*, max_depth: 9 max_features: *None* min_samples_leaf: 11 min_samples_split: 2	76.53%	75.98%	81.31%	78.56%
Naive Bayes (*tf_idf_3)*	alpha: 0.1	86.17%	86.92%	86.92%	86.92%
SVM (*tf_idf_3)*	C: 0.5 gamma: 1 kernel: *rbf*	87.41%	88.63%	87.38%	88.00%
Regresión Logística (*tf_idf_3)*	C: 10, penalty: *l2* solver: *newton-cg*	86.67%	87.74	86.92%	87.32%
MLP (*cv_3)*	1° capa: 10 neuronas (*relu*) 2° capa: 1 neurona (*softmax*)	85.19%	86.32%	85.51%	85.92%

Fuente: Elaboración propia

Si bien se observa que el *recall* de SVM es menor al de KNN, las demás métricas logran superarlas, es por ello que se considera que SVM es el mejor modelo de clasificación de información falsa del COVID-19. Esto se ve reflejado en su desempeño, ya que el modelo logra clasificar correctamente cada registro de covid como *Verdadero* o *Falso* con un 87.41% de certeza, asimismo, clasifica en menor medida los falsos positivos con una precision de 88.63% y, del total de registros de información etiquetados como *Verdadero*, se logra clasificar correctamente el 87.38% del total.

CAPÍTULO 5: CONCLUSIONES Y RECOMENDACIONES

5.1 Conclusiones

Hoy en día las plataformas de redes sociales son muy poderosas en la difusión de información relacionada al COVID-19 y la concientización adecuada a la comunidad sobre los peligros de la desinformación, Estos medios de difusión pueden propagar información incorrecta, antisocial y deficiente de manera rápida. Si bien existe la iniciativa por parte del MINSA de alentar al público peruano a que no comparta fake news, esta no es suficiente, ya que muchas personas lo toman en forma de broma y hacen caso omiso a las recomendaciones que brinda el ministerio. Asimismo, no todas las publicaciones y comentarios que contienen información falsa acerca del covid puede ser detectadas por el MINSA debido al gran número existente circulando por la web, es por ello que se llevó a cabo esta investigación con la finalidad de dar solución a esta problemática.

La inteligencia artificial aporta en gran medida para la solución de problemas actuales, ya que mediante el entrenamiento de modelos de clasificación con dataset de ciertos tópicos, estos pueden determinar algunos patrones presentes en cada registro que hace que el mismo pertenezca a una clase o a otra. Estos modelos se ven potenciados con la combinación de técnicas existentes tanto en Machine Learning como en Natural Language Processing. Para llevar a cabo la investigación, primeramente, fue necesario recopilar datos que trataban de temas relacionados al COVID-19, en total se extrajeron 80 515 registros de la web peruana mediante la técnica de web scraping; sin embargo, no fue posible trabajar con todos, por lo que se escogieron 2 022 de ellos para ser etiquetados manualmente y validados posteriormente por personal del MINSA. Luego, se aplicaron técnicas de Natural Language Processing para vectorizar la data, pues estas facilitan la conversión de datos texturales, como lo son los registros de covid, a datos numéricos mediante la extracción de características propias de dicho registro, si bien en este campo existen diversidad de técnicas, para la investigación solo se optaron por tomar cuatro de ellas, cada una con 3 variantes: BoW, TF-IDF, Word2Vec y FastText. Asimismo, con los datos numéricos listos, se probaron 6 técnicas de Machine Learning para construir el modelo de clasificación: KNN, Árbol de Decisión, Naive Bayes, SVM, Regresión Logística y MLP. Se realizaron muchos

experimentos con el fin de encontrar la mejor combinación de técnicas para el modelo de clasificación. El resultado final fue un modelo con 87,41% de Accuracy que logra distinguir ciertos patrones falsos presentes en los registros del dataset, por ende, se puede decir que sí se cumplió el objetivo general del estudio.

La creación del dataset fue uno de los procesos más complicados, puesto que no solo bastó con la recolección de datos de web, sino que también fue necesario un etiquetado manual y su validación posterior. En la investigación se utilizó la técnica de web scraping para extraer los datos de la web de manera automática, facilitando en gran medida la recopilación de los mismos, para esto se identificaron las fuentes que proporcionarían los datos, se escogieron 3 fuentes generales: Twitter, Facebook y páginas web, puesto que cada una de estas tiene gran variedad de sitios de los cuales se podían extraer los datos, se decidió definir los usuarios específicos de los cuales se tomarían sus publicaciones. Para la plataforma Twitter, 53 878 tweets fueron recopilados de los 24 departamentos del Perú y la provincia Constitucional del Callao; mientras que para Facebook, se recolectaron 21 510 publicaciones de los medios de noticias más conocidos en el país: Perú 21, La República, RPP, El Comercio, El Peruano, Andina, Tv Perú y el MINSA; junto con 1 282 100 comentarios. En lo que respecta a las páginas web, se tomaron 912 noticias de la página del MINSA, 133 y 100 de los factchekers Verificador LR y OjoPúblico, respectivamente. Después de realizar el filtrado y eliminar registros repetidos y no relacionados al COVID-19, se obtuvo un dataset general de 80 515 registros. Seguidamente, se realizó el etiquetado manual basándose en fuentes de información confiables como el MINSA, la OPS y la OMS, este proceso consistió en colocar un registro como *Falso* en caso el contenido sea distinto al de alguna de estas fuentes y *Verdadero* si fuese similar. Todo ello tomó alrededor de 3 meses; sin embargo, no se logró etiquetar mucha data, solo se consiguió el etiquetado de 2 022 registros (1 011 por cada clase), cabe mencionar que en este tiempo se identificaron fuentes no confiables a las cuales los ciudadanos suelen citar como medios de información “verídicos”, algunas de estas fueron TierraPura, La Quinta Columna, Médicos por la Verdad y diversidad de grupos de Telegram en contra de la vacunación. Posteriormente, se necesitó el apoyo de Margarita Arias Vargas, personal del MINSA perteneciente al área de Comunicaciones, para determinar la validez del etiquetado. El resultado final fueron 2 022 registros etiquetados y validados, por lo que se puede decir que se logró el objetivo específico 1.

Asimismo, en lo que respecta al objetivo específico 2, se concluye que el preprocesamiento realizado fue el correcto, ya que este se pudo ver reflejado en el resultado final del modelo. Esto se hizo en base al preprocesamiento mostrado en los antecedentes, y, debido a que todos están en el idioma inglés, se decidió no tomar algunas partes del mismo, como fue el caso de incluir algún tipo de stemmer o no, al inicio se optó por hacerlo, pero los resultados finales no fueron buenos. Sin embargo, esto no quiere decir que agregando nuevos pasos al preprocesamiento no se puedan mejorar los resultados, aún así, con el preprocesamiento expuesto en el trabajo fue suficiente para llevar a cabo un buen entrenamiento del modelo.

La diversidad de técnicas existentes en el campo del Natural Language Processing es una gran ventaja cuando se desea vectorizar registros textuales, ya que las mismas facilitan la conversión de data textual a data numérica. Para la investigación se decidieron tomar 4 técnicas: Bag of Word, Tf-Idf, Word2Vec y fastText, las primeras 3 en base a los antecedentes y la última implementada por decisión propia, cada una con 3 variantes referente a su tamaño (100, 200 y 300). El mejor vectorizador se obtuvo con la técnica Tf-Idf de tamaño 300 con n-gramas de 1 a 2, puesto que fue quien mejor representó las palabras claves del corpus, detectó la información relevante, dejando de lado aquella irrelevante. Con esta técnica no solo se consiguió el mejor modelo, sino también el segundo, tercer, cuarto y quinto mejores modelos. Es decir, la técnica funcionó mejor por encima de las demás, incluso de los modelos de embedding; sin embargo, cabe recalcar que, aunque los embedding no hayan conseguido grandes similaridades en su entrenamiento debido al corpus pequeño con el que se realizó el mismo (11 631), lograron brindar buenas vectorizaciones de los registros, esto quiere decir que si se entrenase con mayor cantidad de datos, el contexto que se le brindaría a las palabras sería más precisos, por lo que probablemente su vectorización sería superior a la Tf-Idf. Con todo lo mencionado, se puede concluir que se alcanzó el objetivo específico 3.

Los distintos experimentos realizados con los algoritmos de Machine Learning permitieron encontrar un modelo de clasificación robusto con Accuracy de 87,41%, este modelo utilizó la técnica SVM con los parámetros de C (0.5), gamma (1) y kernel (*rbf*). Llevar a cabo esto no resultó tan fácil como se creía inicialmente, ya que, para tunear los hiperparámetros no bastó solo

con colocar valores aleatorios, sino que también fue necesario fijarse que dichos valores tengan sentido y no afecten al modelo porque de lo contrario el entrenamiento no sería el correcto, pues se caería en *overfitting* o *underfitting*. Para disminuir el riesgo de estos problemas fue importante realizar validación cruzada en cada entrenamiento, es por ello que, en algunos casos, los modelos tardaban buen tiempo en ejecutarse, además, mientras más parámetros se agregaban o más valores se añadían por cada parámetro, este tiempo de training aumentaba aún más; sin embargo, debido a que la data era pequeña, el tiempo no superaba la hora de ejecución, aún así, es importante tener en cuenta que si se decide aumentar el tamaño del dataset, realizar todos estos experimentos puede llegar a ser agotador. Puesto que se logró encontrar un buen modelo de clasificación de información falsa del COVID-19, se puede decir que se logró el objetivo específico 4.

Finalmente, las métricas de Accuracy, Precision, Recall y F1-score fueron necesarias para determinar el desempeño de los distintos modelos entrenados, con ellas se pudo determinar la confiabilidad de cada modelo y poder conocer cierta certeza cuando se desea predecir si una noticia es falsa o verdadera. Si bien el Accuracy indicó qué tan bien predecía el modelo, también fueron importantes el resto de métricas para tener un panorama general del mismo, ya que en algunos casos se obtenía buen Accuracy pero no tan buenos resultados en los demás; el mejor modelo brindó 87,41% de Accuracy, 88,63% de Precision, 87,39% de Recall y 88% de F1-score. Con lo mencionado, se concluye que se cumplió el objetivo específico 5.

5.2 Recomendaciones

Si bien en esta investigación no se utilizó el url de cada registro como parte del modelado, se recomienda que para trabajos futuros sí se tenga en consideración, ya que este puede brindar información relevante acerca de fuentes fiables o engañosas, esto debido a que el link de algún registro perteneciente a una web confiable como lo es la OMS, siempre tendrá una estructura característica que se diferenciará de otras web no fiables, también sucede a la inversa, fuentes no fiables como TierraPura tienen ciertas tendencias de colocar links atractivos para los lectores. De este modo, se podrían utilizar los metadatos de estas urls para construir el modelo, sumado al contenido de la misma noticia, se lograría desarrollar clasificadores más robustos.

De igual manera, en el estudio solo se utilizaron técnicas de Machine Learning, ello debido a que este tipo de algoritmos se desempeñan mejor cuando no se tiene una dataset muy extenso; sin embargo, si se considerara trabajar con datasets en grandes cantidades que superasen los cientos de miles de datos, sería recomendable aplicar técnicas de Deep Learning, ya que estas trabajan de mejor manera con datasets enormes, lo cual permite construir clasificadores más precisos.

Es recomendable realizar el entrenamiento de los modelos embedding con la mayor cantidad de datos posibles relacionados al COVID-19, ya que brindan un contexto más amplio para las palabras, lo cual favorece posteriormente en la extracción de características. Asimismo, si se desea entrenar estos modelos en el idioma español, es crucial tener en cuenta el tipo de preprocesamiento a realizar, puesto que este idioma es uno de los que más jergas utiliza, sumado con la mala ortografía que comúnmente se encuentra en las publicaciones y comentarios, pueden representar un problema para los modelos.

Asimismo, se recomienda incluir mayor diversidad de registros en el dataset, ya que con el que se llevó a cabo la investigación estuvo conformado en gran parte por temas relacionados a la vacunación, esto quiere decir que, si bien el modelo puede aprender a diferenciar información falsa en el contexto del covid, este sería más propenso a realizarlo de mejor manera cuando se trate de información referente a las vacunas que en otros casos. Aún así, brinda un primer alcance para futuras investigaciones que quieran tomar de referencia este trabajo.

REFERENCIAS

Ahmed-Siddiqui, M., Mushtaq, K., Mohamed, M., Al-Soub, H., Hussein, M., y Yousaf, Z., (2020). *"Social media misinformation" - an epidemic within the COVID-19 pandemic.* Am J Trop Med Hyg 103: 920–921.

Álvarez-Risco, A., Mejía, C., Delgado-Zegarra, J., Del-Águila, A., Arce-Esquivel, A., Valladares-Garrido, M. (2020). *The Peru approach against the COVID-19 infodemic: Insights and strategies.* Am J Trop Med Hyg;103(2):583−6. doi: 10.4269/ajtmh.20-0536

Balarezo-López, G. (2021). *El virus de las "noticias falsas" en la pandemia del COVID-19*. Diagnóstico, *60*(1), 40-49.

Berry, M. W., & Kogan, J. (Eds.). (2010). *Text mining: applications and theory*. John Wiley & Sons.

Bird, S., Klein, E., y Loper, E. (2009). *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc.".

Bojanowski, P., Grave, E., Joulin, A., y Mikolov, T. (2017). *Enriching word vectors with subword information.* Transactions of the association for computational linguistics, 5, 135-146.

Bojjireddy, S., Chun, A., y Geller, J. (2021). *Machine Learning Approach to Detect Fake News, Misinformation in COVID-19 Pandemic.* In DG. O2021: The 22nd Annual International Conference on Digital Government Research (pp. 575-578).

Center for Preparedness and Response (2020). *Responding to Rumors and Misinformation. Emergency Preparedness and Response*. Atlanta, GA: Centre for Disease Control and Prevention.

Centro de Investigaciones Universidad del Pacífico (CIUP,2021). *48% de los peruanos que no se vacunarían contra el COVID-19 creen que faltan más pruebas a las vacunas.* https://ciup.up.edu.pe/analisis/48-de-peruanos-que-no-se-vacunarian-contra-el-covid-19-creen-que-faltan-mas-pruebas-a-las-vacunas/

Chaudhary, A. (2020). A Visual Guide to FastText Word Embeddings. https://amitness.com/2020/06/fasttext-embeddings/

COUNTWORDSFREE (2022). *Word Counter*. https://countwordsfree.com/

Covington, M. A., Grosz, B. J., y Pereira, F. C. (1994). *Natural language processing for Prolog programmers*. Upper Saddle River: Prentice hall.

Elhadad, M., Li, K., y Gebali, F. (2020). *Detecting misleading information on covid-19*. IEEE Access, 8, 165201-165215. doi: 10.1109/ACCESS.2020.3022867.

Federación Internacional de Periodistas (s.f.). *¿Qué son las fake news? Guía para combatir la desinformación en la era de posverdad.* https://www.ifj.org/fileadmin/user_upload/Fake_News_-_FIP_AmLat.pdf

García-Saisó, S., Marti, M., Brooks, I., Curioso, W., González, D., Malek, V. (2021). Infodemia en tiempos de COVID-19. Rev Panam Salud Publica. 45:e89. https://doi.org/10.26633/RPSP.2021.89

Geocode.xyz (2022). *South America.* https://geocode.xyz/?region=SouthAmerica

Gonzales, M. M. P., Cueva, M. A. L., y Tafur, R. D. C. A. (2020). *La red sanitaria y su participación en la difusión o contención de las fake news y bulos relacionados con la COVID-19: el caso de Lima-Perú.* Chasqui: Revista Latinoamericana de Comunicación, (145), 93-118.

Harrington, P. (2012). *Machine learning in action*. Simon and Schuster.

Haykin, S. (2009). *Neural networks and learning machines* (3ra Ed.). Pearson.

Heifin, R. (2020). *Machine Learning with R, the tidyverse and mlr*. Manning

Hernández, R., Fenández, C., y Baptista, P. (2010). *Metodología de la investigación (5ta. Ed.). Santa Fe: Mc GRAW HI.* https://www.icmujeres.gob.mx/wp-content/uploads/2020/05/Sampieri.Met.Inv.pdf

IPSOS (2021). *Redes sociales 2021*. https://www.ipsos.com/es-pe/redes-sociales-2021

Isaakidou, M., Zoulias, E., y Diomidous, M. (2021). *Machine Learning to Identify Fake News for COVID-19.* In Public Health and Informatics (pp. 108-112). IOS Press.

Islam, M. S., Sarkar, T., Khan, S. H., Kamal, A. H. M., Hasan, S. M., Kabir, A., y Seale, H. (2020). *COVID-19–related infodemic and its impact on public health: A global social media analysis.* The American journal of tropical medicine and hygiene, 103(4), 1621.

Kurzweil, R., Richter, R., Kurzweil, R., y Schneider, M. L. (1990). *The age of intelligent machines* (Vol. 580). Cambridge: MIT press. https://www.cs.toronto.edu/~vmnih/docs/dqn.pdf

Lawson, R. (2015). *Web scraping with Python*. Packt Publishing Ltd.

Lederer EM, (2020). *U.N. Chief Antonio Guterres: Misinformation about COVID-19 Is the New Enemy.* TIME. New York, NY: TIME.

Mazzeo, V., Rapisarda, A., y Giuffrida, G. (2021). *Detection of fake news on CoViD-19 on Web Search Engines.* arXiv preprint arXiv:2103.11804.

Medium (13 de setiembre de 2018). *Word2Vec for Talent Acquisition.* https://medium.com/@nuripurswani/word2vec-for-talent-acquisition-ab20a23e01d8

Mikolov, T., Chen, K., Corrado, G., y Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv preprint arXiv:1301.3781.

Ministerio de Salud (2021). *Información Pública: Datos Generales. https://www.minsa.gob.pe/transparencia/index.asp?op=103*

Ministerio de Salud (2021). *Organización de Ministerio de Salud*. https://www.gob.pe/7662-ministerio-de-salud-organizacion-de-ministerio-de-salud

Mitchell, T. (1997). *Machine Learning.* McGraw-Hill Science.

Nieves-Cuervo, G., Manrique-Hernández, E., Robledo-Colonia, A., y Grillo, A. (2021). *Infodemia: noticias falsas y tendencias de mortalidad por COVID-19 en seis países de América Latina. Revista Panamericana de Salud Pública,* 45, e44.

Noticias ONU (2020). *México y la ONU suman esfuerzos para combatir la desinformación a través de la campaña "Verified".* https://news.un.org/ es/story/2020/09/1480502

Nuri (2018, 13 septiembre). *Word2Vec for Talent Acquisition*. Medium. Recuperado 20 de mayo de 2022, de https://medium.com/@nuripurswani/word2vec-for-talent-acquisition-ab20a23e01d8

Organización Mudial de la Salud (2019). *Información básica sobre la COVID-19.* https://www.who.int/es/emergencies/diseases/novel-coronavirus-2019/question-and-answers-hub/q-a-detail/coronavirus-disease-covid-19

Organización Mudial de la Salud (2020). *Gestión de la infodemia sobre la COVID-19: Promover comportamientos saludables y mitigar los daños derivados de la información incorrecta y falsa.* https://www.who.int/es/news/item/23-09-2020-managing-the-covid-19-infodemic-

promoting-healthy-behaviours-and-mitigating-the-harm-from-misinformation-and-disinformation

Organización Panamericana de la Salud. (2020). Entender la infodemia y la desinformación en la lucha contra la Covid-19. *Hoja informativa*, (5). https://iris.paho.org/handle/10665.2/52053

Palomino, M., Lovón, M., y Arellanos, R. (2020). *La red sanitaria y su participación en la difusión o contención de las fake news y bulos relacionados con la COVID-19: el caso de Lima-Perú*. Chasqui (13901079), (145).

Peña Ascacíbar, G., Bermejo Malumbres, E., y Zanni, S. (2021). *Fact checking durante la COVID-19: análisis comparativo de la verificación de contenidos falsos en España e Italia*. Revista de Comunicación, 20(1), 197-215.

Pennington, J., Socher, R., y Manning, C. (2014, October). *Glove: Global vectors for word representation*. In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP) (pp. 1532-1543).

Runebook.dev (2020). *Validación cruzada:evaluación del rendimiento del estimador.* https://runebook.dev/es/docs/scikit_learn/modules/cross_validation

Russell, S., y Norvig, P. (2004). Inteligencia artificial. Un enfoque moderno. (2da Ed.). Pearson

Saiful, M., Tonmoy, S., Sazzad, H., Abu-Hena, M., Murshid S., Kabir, A., Yeasmin, D., Mohammad, A., y Holly, S. (2020). *COVID-19–Related Infodemic and Its Impact on Public Health: A Global Social Media Analysis.* 103(4), pp. 1621–1629

Staff Reporter (2020). *Twelve Taken Ill after Consuming 'Coronavirus Shaped' Datura Seeds*. The Hindu. Andhra Pradesh, India: The Hindu.

Wardle, C., y Derakhshan, H. (2017). *Information disorder: Toward an interdisciplinary framework for research and policymaking*. Council of Europe report

Witten, I., Frank, E., y Hall, M. (2011). Data Mining: Practical Machine Learning Tools and Techniques. (3ra Ed.). Morgan Kaufmann Publishers

ANEXOS

Anexo 1 Tabla de papers

Tipo	Nro.	Antecedente	Autor (es)	País	Año
Problema	1	Detecting misleading information on covid-19	Elhadad, M., Li, K., y Gebali, F.	Canadá	2020
	2	Machine Learning to Identify Fake News for COVID-19.	Isaakidou, M., Zoulias, E., y Diomidous, M.	Grecia	2021
Propuesta	3	Detection of fake news on CoViD-19 on Web Search Engines.	Mazzeo, V., Rapisarda, A., y Giuffrida, G.	Italia	2021
	4	ANTi-Vax: a novel Twitter dataset for COVID-19 vaccine misinformation detection.	Hayawi, K., Shahriar, S., Serhani, M., Taleb, I., y Mathew, S.	Emiratos Árabes	2022
Técnica o herramienta	5	Machine Learning Approach to Detect Fake News, Misinformation in COVID-19 Pandemic.	Bojjireddy, S., Chun, A., y Geller, J.	EE.UU.	2021
	6	Fake news detection in Arabic tweets during the COVID-19 pandemic.	Mahlous, A., y Al-Laith, A.	Arabia Saudita	2021
Sector o Escenario	7	CoAID: Covid-19 healthcare misinformation dataset.	Cui, L., y Lee, D.	EE.UU.	2020

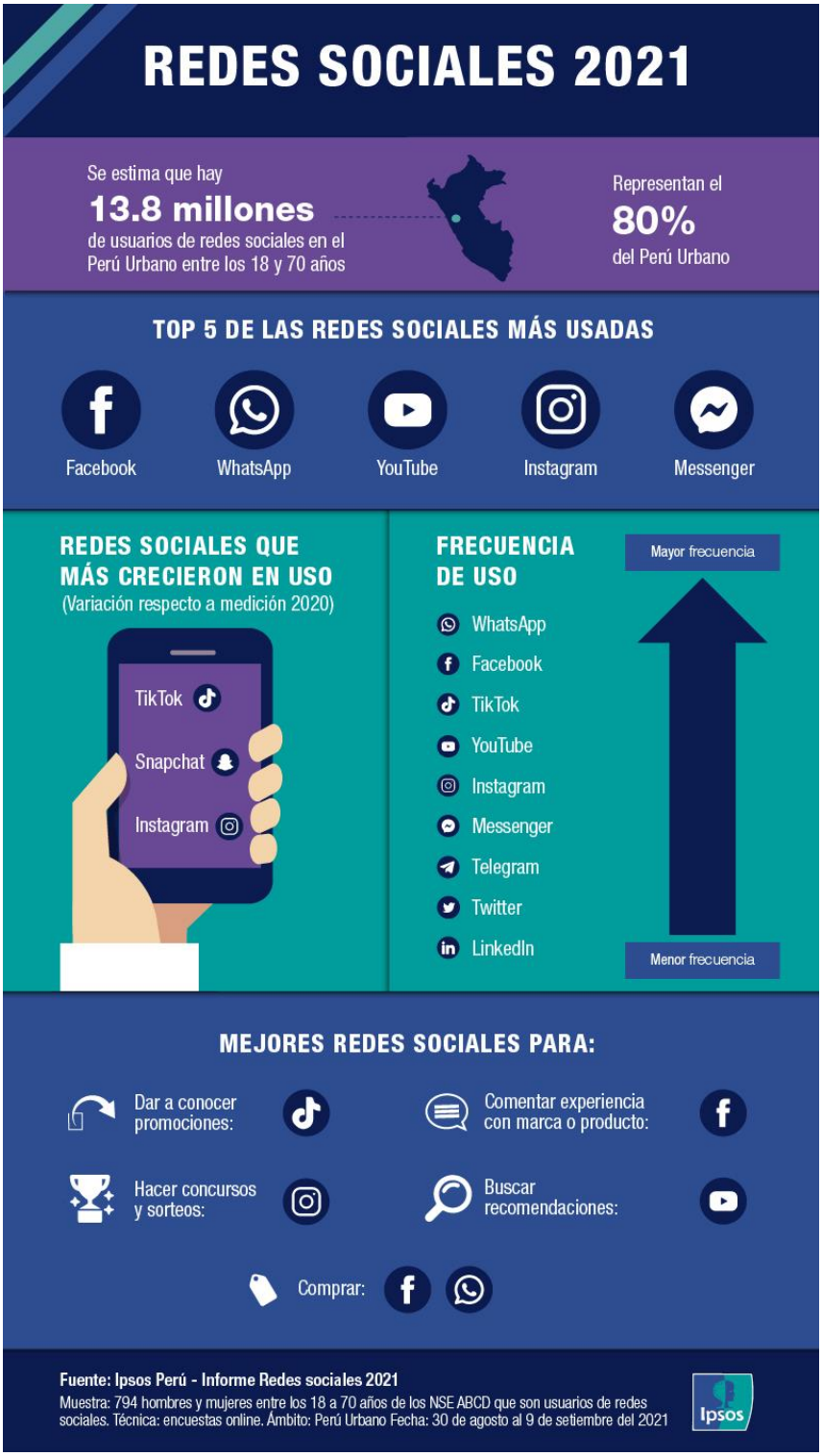
Anexo 2 Matriz de consistencia

Propuesta de modelo para la detección de información falsa del COVID-19 aplicando técnicas de Machine Learning y Natural Language Processing			
PROBLEMA GENERAL	**OBJETIVO GENERAL**	**HIPÓTESIS GENERAL**	**VARIABLES**
¿Es posible detectar información falsa del COVID-19 aplicando técnicas de Machine Learning y Natural Language Processing?	Desarrollar un modelo para la detección de información falsa del COVID-19 aplicando técnicas de Machine Learning y Natural Language Processing.	La información falsa del COVID-19 que circula en la web peruana puede ser detectada mediante técnicas de Machine Learning y Natural Language Processing.	Variable Independiente: **Machine Learning** Según Mitchell (1997), “Se ocupa de la cuestión de cómo construir programas informáticos que mejoren automáticamente con la experiencia." **Natural Language Processing** Según Covington (2013): "Es el uso de las computadoras para comprender el lenguaje humano (natural) como el inglés, el francés o el japonés".

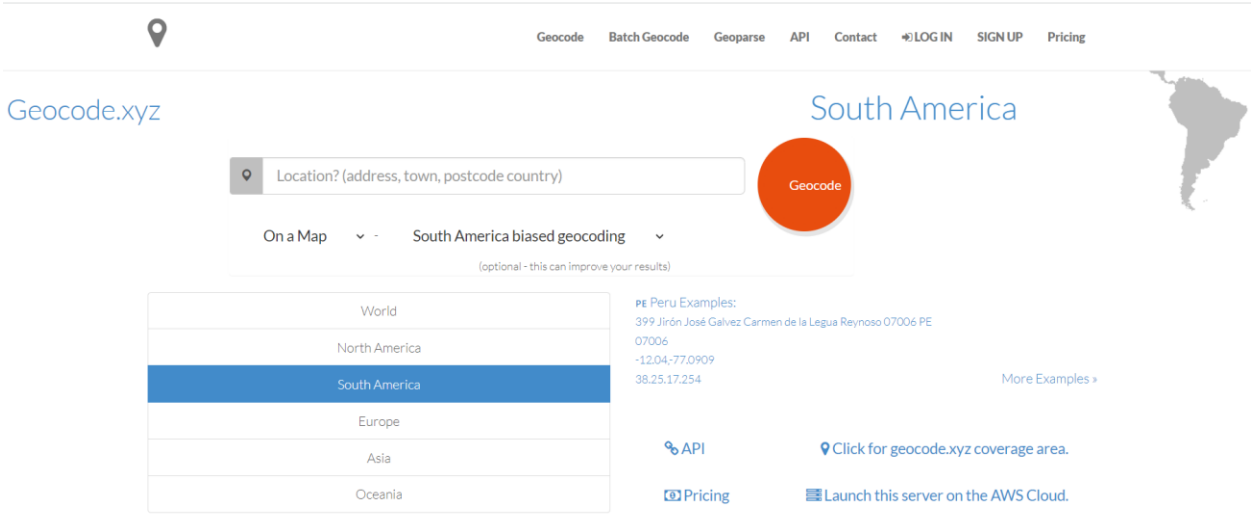
PROBLEMAS ESPECÍFICOS	**OBJETIVOS ESPECÍFICOS**	**HIPÓTESIS ESPECÍFICAS**	
¿Existe suficiente información del COVID-19 en la web peruana para la creación del dataset?	Crear un dataset de información verdadera y falsa del COVID-19 en el contexto peruano.	La recolección de suficiente información del COVID-19 en la web peruana permite la construcción del dataset.	Variable Dependiente: **Información falsa** Combinación de tres nociones importantes: misinformation, disinformation and malinformation, esta información puede ser creada, producida o distribuida por "agentes" con la intención de hacer daño, como también, por "agentes" sin dicha intensión. (Wardle y Derakhshan, 2017)
¿Realizar el preprocesamiento permite un mejor entrenamiento del modelo?	Desarrollar el preprocesamiento para cada documento del dataset.	Realizar el preprocesamiento influye significativamente en un mejor entrenamiento del modelo.	
¿Qué técnicas de Natural Language Processing podemos utilizar para una adecuada vectorización de características?	Utilizar técnicas de Natural Language Processing que permitan obtener vectores característicos de cada documento.	La implementación de técnicas de Natural Language Processing favorece en la extracción de características de cada documento.	
¿Qué técnicas de clasificación de Machine Learning podemos aplicar para construir un modelo de clasificación robusto de	Utilizar técnicas de clasificación de Machine Learning para construir un modelo de clasificación robusto de información falsa del COVID-19.	La implementación de técnicas de clasificación de Machine Learning permite construir un modelo de clasificación robusto	

información falsa del COVID-19?		de información falsa del COVID-19.
¿Qué métricas existen para evaluar el desempeño del modelo de clasificación de información falsa del COVID-19?	Utilizar métricas para evaluar el desempeño del modelo de clasificación de información falsa del COVID-19.	La utilización de métricas influye significativamente en la evaluación del desempeño del modelo de información falsa del COVID-19.

Anexo 3 Tendencia del uso de redes sociales en el 2021



Anexo 4 Página Geocode.xyz

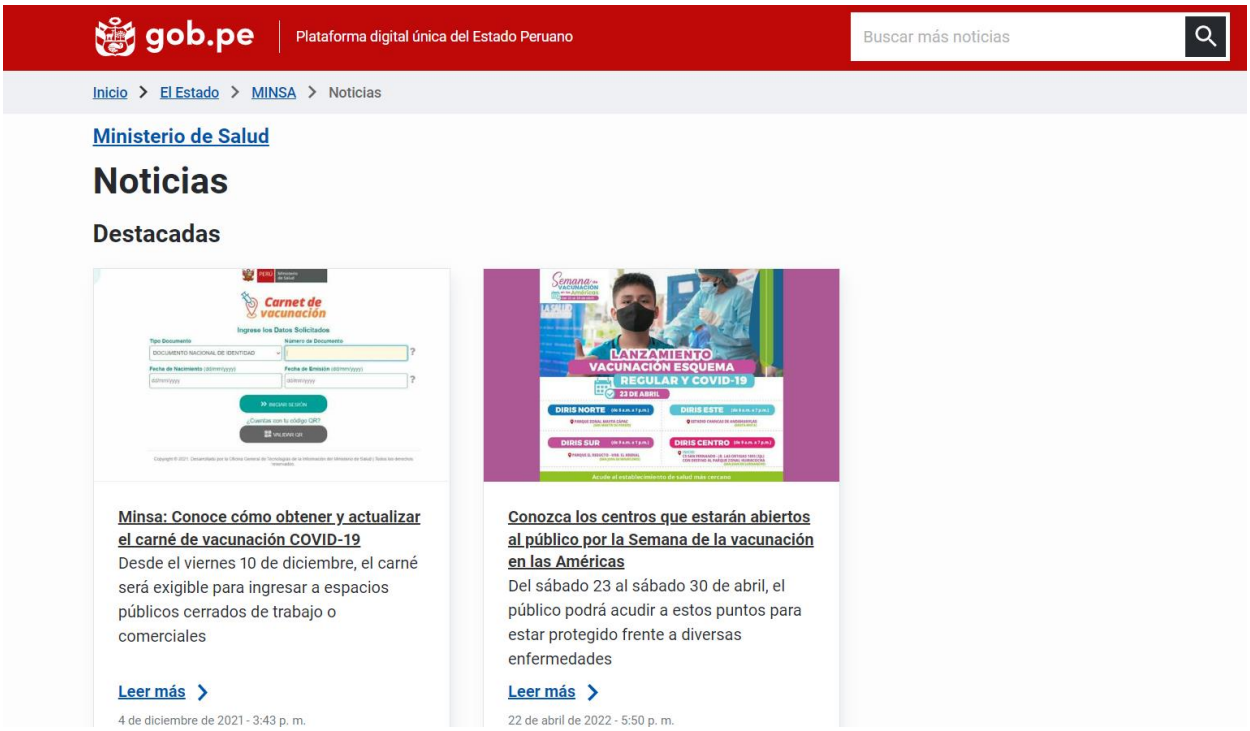


Anexo 5 Tiempo de espera por requests excedidos

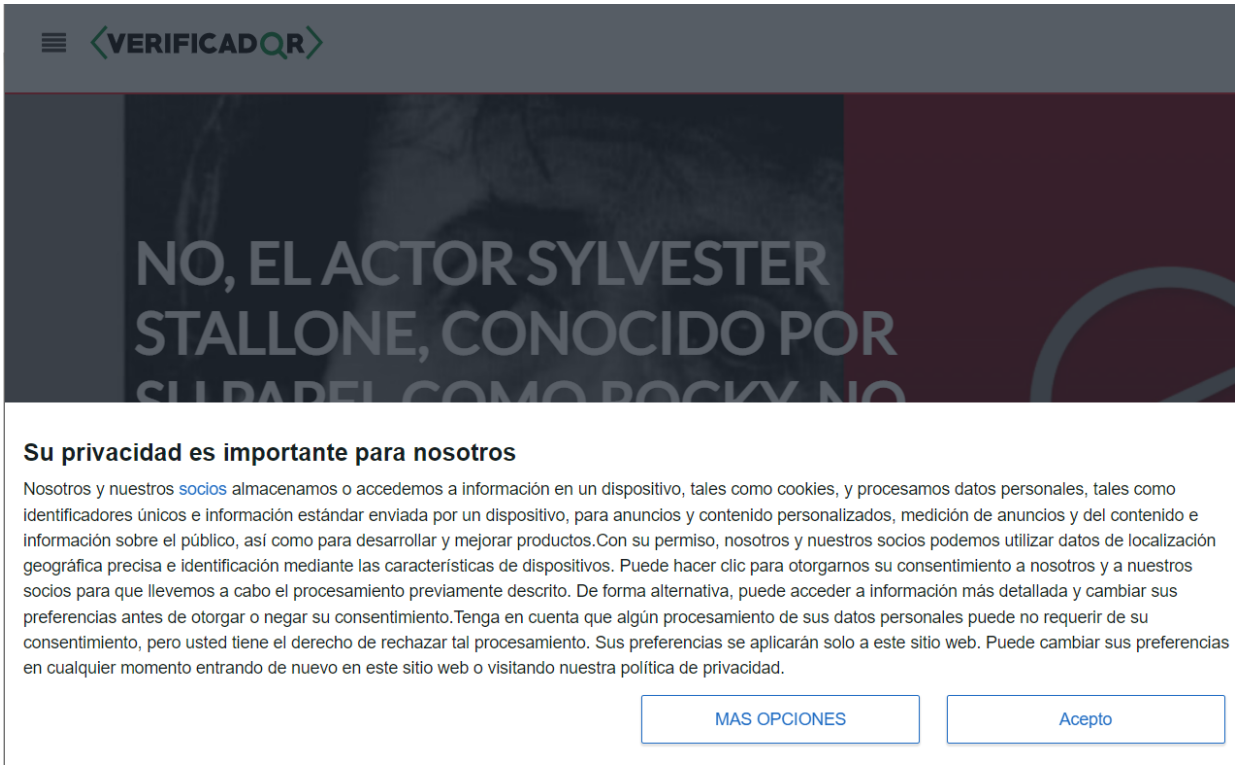
```
searchString = "vacuna"
tweets22 = []
for tweet in tweepy.Cursor(api.search, q=searchString, lang="es",tweet_mode='extended',geocode="-12.72465,-71.31966,1500km").items(2500):
    tweets22.append(tweet._json)
len(tweets22)
```

```
Rate limit reached. Sleeping for: 766
```

Anexo 6 Noticias destacadas del MINSA



Anexo 7 Mensaje de privacidad de Verificador LR



Anexo 8 Mensaje de privacidad de Ojo Público



Anexo 9 Mensaje emergente de Ojo Público



Anexo 10 Declaración de validado del etiquetado

DECLARACIÓN DE VALIDACIÓN DEL ETIQUETADO DEL DATASET

Yo **Margarita Arias Vargas** con el cargo de **Coordinadora de Comunicaciones en el Ministerio de Salud (MINSA)**, declaro haber validado el correcto etiquetado de los registros de COVID-19 del estudiante **Yair Andrey Salinas Bolaños** para el trabajo de Tesis 2 titulado "Propuesta de modelo para la detección de información falsa del COVID-19 mediante técnicas de Machine Learning y Natural Language Processing".

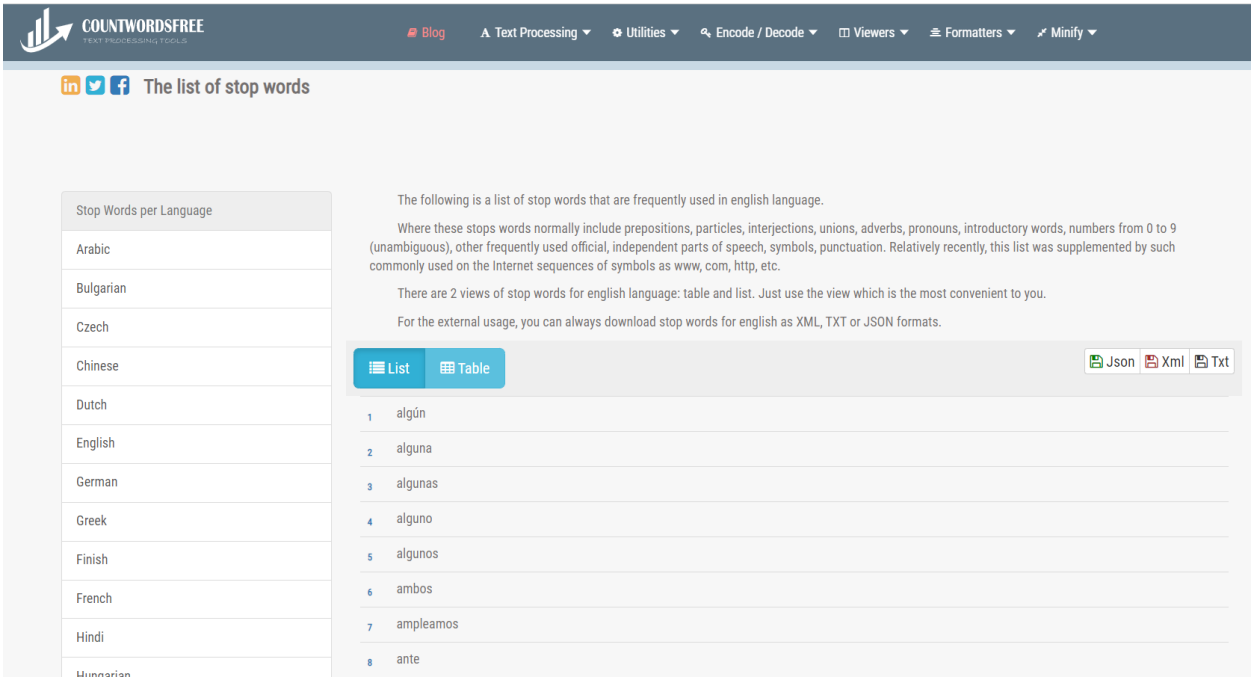
Lima, 30 de junio del 2022

Lic. M. Arias.

Firma

DNI: 10225624

Anexo 11 Web de COUNTWORDSFREE



Anexo 12 Listado de caracteres ASCII imprimibles

Caracteres ASCII imprimibles					
32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	|
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		